Learning to Select Actions for Complex Hand Tasks: A Comparative Study of Visuomotor Policies and Latent World Models

Graham Pellegrini

Supervisor: Prof. Alexiei Dingli

[PLACEHOLDER: Submission Date]

Submitted in partial fulfilment of the requirements for the degree of Master of Science in Artificial Intelligence.

L-Università ta' Malta
Faculty of Information & Communication Technology

Plagiarism Declaration

I, the undersigned, hereby declare that the work contained in this dissertation is my own original work and that I have not previously in its entirety or in part submitted it at any university for a degree.

I declare that this submission is my own work and that, to the best of my knowledge and belief, it contains no material previously published or written by another person, except where due acknowledgement is made in the text.

I declare that the use of artificial intelligence tools in the preparation of this dissertation was limited to writing assistance, code suggestions, and literature search support. All research design, experimental decisions, implementation choices, result interpretation, and academic claims are my own.

Signature: ______________________

Name: Graham Pellegrini

Date: ______________________

Abstract

This dissertation focuses on latent world models, a recent research area that has been gaining traction within the wider field of machine learning. A latent world model learns to predict how a scene will evolve in a compact learned representation rather than in raw pixels, and its promise for robotics lies in judging a proposed action by its predicted consequences before that action is executed. The guiding idea behind this work casts the imitation policy as an actor and the world model as the brain that evaluates the actor's proposed motions, so that the pair can select better and safer actions leading to more plausible end states.

The research question is whether this pairing improves the selection of future bimanual hand motions under a controlled offline benchmark. The study deliberately favours quality of evidence over quantity of tests. The benchmark evaluates 7,607 held-out windows from the EgoDex egocentric manipulation dataset, where each window pairs 16 observed frames with a 16-frame prediction target and actions are represented as 18-dimensional bimanual wrist-motion chunks. Policy and world model are interfaced through three selection branches: the policy alone, world-model scoring with the policy's preferred candidate retained, and world-model scoring with that candidate excluded. All models are trained under a matched fair-training budget, and one pairing emerged as the clean, fully controlled positive result. Diffusion Policy candidates scored by V-JEPA2-AC reduce first-step translation error by 6.62 per cent and rotation error by 1.08 degrees relative to the policy alone. This improvement arrives without any retraining, since the world model simply re-ranks the five motion proposals the policy already produces at inference time. The same constraint sets a natural ceiling on what any selector could achieve: a perfect selector with access to ground truth would reduce translation error by at most 16.8 per cent on the same candidate pool, so the world model recovers roughly forty per cent of the attainable headroom. Every other implemented route (ACT, DexWM, LeWM, DINO-WM, and JEPA-WM) returns a negative or bounded outcome, and each is followed far enough to trace its shortfall to a concrete caveat such as candidate collapse or an unavailable pretrained checkpoint.

Beyond the headline numbers, the dissertation contributes a reproducible protocol for separating genuine world-model guidance from misleading shortcuts, together with a documented account of the process that exposes further areas of future research. All conclusions concern offline wrist-trajectory prediction on recorded demonstrations.

Acknowledgements

Placeholder for acknowledgements.

Contents

List of Figures

List of Tables

List of Abbreviations

ACT Action Chunking Transformer.

ALOHA A Low-cost Open-source Hardware System for Bimanual Teleoperation.

BC Behaviour Cloning.

CDiT Conditional Diffusion Transformer.

CEMP Cross-Entropy Method with Policy.

CNN Convolutional Neural Network.

CVAE Conditional Variational Autoencoder.

DDIM Denoising Diffusion Implicit Models.

DDPM Denoising Diffusion Probabilistic Models.

DexWM Dexterous Manipulation World Model.

DiT Diffusion Transformer.

DP Diffusion Policy.

HDF5 Hierarchical Data Format 5.

I-JEPA Image Joint Embedding Predictive Architecture.

JEPA Joint Embedding Predictive Architecture.

LeWM LeWorldModel.

RMSE Root Mean Square Error.

SSL Self-Supervised Learning.

V-JEPA2 Video Joint Embedding Predictive Architecture 2.

V-JEPA2-AC Action-Conditioned Video Joint Embedding Predictive Architecture 2.

VAE Variational Autoencoder.

ViT Vision Transformer.

VLA Vision Language Action Model.

VLM Vision Language Model.

List of Notations

I Identity matrix; gives the isotropic unit Gaussian covariance.

$\mathcal{N}(0, I)$ Standard isotropic Gaussian prior.

ε Reparameterisation noise, $\varepsilon \sim \mathcal{N}(0, I)$.

μ, σ Encoder-predicted mean and standard deviation.

z CVAE latent variable sampled from the prior $\mathcal{N}(0, I)$.

$\bar{\alpha}_t$ Cumulative noise-schedule product at diffusion timestep t.

ε_θ Noise-prediction network parameterised by θ.

t_{prev} Next step index in a subsampled inference schedule.

$\hat{x}_0$ Predicted clean sample in a DDIM reverse step.

x_T Noise sample at the diffusion schedule endpoint T.

B_{global} Effective global batch size; equals $B \times G \times A$ where B is the per-device batch size, G the gradient-accumulation steps, and A the number of accelerator devices.

E Effective training exposure, measured as total training items consumed, equal to optimisation steps multiplied by effective global batch size.

K Number of sampled candidate action chunks scored for one observation window.

N Number of evaluated samples or windows, depending on the metric context; pairwise comparisons report the number of held-out windows or candidate pairs explicitly.

S Number of optimisation steps.

$\mathbf{a}_{t:t+15}$ Sixteen-step future action chunk predicted from sixteen observed frames; each step has 18 action dimensions.

a_t Action vector at timestep t; in this thesis each wrist-action step is an 18-dimensional bimanual vector, with left and right hands each represented by 3 translation and 6 rotation coordinates.

z_t Latent embedding of observation frame or video state at timestep t; dimensionality depends on the encoder family.

1 Introduction

Machine learning offers two broad routes to acquiring behaviour in robotics. Reinforcement learning improves a policy through trial and error against a reward signal, while imitation learning acquires behaviour directly from demonstrations [1, 2]. This dissertation works primarily in the imitation setting, which is attractive for manipulation because useful behaviour often depends on subtle motion patterns that are difficult to encode by hand, and demonstrations capture those patterns without requiring a hand-designed reward. Behaviour cloning provides one direct route by learning a mapping from observations to demonstrated actions, while inverse reinforcement learning approaches the same problem by trying to infer the objectives that make a demonstration sensible [2, 3]. Both perspectives frame learning as extraction from observed behaviour, but neither resolves how a system should judge whether a predicted future motion is physically plausible. Reward-driven adaptation returns later in the dissertation, when closely related work that fine-tunes policies inside learned world models is examined.

This issue becomes sharper in dexterous manipulation. Human hand motion is temporally structured, object dependent, and sensitive to small changes in pose. This follows from the timing of natural prehension, the visuomotor transformation required for grasping objects, and the postural synergies used to shape the hand for tools and objects [4–6]. In imitation settings, this matters because small prediction errors can compound across a future sequence, so local agreement with a demonstration does not guarantee sequence-level alignment [7]. This thesis is therefore concerned with prediction and evaluation of future hand motion, rather than deployed control. The work remains relevant to robotic control because robotic manipulation systems ultimately require reliable motion selection, but the evidence in this dissertation is restricted to an offline computational benchmark.

Visuomotor policies are a common way to connect visual observations with future motion. This dissertation studies two policy families that span the main design axes of current practice: the ACT, which regresses temporally extended action chunks in a single deterministic pass [8], and DP, which samples action chunks from a learned generative process [9]. Both share the chunked formulation, which suits hand motion because it unfolds over short sequences rather than through single-frame decisions. However, a policy that proposes future chunks still requires a criterion for choosing between candidate futures, and this is the opening for world models. World models learn predictive representations of how a state may evolve, so they can judge a candidate motion by the future it implies. In this benchmark the world model never generates motion of its own. Every candidate in the pool is proposed by the policy, and the world model acts only at the moment of selection. The comparison then varies how much weight the policy's own preference carries. One branch keeps the policy's preferred candidate available while the world model re-ranks the pool, and another removes that preferred candidate so the world model must rank the remaining proposals without any bias toward the policy's choice. This graded transfer of the final decision from policy to world model motivates the comparative structure of the thesis.

This dissertation studies that structure through a comparative benchmark framework for evaluating whether world-model support changes the quality of future hand-motion prediction. The work began as a broad comparison across several world-model architectures, and during implementation that plan exposed a second research problem: published world-model systems are not equally reproducible, action-compatible, or fair to compare under one offline benchmark. The study therefore goes beyond training models and reporting scores. Closely related systems are reproduced and examined, each candidate route is instrumented with diagnostic side experiments, and every shortfall is traced to a concrete cause such as an unavailable pretrained checkpoint or an incompatible action space. The final contribution is a controlled benchmark with one clean positive pairing, several boundary cases, and a fully documented account of what world-model adaptation requires in practice before it can support headline claims.

1.1 Motivation

The motivation for this work sits at the intersection of robotics and machine learning. Modern manipulation research increasingly depends on learned models that can process visual observations, predict future motion, and adapt to complex physical interaction. At the same time, the pace of open-source machine learning development can make it difficult to separate robust scientific progress from promising but weakly benchmarked demonstrations. This concern is especially important for industrially relevant settings, where unreliable claims about physical reasoning or manipulation capability can lead to poor engineering decisions.

The direction of the work follows a wider debate in the field. LeCun has argued that systems which generate actions without predicting the consequences of those actions cannot plan or act reliably. He directs this critique at the VLAs that currently dominate robot learning, describing the scaling of autoregressive architectures for physical intelligence as a dead end and agentic systems built on them as "intrinsically unsafe" [10, 11]. His proposed alternative places a learned world model at the centre of the agent, so that candidate actions are evaluated through their predicted future states before any action is taken. That position gained a concrete instance in DiWA, which fine-tunes a diffusion policy entirely inside a frozen world model's imagination using reinforcement learning [12]. Reading DiWA strengthened the motivation and sharpened the question: DiWA uses the world model as a training substrate that updates the policy, so a complementary study could ask whether a frozen world model can instead improve the action selected at inference time, without retraining the policy. That question came to form this work.

The work was also shaped by the RSO I activity and the University of Malta M.AI.ESTRO project, where the convergence of robotics, machine learning, and Industry 4.0 raised a practical need for clearer evaluation of emerging model families. This dissertation does not claim to deliver a deployment system. It uses that practical motivation to ask a narrower benchmark question about offline candidate selection on recorded hand-motion demonstrations.

1.2 Research Question and Scope

The central research question is:

> *Do latent world models improve the selection of future bimanual hand-motion chunks when they act as the final decision stage over candidates proposed by frozen visuomotor policies (ACT and DP), under a controlled offline benchmark?*

The phrase *final decision stage* marks the boundary of the claim. The world model is not integrated into the policy and never updates its weights, in contrast to adaptation methods such as DiWA where the world model serves as a training substrate for the policy itself [12]. Here every candidate motion is produced by a frozen policy, and the world model only chooses among those candidates.

This wording is a rescoping of the original proposal question. The proposal asked whether integrating a latent world model into an imitation learning policy would improve physical grounding and robustness, measured through latent agreement error and task success under perturbations. The final implementation does not evaluate deployed task success or perturbation robustness, and latent mean-squared error is used only as a training diagnostic for the world-model transition head rather than as the main thesis result. The finished thesis therefore frames the question around offline prediction quality: whether world model support selects future hand-motion chunks that are closer to demonstrated motion under a shared benchmark.

The benchmark focuses on future bimanual hand motion and transform state. In implementation terms, the main prediction surface is built around future left-hand and right-hand transform chunks, but the detailed representation is defined later in Chapter 4 and Chapter 5.

The scope is deliberately bounded. The thesis does not test closed-loop robotic deployment, real-world task completion, force interaction, contact stability, or general machine common sense. It also does not claim that any one world model family is universally superior. Instead, it asks whether selected world-model families improve an offline prediction benchmark when compared under a shared evaluation surface. The final evidence focuses on DP candidates scored by V-JEPA2-AC as the clean positive pairing. ACT, DexWM, LeWM, DINO-WM, JEPA-WM, PointWorld proxy controls, and a secondary DiWA/CALVIN reproduction and guided-selection study are used to define the boundaries around that result.

This scope also explains the structure of the comparison. The benchmark is organised around three branches:

- **Policy-only branch:** the baseline, where the policy's preferred candidate provides the selected future motion without world-model involvement.
- **Hybrid branch:** the policy proposes a pool of candidate chunks, and the world model re-ranks the pool while the policy's preferred candidate remains available as a fallback.
- **World-model-driven branch:** the policy's preferred candidate is excluded from selection, so the world model ranks the remaining policy-generated proposals without any bias toward the policy's own choice.

In every branch the candidate pool itself comes from the policy, and the branches differ only in how much authority the world model holds over the final choice. The purpose of this structure is to separate the value of the world model from the value of the underlying policy as far as possible within an offline benchmark.

1.3 Contributions

This thesis contributes a comparative benchmark framework for studying world-model support in bimanual hand-motion prediction. The framework fixes the policy candidates, separates training support from held-out test windows, and records which information a selector is allowed to consume. This provides a controlled basis for asking whether predictive future-state models improve selected hand-motion chunks under the same offline evaluation conditions.

The thesis also contributes an implemented evidence hierarchy. V-JEPA2-AC is the main positive semantic world-model scorer. LeWM action-prior scoring is a secondary action-regularity result. DexWM, DINO-WM, JEPA-WM, ACT, and the

PointWorld proxy expose limitations involving checkpoint availability, pretraining scale, action-space compatibility, candidate diversity, and evaluation fairness.

Finally, the thesis contributes a reproducibility account. Several model routes were implemented far enough to identify why they could not support headline claims, and these negative and mixed outcomes are reported as part of the benchmark contribution because they show what must be true before world-model guidance becomes valid scientific evidence.

1.4 Dissertation Structure

——To be updated at the end ———–

The remainder of the dissertation is organised as follows:

- Chapter 2 introduces the technical background required to follow the dissertation, covering transformers, self-supervised learning, variational autoencoders, rigid body pose, visuomotor policies, and world models at a foundational level.
- Chapter 3 positions the work within the relevant literature on imitation learning, visuomotor policies, world models, and offline evaluation.
- Chapter 4 defines the benchmark design, scope, branch structure, leakage controls, metrics, and reporting rules.
- Chapter 5 describes the implemented system, the model-specific routes, and the reproduction constraints encountered during the study.
- Chapter 6 presents the benchmark results.
- Chapter 7 interprets the findings and states the main limitations.
- Chapter 8 closes the dissertation and identifies future work.

2 Background

This chapter assumes familiarity with the basic mechanics of a neural network and introduces the concepts central to this work: the main learning paradigms, visuomotor policies, rigid-body pose in SE(3), latent representations, conditional VAEs, transformer attention, and self-supervised video prediction. These foundations support understanding of the specific architectures in Chapter 3 and underpin the candidate selection mechanism and world model evaluator design examined in Chapter 4. Together, they explain the thesis question: whether a latent world model, acting as the final decision stage, can improve the selection of bimanual action chunks proposed by frozen visuomotor policies before any action is executed.

2.1 Learning paradigms

Machine learning methods can be grouped by how they receive feedback during training, as illustrated in Figure 2.1. Supervised learning trains on data with explicit labels: the correct answer is provided for every input, and the model learns to reproduce it. Unsupervised learning receives no labels at all and instead recovers structure, such as clusters or compact descriptions, from the data alone. Reinforcement learning replaces labels with a reward signal: the model acts in an environment and learns which state-action sequences lead to good outcomes.

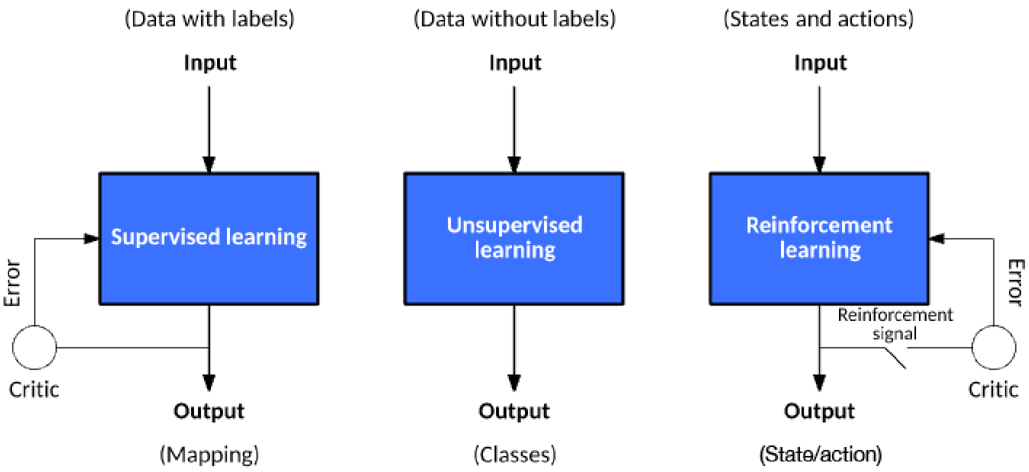

Figure 2.1 The three classical machine learning paradigms: supervised learning trains on labelled data, unsupervised learning extracts structure from unlabelled data, and reinforcement learning learns from a reward signal over states and actions. SSL, central to the world models in this work, is a modern form of learning from unlabelled data in which prediction targets are constructed from the data itself [13].

Reinforcement learning is defined by the interaction loop between an agent and an environment. At each step the agent observes a state, selects an action, and receives a scalar reward, and the objective is a policy that maximises the expected cumulative reward over time [1]. Learning therefore proceeds by trial and error rather than by copying labelled examples. This allows an agent to improve beyond the behaviour it started with, but it requires an environment, real or simulated, in which the consequences of actions can be observed. The requirement matters later in the dissertation: the offline benchmark studied here provides no such environment, while closely related work discussed in Section 3.2.1 fine-tunes a policy with reinforcement learning inside a learned world model instead of the real world.

SSL sits between the supervised and unsupervised settings. It trains on unlabelled data by constructing prediction tasks from the data itself: part of the input is hidden, and the model learns to predict it from the remainder [14]. Behaviour Cloning (BC), introduced in the next section, is a direct example of the supervised setting in which the expert demonstration acts as the label, while the world models studied in this dissertation are trained in the self-supervised setting. The close of this chapter develops that idea for video.

2.2 Visuomotor policies

A visuomotor policy is a learned function that takes a visual observation as input and produces an action or sequence of future actions as output. In a manipulation setting, the visual observation is typically a single camera frame showing the hand and the object being manipulated, and the action describes how the hand should move next. The policy is called visuomotor because it connects vision directly to motor output, without requiring a hand-crafted intermediate representation of what the scene contains [2].

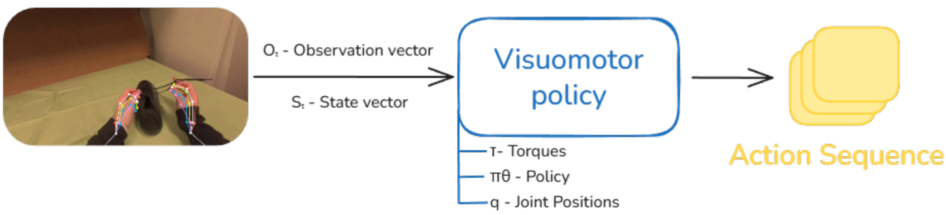

Figure 2.2 A visuomotor policy maps an EgoDex-style camera frame to future bimanual wrist actions in the thesis setting.

Learning this mapping from data is appealing because useful hand behaviour depends on subtle visual cues that are difficult to specify by hand. The timing of a reaching movement, the grip shape adopted before contact, and the postural adjustments made in response to object geometry all depend on information visible in the camera image but difficult to encode as explicit rules [4–6]. A visuomotor policy learns these associations directly from recorded demonstrations, capturing the relationship between what the camera sees and what the hand does without requiring the programmer to describe it explicitly.

2.2.1 Behaviour cloning and compounding error

The most direct way to train a visuomotor policy from demonstrations is BC [2]. Here an observed frame predicts the action taken by the expert at that moment, and the policy is trained to minimise the difference between its predicted action and the recorded expert action across all frames in the dataset [7]. The problem arises when a small prediction error causes the hand to move to a state that is slightly different from any state in the training data. This results in a compounding error: as the policy encounters increasingly unfamiliar states, its prediction errors grow larger over time.

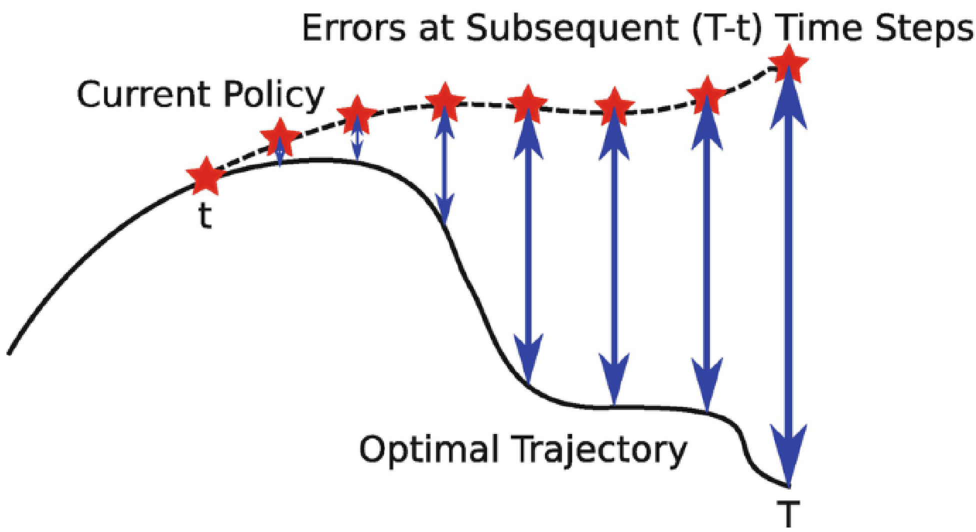

Figure 2.3 Compounding error in behaviour cloning. The policy trajectory drifts progressively further away at each step. The growing gap between the two paths illustrates how small errors can accumulate over time [15].

2.2.2 Action chunking

Action chunking addresses the compounding error problem by changing what the policy predicts. Instead of predicting a single next action at each step, the policy predicts a short sequence of future actions in one forward pass. This sequence is called an action chunk [8, 9]. In this benchmark, the policy observes 16 frames and predicts a 16-frame future action chunk. Each predicted step is an 18-dimensional vector: two hands, each with a six-dimensional rotation representation and a three-dimensional translation. This is why Figure 2.2 shows the action sequence as a stack of multiple action blocks. By committing to a chunk of k future steps, where k is a hyperparameter set per task, the policy makes one prediction decision every k frames rather than one per frame. With fewer decision points, there are fewer opportunities for errors to accumulate, and the hand stays closer to the training distribution for longer [12].

2.3 Rigid body pose and the $\mathrm{SE}(3)$ representation

To predict how a hand will move, a system needs a precise way to describe where the hand is and which way it is pointing. These two quantities together are called the pose of a rigid body. For the human wrist, pose is fully described by its position in space and its orientation relative to a fixed reference frame [16].

Translation. Position is represented as a vector of three numbers, one for each spatial dimension: how far the wrist is to the right or left, how far forward or backward, and how far up or down [4]. This vector $\mathbf{t} = [t_x,\ t_y,\ t_z]^\top$ is straightforward to predict with no special constraints on the output.

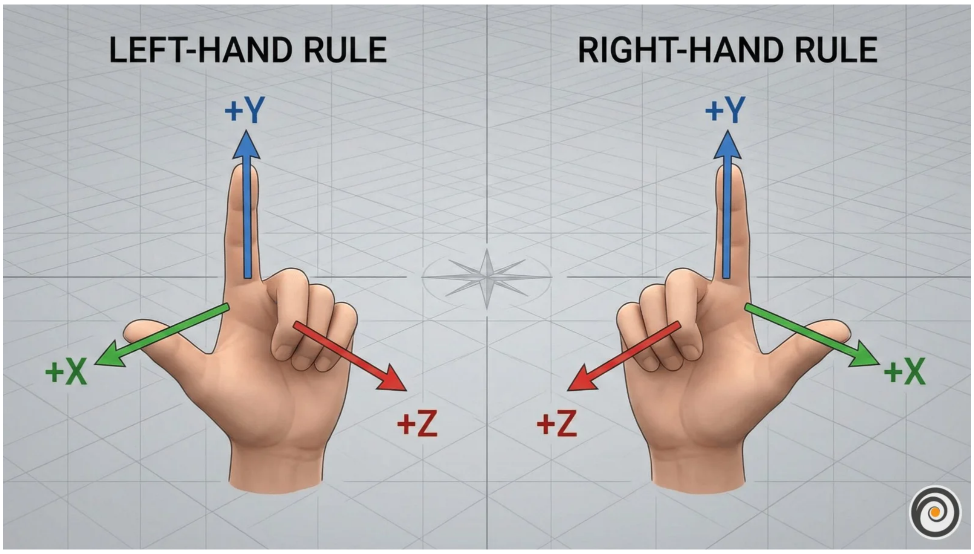

Figure 2.4 Translation for each arm can be represented by a movement along each of the three spatial axis [17].

Rotation. Orientation requires more care. A common representation uses three angles known as Euler angles, one per axis. These are intuitive but problematic for learning. At certain configurations the axes align and the system loses a degree of freedom, a singularity called gimbal lock. A subtler problem is discontinuity: a wrist rotated 359 degrees and one rotated 1 degree are almost physically identical, yet their angle values are numerically far apart. A network minimising prediction error will produce large errors near these boundaries even when the true motion is smooth [18].

A rotation matrix R avoids both issues: small physical rotations produce small numerical changes in the matrix entries, and there are no singularities.

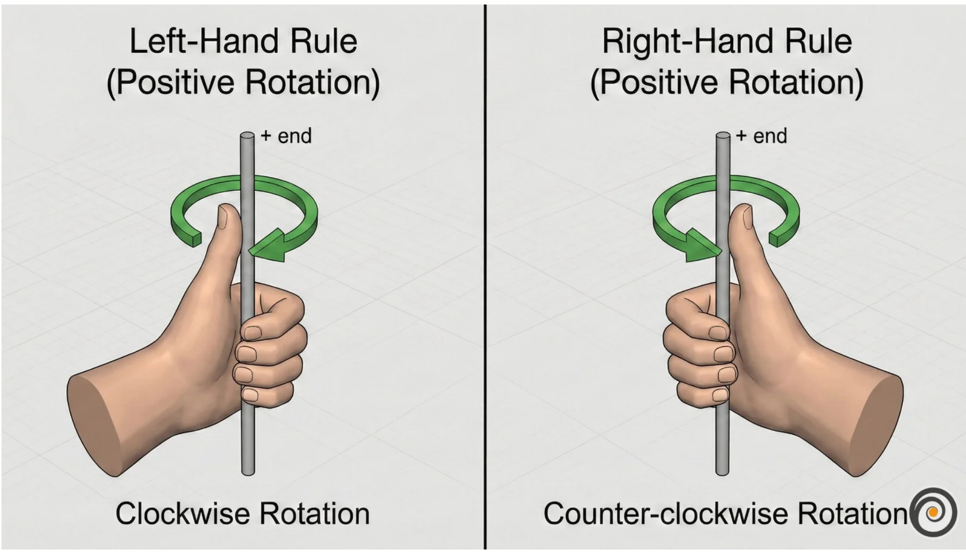

Figure 2.5 Rotation around the y-axis for each arm. The same circular movement can be imagined around the x and z axes; the total orientation of the wrist is the combined result of rotations around all three [17].

Homogeneous transform. Translation and rotation are combined into a single 4×4 matrix, the standard representation of a pose in the special Euclidean group SE(3):

$$T = \begin{bmatrix} R & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix} \tag{2.1}$$

The benchmark does not predict the full human body. It predicts left- and right-wrist motion. For each future step, the benchmark stores one rotation and one translation per wrist, giving the 18-dimensional bimanual action vector used by the policy and world-model scorers.

2.4 Latent representations

A latent representation is a learned internal description of an observation. An image starts as a grid of pixel values, where each pixel stores three or four channel intensities (typically red, green, and blue, with an optional transparency value). A model does not usually reason with these raw numbers directly. Instead, it transforms them through several layers into a new set of values that no longer correspond to individual pixel coordinates or colours. These new values form a compact encoding that captures patterns in the image such as shape, position, motion, and spatial structure. The collection of these encodings is often called the latent space. Figure 2.6 shows an example in which similar items cluster together in the learned representation.

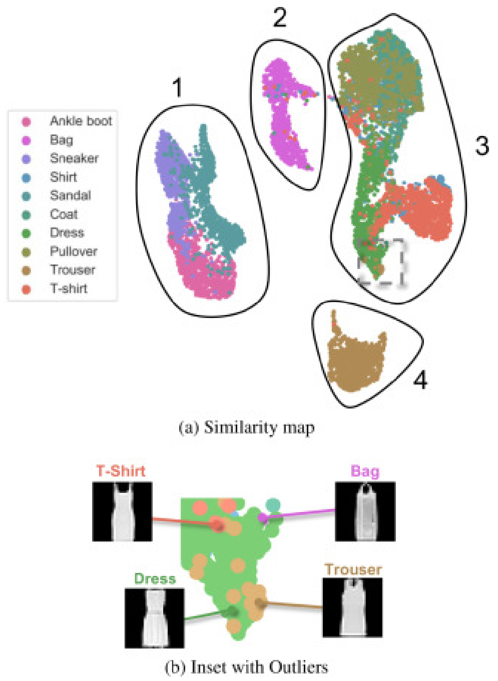

Figure 2.6 Example latent-space plot showing how structurally similar items cluster together in the learned representation. Structural similarity here refers to items that share shape, geometry, and likely physical behaviour [19].

The purpose of this transformation is to keep task-relevant structure while discarding unnecessary visual detail. For a manipulation scene, a useful latent state might encode hand position, object pose, contact progression, or motion trend without preserving every texture or lighting variation. A latent state is therefore not directly interpretable in the way an image is, but it can still preserve the information needed to reason about what happens next.

This is why latent prediction is often a better fit than pixel-level prediction for high-dimensional video observations. Predicting the next frame at the pixel level requires a model to account for every visual detail of the scene, including lighting changes, texture variation, and background movement that carry no information about task-relevant dynamics. A latent predictor instead operates on compact embeddings and concentrates predictive capacity on the structure that determines what actually happens next [20]. This design trade-off motivates the world model architectures reviewed in Chapter 3.

2.5 Conditional variational autoencoders

A VAE maps an input to a distribution of latent vectors described by a mean μ and standard deviation σ, rather than a single point. Sampling different latent vectors lets the decoder produce different but plausible outputs rather than one fixed answer. Figure 2.7a shows this standard encoder-latent-decoder flow [21].

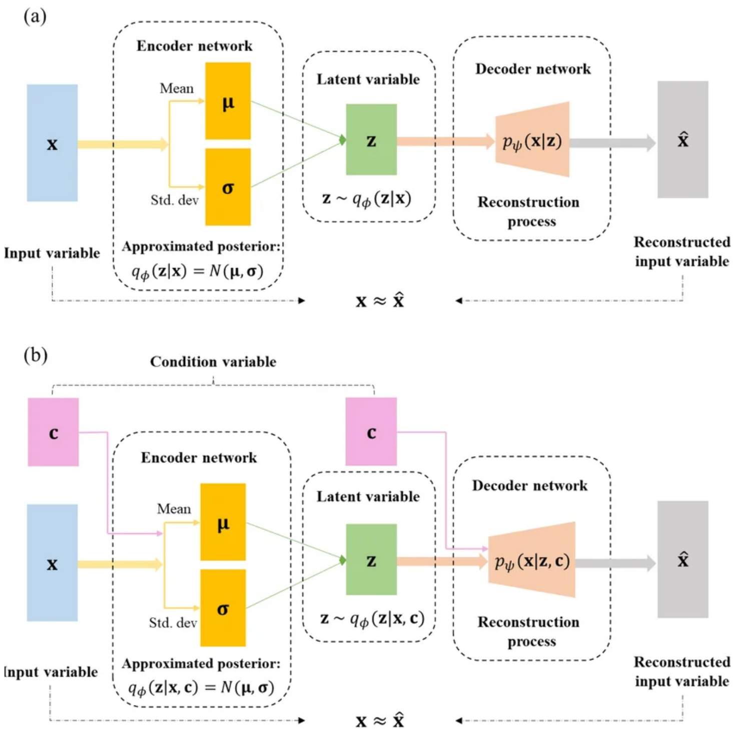

Figure 2.7 Standard VAE (a) and conditional CVAE (b) encoder-decoder pipelines [22].

For the policy to be trainable end-to-end, gradients must flow from the action prediction loss all the way back to the encoder. Sampling z directly from a distribution blocks this path. The reparameterisation trick resolves this by writing $z = \mu + \sigma\epsilon$, where ϵ is standard normal noise drawn independently of the learned parameters. Figure 2.8 shows the full encoder-latent-decoder pipeline alongside its training loss: a reconstruction term that rewards accurate output and a regularisation term that keeps the latent distribution close to a standard Gaussian, together forming the evidence lower bound [21]:

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}[\log p_\theta(\mathbf{x} \mid \mathbf{z})] - D_{\text{KL}}(q_\phi(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z}))\,. \tag{2.2}$$

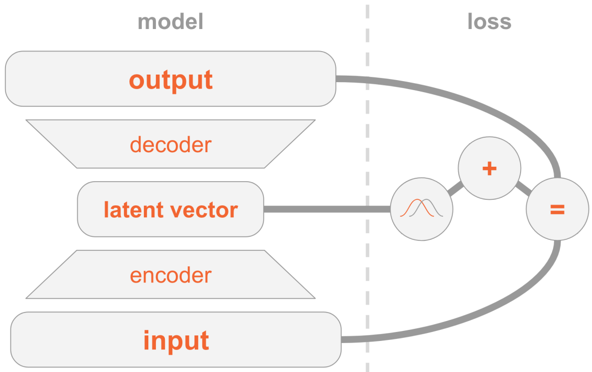

Figure 2.8 VAE encoder-decoder architecture and two-part training loss (reconstruction and regularisation) [21].

A CVAE adds an extra conditioning input to both encoder and decoder, shown in Figure 2.7b. In a visuomotor policy, this condition is the current observation frame, so the latent variable only needs to capture how the future action varies within that scene [22]. The ACT architecture uses this to sample multiple latent vectors from one observation and decode each into a candidate action chunk [8].

2.6 Transformer networks and the attention mechanism

A transformer processes a sequence as a set of tokens. A token can represent a word, an image patch, a timestep in an action sequence, or any vector produced by an earlier model component. The architectural idea needed here is simple: each token is updated by looking across the other tokens and deciding which of them are most relevant. This relevance-weighted exchange of information is called attention [23].

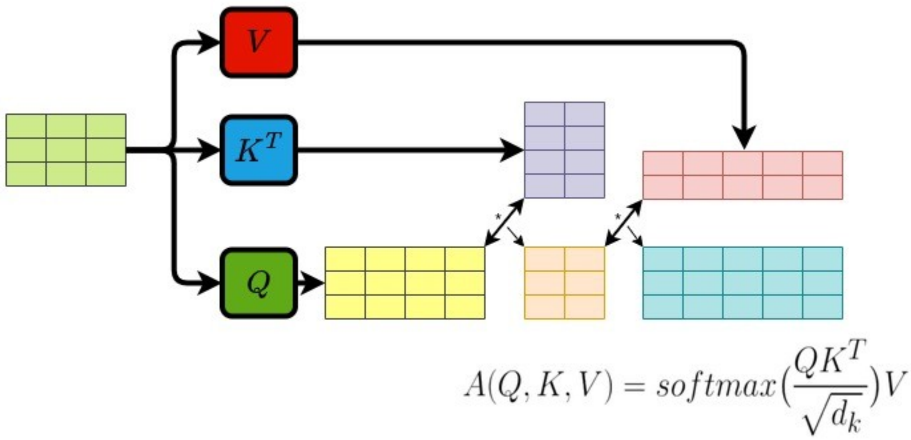

Figure 2.9 Transformer attention block: each token queries all others with Q, gathers weighted values V via keys K, and the weighted sum yields the updated output token [23].

Attention forms three learned views of every token:

- **Query** (Q) – what this token is looking for.
- **Key** (K) – what another token contains.
- **Value** (V) – what another token contributes if attended to.

Figure 2.9 shows how each token (highlighted) sends its query outward to every other token, receives weighted values in return, and produces an updated output. The bidirectional arrows indicate that every token attends to every other token simultaneously, and the crossing arrow beneath represents the weighted sum that combines those values into the final output. For each token, the transformer compares its query with the keys of all tokens. The resulting scores become weights, and those weights are used to combine the corresponding values. This operation is usually written as [23]:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V. \tag{2.3}$$

Transformers usually repeat this operation in several heads and add positional information so the model knows where each token sits in the sequence. Multiple heads allow different relationships to be represented at the same time, while positional information preserves order [23].

These concepts meet in self-supervised video prediction, the training signal behind the world models in this work. Given a sequence of observed frames, one or more future frames are hidden and the model learns to predict them from the frames that came before; the recording itself provides the target, so no annotation is needed. Predicting raw pixels runs into the ambiguity problem described in Section 2.4, where the model averages over possible futures and produces a blurred image. Predicting in latent space avoids this: the model predicts the compact embedding of the future frame, discarding background motion and fine texture so that capacity is spent on the structure that determines what happens next [20].

When the predictor is also given the action the agent intends to take, the result is a world model: a learned function that predicts how the scene embedding will change in response to a proposed action. If a policy proposes five wrist-motion chunks, a latent world model can predict the future embedding of each and favour the candidate whose predicted future is most plausible under its learned dynamics. This ties the chapter together: visuomotor policies produce action chunks, SE(3) describes the wrist motion being predicted, latent representations provide the scoring space, and self-supervised prediction supplies the world-model training signal. The architectures built on these foundations are reviewed in Chapter 3.

3 Literature Review

This chapter reviews the literature that leads directly to the central research question set out in Section 1.2: whether a world-model evaluator can improve action-chunk selection for bimanual EgoDex wrist-action prediction, and whether any such improvement holds once the underlying policy family changes. The reviewed work therefore has two roles: first, to justify ACT and Diffusion Policy as the candidate policy families; and second, to motivate latent, action-conditioned world models as candidate selectors. Whether an offline benchmark of this kind can be trusted at all is a question the surveyed literature also bears on directly, and it is picked up again once results are presented.

The review is also selective about evidential role. Some papers motivate the final positive test, especially Diffusion Policy and V-JEPA2-AC; others explain why negative or exploratory routes are still scientifically useful, especially DexWM, LeWorldModel (LeWM), DINO-WM, and JEPA-WM. DiWA is treated as the closest adjacent policy-improvement study because it uses a world model to improve Diffusion Policy performance, but its training-time adaptation setting differs from this thesis's inference-time candidate selection setting. Table 3.1 summarises how each literature group maps to the benchmark design.

The chapter assumes familiarity with the technical foundations in Chapter 2; world models are introduced here.

Table 3.1 Literature roles in the benchmark design.

Work	Relevant strength	Relevance and limitation for this thesis
ACT [8]	Efficient bimanual action chunking with a structured latent variable.	Provides the transformer policy baseline, but its sampled candidates collapsed in the final implementation, limiting its use as a selector stress test.
Diffusion Policy [9]	Strong multimodal visuomotor policy with diverse sampled action chunks.	Main policy family for the headline result, with higher inference cost than ACT.
V-JEPA2 and V-JEPA2-AC [24]	Internet-scale video representation and action-conditioned latent prediction.	Main positive world-model evaluator; the action-conditioned head gives the most direct match to candidate scoring.
DexWM [25]	Dexterous manipulation world model with hand-consistency supervision.	Important comparator, but no public pretrained checkpoint was available and full upstream-scale pretraining fell outside the fair local budget.
LeWM [26]	Lightweight end-to-end latent dynamics model.	Useful for action-prior evidence, but the visual rollout route did not support the main semantic-selection claim.
DINO-WM and JEPA-WM [27]	Official checkpoint route for image/JEP-style latent prediction.	Exploratory official-checkpoint comparators; action-interface mismatch required an adapter and results remain appendix-level.
DiWA [12]	Uses imagined experience to improve Diffusion Policy.	Closest related study; it adapts the policy at training time rather than selecting candidates at test time. Its released policies are examined in a secondary reproduction and guided-selection study.
SimplerEnv and WorldEval [28, 29]	Evidence that offline/simulated rankings can correlate with real-world outcomes.	Supports the offline-evaluation premise, while requiring careful claims about calibration and deployment validity.

3.1 Visuomotor policies

Two policy families dominate current imitation learning for manipulation, and they attack the same pair of failure modes, compounding error and multimodal demonstrations, from opposite representational directions. This section reviews each in turn and then examines what the comparative literature actually establishes about their relative strengths, which is less than is commonly assumed.

3.1.1 Action Chunking with Transformers (ACT)

ACT was introduced by Zhao et al. to address the compounding error and multimodal distribution challenges that make bimanual manipulation demanding for imitation learning, achieving success rates of up to 90% on fine-grained tasks with the A Low-cost Open-source Hardware System for Bimanual Teleoperation (ALOHA) platform [8]. In SE(3) wrist-pose prediction specifically, small deviations in predicted position or orientation alter contact geometry in ways that cascade into grasp failure [2], and a policy trained with a naive regression loss produces averaged predictions that may not represent any plausible trajectory when the demonstrated distribution is multimodal [9]. ACT's CVAE architecture addresses both of these failure modes directly.

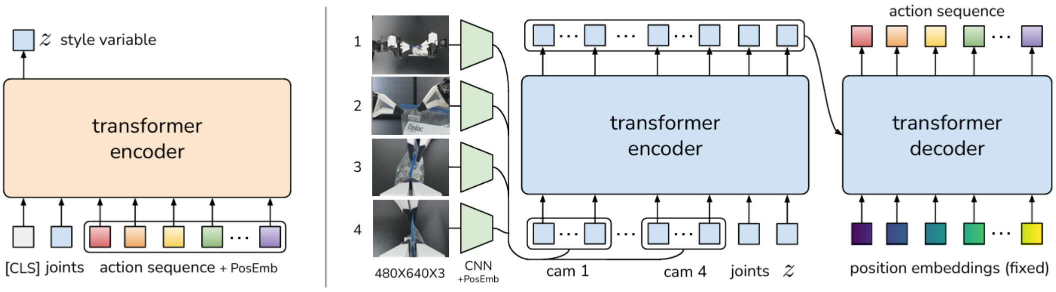

Figure 3.1 Architecture of the Action Chunking with Transformers (ACT) policy [8].

ACT does so through the CVAE design of Section 2.5, depicted in Figure 3.1. A CVAE encoder compresses the demonstrated action sequence and joint observation into a style variable z that captures the multimodal distribution of plausible futures, and a transformer decoder predicts the future action chunk from the visual observations, joint positions, and z. The encoder is discarded at test time. In the original formulation, z is set to the mean of the prior (zero) to produce a single deterministic action chunk per observation; when a pool of candidate chunks is required, multiple values of z are sampled independently from the prior and each is decoded into a separate candidate, all conditioned on the same observation [8]. At inference, Zhao et al. additionally apply temporal ensembling, averaging overlapping chunk predictions with recency weights to smooth the executed motion [8].

ACT carries known limitations alongside these strengths. The Gaussian prior on the style variable z constrains how faithfully the CVAE can represent highly multimodal action distributions: at ambiguous transitions the latent space can average across modes rather than sample cleanly from each one [9]. A second limitation follows from the architecture's reliance on the decoder to make use of z. The decoder conditions on both the observations and the latent variable, and nothing in the training objective forces it to draw on z when the observations alone suffice to reconstruct the demonstrated actions. If the decoder learns to ignore the latent variable, sampling different values of z no longer changes the predicted chunk and the candidate pool degenerates into copies of a single output; this failure mode is known in the VAE literature as posterior collapse [30].

The VAE literature also offers mitigations, including annealing the Kullback-Leibler weight during training [30] and reserving a minimum information budget for the latent through free bits [31], yet the standard ACT recipe trains with a fixed Kullback-Leibler weight and adopts neither. Zhao et al.'s own ablation sharpens the picture of when the latent matters: removing the CVAE objective makes almost no difference on scripted, deterministic demonstrations, but on human demonstrations success collapses from 35.3 per cent to 2 per cent [8]. The latent variable is therefore load-bearing exactly when the data is multimodal, which is also the regime in which its training is most fragile. Beyond the latent, chunk length is a fixed hyperparameter that must be tuned per task, and temporal ensembling introduces a recency-weighted lag that can slow the policy's response to sudden environmental changes [8].

3.1.2 Diffusion Policy

Diffusion Policy approaches the same compounding-error and multimodality challenges from the opposite representational direction. Instead of a structured latent variable, it starts each prediction from Gaussian noise and iteratively denoises it into an action chunk, conditioned on the recent observations [9]. Because every denoising pass explores the full action distribution rather than committing to a single latent mode, sampled chunks capture multimodal demonstrations without mode averaging, and independent samples from the same observation form a naturally diverse candidate pool. Figure 3.2 shows the general formulation and its two published heads, a CNN variant with FiLM observation conditioning and a transformer variant with causal cross-attention; their architectural details are given in the original papers [9, 32, 33]. The transformer variant is the form used in this benchmark.

Chi et al. benchmark Diffusion Policy across twelve tasks from four robot

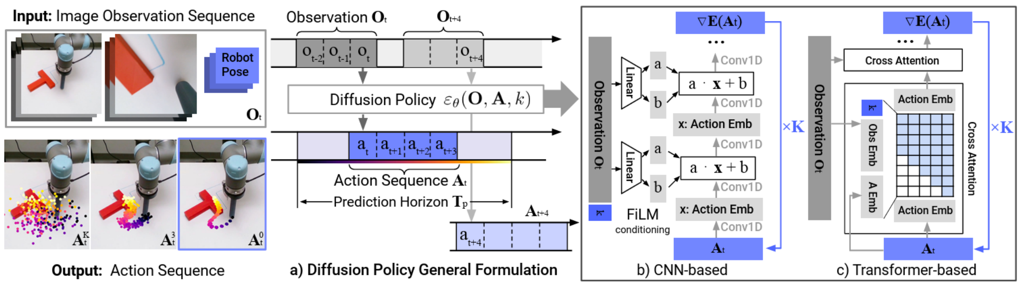

Figure 3.2 Diffusion Policy: general formulation (a), CNN variant (b), transformer variant (c) [9].

manipulation benchmarks and report an average improvement of 46.9 per cent over the prior state of the art, including implicit and mixture-density policy baselines [9]. The advantage is attributed to iterative denoising exploring the full action distribution at each step rather than committing to a single latent mode as a CVAE prior must. Notably, ACT is not among the compared methods: the two papers appeared within weeks of each other and neither evaluates the other, a point taken up below.

The main limitation of Diffusion Policy is inference cost. Generating a single action chunk requires multiple sequential network evaluations: standard Denoising Diffusion Probabilistic Models (DDPM) variants use on the order of 100 denoising steps compared with the single forward pass of ACT's decoder, which substantially reduces the achievable control frequency [9]. The scale of the follow-up effort is itself evidence of how seriously the field treats this defect. Efficient transformer variants recover speed through progressive noise scheduling and parameter sharing [33], Consistency Policy distils the denoising chain into a one- or few-step generator [34], and the π_0 generalist policy abandons discrete-step diffusion for flow matching to reach real-time control rates [35]. Iterative denoising in its original form is thus increasingly treated as a starting point to be distilled away rather than a final design.

In practice, the two families suit different pressures. ACT's single decoder pass makes it the cheaper option and well suited to structured, repeatable bimanual tasks, but the CVAE latent it depends on for diversity is exactly what degrades on highly multimodal human demonstrations, as Zhao et al.'s own ablation shows [8]. Diffusion Policy pays for its iterative denoising with inference cost, but that same process is what lets it explore multimodal action distributions without collapsing to an average [9]. Both are used extensively for hand and bimanual manipulation, and the strongest signal that this trade-off is real rather than incidental is that the ACT authors' own follow-up, ALOHA Unleashed, replaces the CVAE with a diffusion head once the tasks become deformable-object and contact-rich enough [36]. What the main literature surveyed here does not offer, however, is a direct comparison of the two on shared bimanual data under a matched budget: Diffusion Policy's twelve-task study predates ACT and does not include it, ACT is benchmarked only against BC-ConvMLP, BeT, RT-1, and VINN, and later multi-policy studies such as Lee et al.'s behaviour-generation comparison include Diffusion Policy but still omit ACT [8, 9, 37].

3.2 World models

A world model is a learned predictor of how a system evolves over time. Given the current state and a proposed action, it estimates the state that is likely to follow. In a manipulation setting, the state may be a raw image, a latent representation of the scene (Section 2.4), or a structured description of hand and object configuration. Regardless of the representation chosen, the underlying principle is the same: the model internalises the dynamics of the environment and uses them to project forward from any given state [38].

The case for world models is itself contested, and the disagreement has deep architectural roots, traced from a common ancestor in Figure 3.3. The generative camp treats scale as the path to competence: VLAs such as RT-2, OpenVLA, and π_0 couple web-scale vision-language backbones to autoregressive or flow-based action decoding, and their authors report broad generalisation to novel objects and instructions as evidence that this route is working [35, 39, 40]. LeCun has argued the opposite: architectures that generate actions without predicting their consequences cannot plan reliably, and scaling autoregressive models toward physical intelligence is a dead end [10, 11]. The joint-embedding branch, and the latent world models built on it, is the constructive form of that critique. This dissertation does not adjudicate the debate; it tests one specific claim from the world-model side, that predicted latent futures carry enough signal to rank candidate actions.

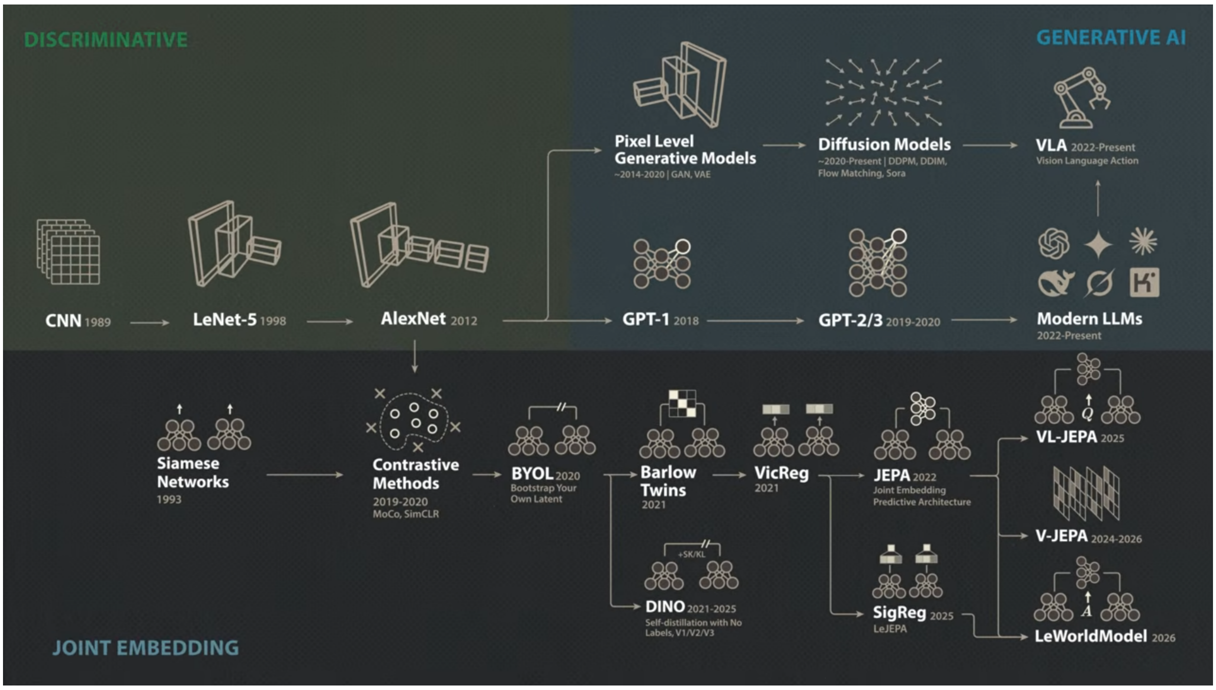

Figure 3.3 The post-AlexNet split of visual representation learning: a generative branch (top), culminating in diffusion models and the VLAs debated above, and a joint-embedding branch (bottom), culminating in the JEPA-style world models evaluated in this benchmark. Screenshot from [41].

World models become especially useful when a pool of candidate actions must be evaluated before any one is committed to. Each candidate is rolled forward through the model, producing a predicted future state; those predictions are ranked by predicted cost or consistency. The quality of this ranking depends heavily on the chosen representation: pixel-space world models must reconstruct every visual detail and suffer from mode-averaging at multimodal transitions, while models that operate in the compact learned latent space introduced in Section 2.4 can concentrate predictive capacity on the dynamics that matter for task outcomes [20]. The following subsections survey how world models have been used to improve policies, the literature supporting the evaluator design, and the architectures implemented in the benchmark.

3.2.1 World models as policy refiners

The most direct way to use a world model to improve a policy is to adapt the policy inside it. DiWA develops this idea on CALVIN, a long-horizon manipulation benchmark where a Franka arm operates a tabletop drawer, sliding door, lightbulb, LED, and coloured blocks, shipped with hours of unstructured teleoperated play alongside expert demonstrations [42]. DiWA treats the world model as a training substrate: a diffusion policy is fine-tuned with reinforcement learning (Section 2.1) entirely inside a frozen world model's imagination, following the recipe in Figure 3.4. That world model is a DreamerV2-style recurrent state-space model trained on roughly six hours of task-agnostic play, about 500,000 transitions, with reward supplied by a binary success classifier trained on latent embeddings of the expert demonstrations. The denoising chain of the diffusion policy is framed as a Markov decision process, letting proximal policy optimisation run on imagined rollouts while a behaviour-cloning regulariser keeps the policy close to its pretrained initialisation [12].

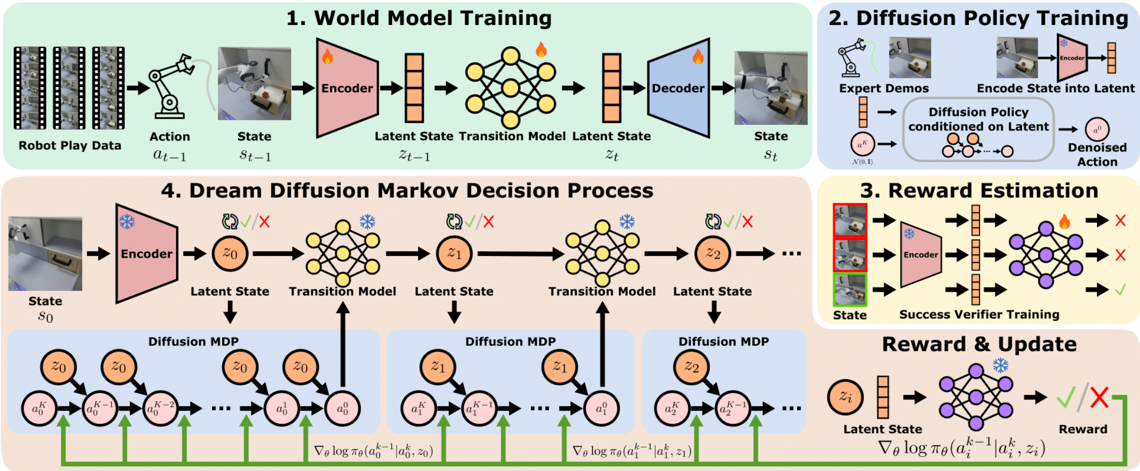

Figure 3.4 The three-stage DiWA recipe: a world model is trained on task-agnostic play data, a diffusion policy is pretrained by imitation in its latent space, and the policy is then fine-tuned with reinforcement learning on rollouts imagined entirely inside the frozen world model [12].

Across all eight CALVIN skills tested, this purely imagined fine-tuning raises task success substantially: baseline success ranges from 35.63% to 62.55%, and DiWA fine-tuning lifts every skill above 74%, up to 91.95% on close-drawer, averaged over three seeds [12]. A model-free baseline that fine-tunes the same policy in the real environment needs hundreds of thousands to millions of physical steps to reach comparable performance, where DiWA uses none. The authors also report the first fine-tuning of diffusion policies for real-world robotic skills through an offline world model, improving three physical manipulation skills after imagined fine-tuning on a world model trained from roughly four hours of teleoperated play [12]. They are explicit about the limits of this evidence: because the world model is frozen, its errors persist through fine-tuning and can be exploited by the policy, and gains measured in imagination do not always survive execution on the physical robot [12].

3.2.2 World models as policy evaluators

WorldGym demonstrates that a learned world model used as a simulated environment for policy evaluation produces rankings that align with real-world task success with meaningful fidelity [43]. Candidate policies are evaluated not by deploying them physically but by rolling them out inside the model's predicted environment, reducing the cost of policy comparison to a forward pass.

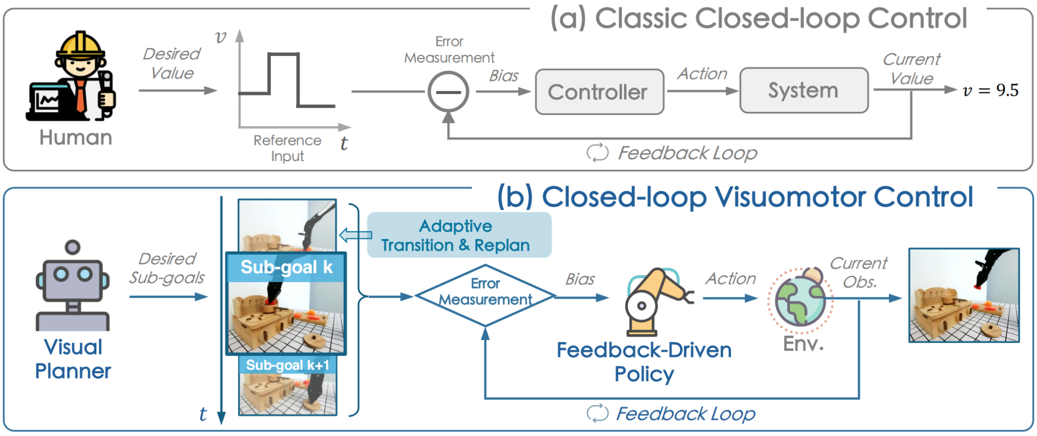

Figure 3.5 Classic closed-loop control (a) versus CLOVER's closed-loop visuomotor control (b) [44].

CLOVER extends this to closed-loop control, as shown in Figure 3.5. A visual planner generates a sequence of sub-goal frames; the executor measures the embedding-space error between the predicted sub-goal and the current observation. When the error exceeds a threshold, the system triggers an adaptive transition and replans rather than committing to the original trajectory. The feedback loop, absent from open-loop imitation policies, is the mechanism that translates world model prediction accuracy into task completion gain [44]. Together, WorldGym and CLOVER establish that world model rankings are informative both before and during execution.

The practical value of this evaluator loop is measurable. Table 3.2 reproduces CLOVER's real-world evaluation on a multi-step robot manipulation suite, reporting the average number of consecutive sub-tasks completed (`avg_len`) for representative baselines and CLOVER [44].

Table 3.2 Average consecutive sub-task completions (`avg_len`) on a real-world multi-step manipulation suite. Higher is better. CLOVER's closed-loop feedback via world model prediction nearly doubles the completion depth of the strongest open-loop baseline. Data from [44].

Method	avg_len
ACT [8]	0.6
R3M (pretrained visual repr.) [45]	0.7
RT-1 [46]	1.1
CLOVER (closed-loop)	**2.1**

Alongside the task completion gain, CLOVER also reduces embedding-space prediction Root Mean Square Error (RMSE) relative to open-loop baselines, suggesting prediction accuracy and task success move together under the evaluator framing. This correlation is not guaranteed by construction. However, Lambert et al. document an objective mismatch at the heart of model-based reinforcement learning, since world models are trained to minimise prediction error, yet low prediction error does not by itself produce good control, and the correlation between the two can be weak [47]. A world model's rankings therefore inherit no guarantee from its training objective, and whether they are informative, including whether they transfer to physical outcomes and remain reliable out of distribution, must be established empirically in each setting, a question revisited when results are presented.

3.2.3 Latent predictive architectures

The energy-based framework of LeCun and Dawid establishes why prediction should operate in latent rather than pixel space [20]. The same representational argument introduced in Section 2.4 applies directly to the prediction target, favouring compact embeddings over pixel reconstructions that force the model to account for irrelevant visual detail and average over all plausible continuations at ambiguous transitions.

The pixel-space route is nonetheless pursued at scale, and its supporters reach the opposite conclusion. GAIA-1 predicts future driving video directly [48], and Genie learns action-controllable interactive environments from unlabelled internet video [49], treating faithful visual generation itself as evidence that a useful world model has been learned. The joint-embedding position is that this fidelity is bought with capacity spent on task-irrelevant detail. The architectures reviewed below are built on that counter-position: they are latent predictors, and this benchmark evaluates them on whether their predictions rank candidate actions well, not on whether they can generate a convincing video.

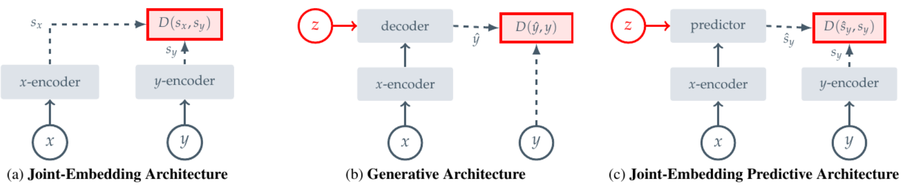

Figure 3.6 Three SSL prediction paradigms: joint-embedding (a), generative (b), and joint-embedding predictive (c) [14].

Image Joint Embedding Predictive Architecture (I-JEPA) establishes the joint-embedding predictive architecture as a viable approach to learning rich visual representations without any pixel reconstruction objective [14]. As Figure 3.3 traces, this predictive objective is the direct ancestor of most world models surveyed in this section. Panel (c) of Figure 3.6 shows the data flow directly: a context block x from the image is passed through the context encoder to produce latent tokens z_x. A lightweight ViT predictor then takes z_x together with positional tokens indicating where the masked target regions y are located, and outputs predicted latent representations $\hat{s}_y$ for those regions. A separate target encoder, whose weights are a slow exponential moving average of the context encoder, independently encodes the actual target pixels into ground-truth latents s_y. The loss MSE$(\hat{s}_y, s_y)$ is computed entirely in latent space: no decoder, no pixel generation, no reconstruction of lighting or texture. The outcome is a context encoder whose representations capture semantically meaningful structure (object shape, spatial layout, and scene dynamics) because that is all the predictor ever needs to get the target embeddings right. Training is approximately 2.5 times faster than pixel-reconstruction methods as a result. V-JEPA2 extends this same joint-embedding predictive architecture from images to video sequences. The following subsections cover the fundamental literature of the architectures explored throughout this benchmark.

V-JEPA2

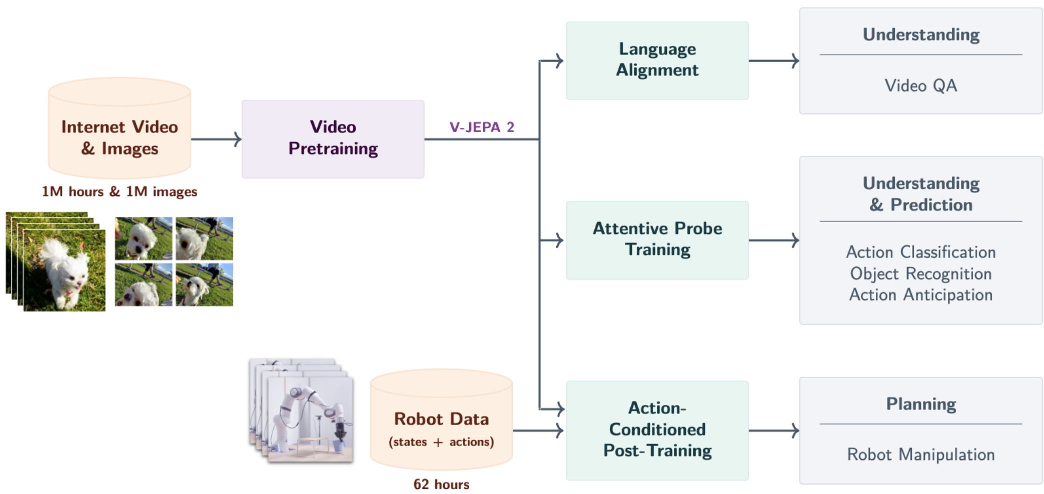

Figure 3.7 V-JEPA2 post-training pathways from internet-scale backbone to specialised applications [24].

V-JEPA2 extends the I-JEPA framework from static images to video, pretraining on over one million hours of internet video [24]. The pretraining shifts from spatial to spatio-temporal masking: the context encoder receives past observation frames, and the predictor must recover future latent states. Training combines two regimes: one where the predictor is always shown the true preceding frames at each step, and a second, multi-step rollout regime, where it instead conditions on its own earlier predictions. The former keeps short-horizon predictions accurate, and the latter keeps long-horizon trajectory representations consistent once prediction errors are allowed to compound. The result is a frozen ViT-Large encoder, approximately 300 million parameters, that encodes physical priors about hand and object interactions acquired from internet-scale pretraining. As shown in Figure 3.7, the pretrained backbone supports three post-training pathways: Language Alignment, which grounds visual embeddings in natural language; Attentive Probe, which adds a lightweight classification head; and Action Conditioning, which extends the predictor to take robot actions as input and forms the basis of V-JEPA2-AC.

V-JEPA2-AC

The frozen ViT-Large encoder from V-JEPA2 serves as the backbone of V-JEPA2-AC, producing a fixed latent embedding for each observation frame with no gradient updates to the backbone weights. A lightweight action-conditioned transition head is trained on top of these stored embeddings. It takes the current frame latent together with a proposed action chunk (robot joint states and end-effector poses) as the conditioning signal z, and predicts the latent representation of the future frame. The head is trained with an L1 objective (the mean absolute difference between predicted and actual embeddings) against the embedding of the actual future frame. At planning time, the original work scores candidate actions by the distance between their predicted future embeddings and the embedding of a given goal image [24]. This design, shown on the right of Figure 3.8, separates what the network must learn into two parts. The frozen encoder supplies rich visual features that generalise from internet-scale pretraining, and the small trained head learns only how actions propagate through those features.

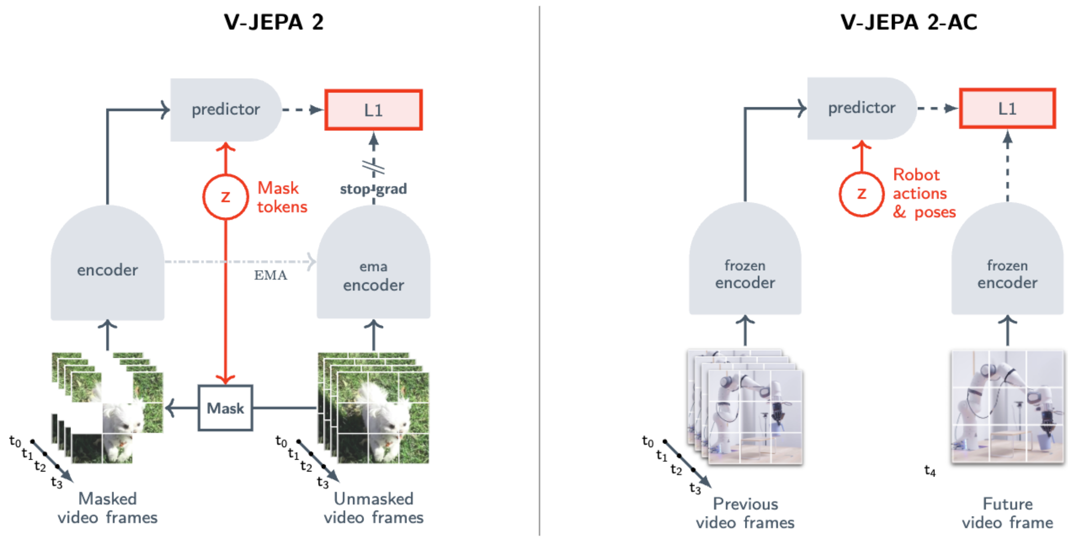

Figure 3.8 Standard V-JEPA2 (left) versus action-conditioned V-JEPA2-AC (right) [24].

An important qualification is raised by Tsagkas et al., who show that pretrained visual representations can encode spatial plausibility well while providing a weaker signal for fine-grained temporal dynamics [50]. A frozen encoder trained on internet video may represent hand shape and position accurately at each individual frame while providing a less reliable signal about whether the predicted trajectory is kinematically consistent across the full chunk horizon. This temporal limitation provides context for the path shape results in Chapter 7.

DexWM

DexWM [25] is a dexterous manipulation world model from Meta FAIR, shown in Figure 3.9. Each input frame is encoded by a frozen DINOv2 image encoder into a grid of spatial patch tokens, a CDiT predictor advances the latent state under action and camera-motion conditioning, and a lightweight decoder maps the result back to image and keypoint space. Its distinguishing feature is the Hand Consistency loss: an auxiliary head predicts fingertip and wrist heatmaps from the latent state, imposing a hand-geometry prior that is absent from both V-JEPA2-AC and LeWM. Goswami et al. report a 50% improvement over Diffusion Policy on grasping and placing tasks and an 83% zero-shot real-world grasping success rate under latent planning [25]. These figures come from a single study with no public checkpoint at the time of writing, and have not been independently reproduced. Where V-JEPA2-AC transfers a video-pretrained backbone, DexWM transfers one pretrained on static images, so the comparison in this benchmark isolates the contribution of video-specific temporal pretraining.

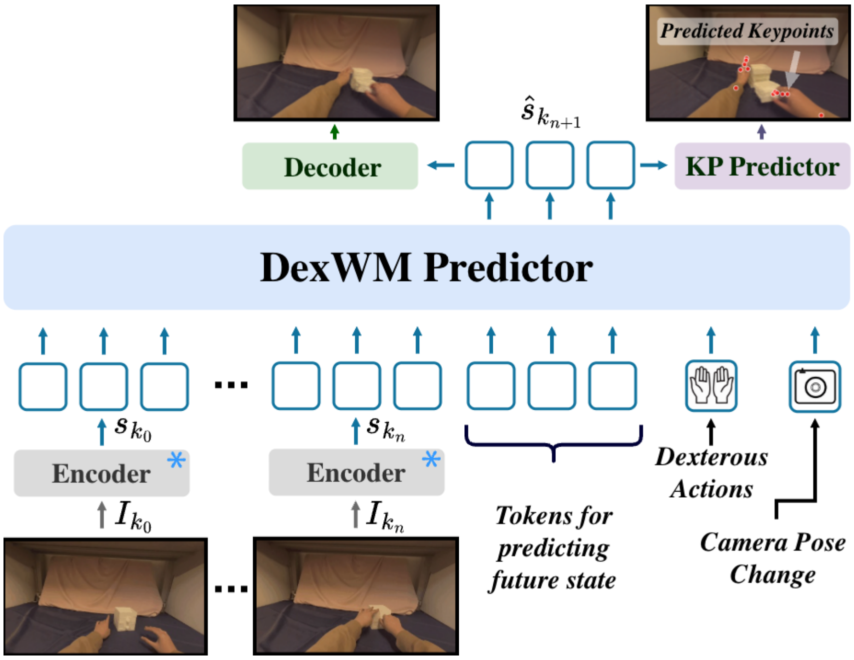

Figure 3.9 High-level layout of the DexWM world model pipeline, illustrating how predicted keypoints are incorporated alongside the encoder, CDiT predictor, and decoder stages [25].

LeWorldModel

LeWM is a latent world model trained end-to-end from random initialisation on raw pixel observations [26]. As shown in Figure 3.10, a ViT-Tiny encoder of roughly 15 million parameters maps each frame to a single 192-dimensional token, and a dynamics predictor estimates the next latent from the current token and the action. Training combines a next-embedding prediction loss with SIGReg, a regulariser that enforces approximate Gaussianity of the latent distribution along random univariate projections. This single regularisation term stabilises end-to-end training without contrastive pairs or a moving-average target encoder, reducing the tunable loss hyperparameters of prior JEPA-style designs from six to one [26].

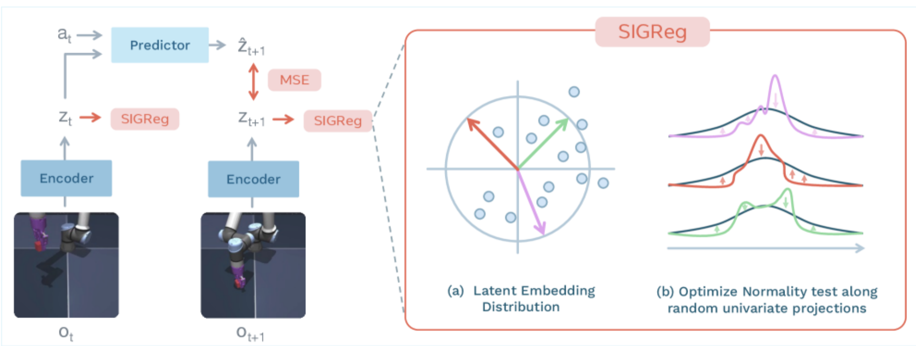

Figure 3.10 LeWM training pipeline: encoder, dynamics predictor, and SIGReg regulariser [26].

The design trades spatial resolution for speed. LeWM trains on a single GPU in a few hours and, at planning time, its compact latent achieves a 48× speedup over models that predict over frozen DINOv2 patch grids, while still detecting physically implausible transitions and tracking agent and object locations across frames [26]. As with DexWM, these results are self-reported by a single group, and the model is recent enough that independent evaluations do not yet exist.

Read together, the three architectures occupy different points on a cost-versus-prior trade-off, though only LeWM's speedup claim is directly benchmarked against a comparable model in its source paper. V-JEPA2-AC carries a heavy but frozen video-pretrained Vision Transformer (ViT)-Large backbone with only a lightweight trained transition head. DexWM is the most resource-intensive of the three: on top of a similarly heavy DINOv2 backbone it adds a second learned CDiT predictor and an auxiliary hand-geometry head, the extra machinery behind its reported Hand Consistency gains. LeWM, unlike DexWM, discards pretraining altogether for a from-scratch ViT-Tiny encoder, which its authors report is far cheaper to train and run, consistent with the 48× planning speedup above. This compact from-scratch route is what the benchmark tests against the pretrained-transfer routes of V-JEPA2-AC and DexWM, a comparison returned to with results in Section 6.4.

DINO-WM and JEPA-WM

Two further latent world models enter the benchmark as exploratory official-checkpoint routes, and each pairs with one of the architectures above. DINO-WM performs latent prediction over a frozen DINOv2 patch grid [27], the same backbone family DexWM builds on, and Figure 3.3 places both on the self-distillation branch rather than the predictive JEPA branch. JEPA-WM denotes the officially released JEPA world-model checkpoints trained on robot manipulation data [51]. Like LeWM, it sits directly on the predictive branch established by I-JEPA above, though as an independently pretrained checkpoint rather than a from-scratch design. Both DINO-WM and JEPA-WM provide pretrained checkpoints, but their expected action interfaces do not match the 18-dimensional bimanual wrist space, so each is evaluated through an adapter and treated as an exploratory comparator paired with its architectural relative. Their architectures and validation status are documented in Appendix A.

PointWorld

PointWorld is a large pretrained 3D world model for in-the-wild rigid arm manipulation [52]. Unlike the latent-space models described above, it operates directly in 3D geometry. Scene state is lifted from RGB-D images into a full-scene point cloud, and robot actions are represented as the 3D point flows traced by robot surface geometry through forward kinematics. This embodiment-agnostic representation allows the model to reason over physical interaction geometry rather than learned abstract features.

Such a model is not designed to handle an 18-dimensional bimanual wrist-pose action space. Its forward-kinematics representation assumes rigid arm geometry with a discrete gripper aperture, making it structurally mismatched when applied to continuous SE(3) wrist transforms from human hand demonstrations.

4 Methodology

This chapter defines how the central research question (Section 1.2) was translated into a controlled, testable benchmark. Answering whether a frozen world model can improve candidate selection, and whether that holds across policy families, requires isolating the selection mechanism from every other source of variance: the data and action space used, the branch structure that separates policy-only from world-model-assisted selection, how each policy family generates candidates, which evidence standard a world-model family must meet before its result is treated as a claim, and the fairness rules that keep training and evaluation exposure comparable across families. Section 4.1 sets out the dataset and action space, Section 4.2 the three branch definitions, Section 4.3 the policy and candidate-generation routes, Section 4.4 the evidence roles assigned to each world-model family, and Section 4.5 the fairness and leakage rules that separate held-out evidence from exploratory implementation work. Evaluation metrics and scope limitations are addressed in Sections 4.6 and 4.7.

4.1 Dataset and Experimental Setup

4.1.1 EgoDex: source and selection rationale

The benchmark is built on EgoDex [53], a large-scale egocentric dataset of bimanual dexterous manipulation collected using Meta Quest Pro head-mounted cameras. It spans more than 100 task categories across its full release, including folding, slotting, pouring, and kneading, covering a wide range of fine-grained hand motions. The full release comprises five training parts and supplementary additional-data splits; this study uses the first three training parts (Section 4.1.3). Each episode provides synchronised RGB video and full six-degree-of-freedom SE(3) wrist annotations for both hands at approximately 30 frames per second.

The choice of dexterous manipulation as the evaluation domain is not incidental. This dissertation forms part of the University of Malta's ongoing M.AI.ESTRO research project [54], which investigates the use of AI for scene and task prediction on dexterous manipulation tasks to support industrial transfer learning and human skill digitisation. M.AI.ESTRO's immediate pilot targets human hand pose prediction and guidance from egocentric demonstration; a second, later pilot addresses robotic hand manipulation, and the intended pathway between them translates poses learned from human demonstration into low-level actuator commands for the robot controller, rather than having the system generate robotic actions directly. The research domain for this dissertation was therefore set by the scope of the first pilot. The introduction situates this motivation more broadly, and the present chapter focuses on the methodological consequences of the domain choice.

Within the dexterous manipulation setting, EgoDex was selected on three properties:

- The benchmark required a fine-grained bimanual hand pose action space, provided through an eighteen-dimensional bimanual wrist pose representation that large-scale corpora built around rigid grippers or arm contact do not provide.
- The first-person perspective aligns with the intended downstream deployment setting of a wearable assistive device (a VR guidance application).
- The corpus is large enough to support simultaneous training of five world model families and two policy architectures on declared non-overlapping splits, while retaining a held-out test set of meaningful size.

EgoDex's demonstrations are human rather than robotic, and it carries no contact or force signal, only kinematic SE(3) pose. Neither is a limitation as scoped here; both are discussed further in Section 4.7.

4.1.2 Windowing protocol

The dataset is stored as paired `.mp4` and `.hdf5` episode files. A deterministic index stage discovers all valid paired episodes, infers their lengths from HDF5 metadata, and writes canonical observation and prediction windows for each split. This shared index is the single source of truth for all downstream modules: policy training, world model extraction, and evaluation all consume the same window file for their respective split, ensuring that output differences across the experimental branches can be attributed to branch design rather than to differences in the data presented to each component. The benchmark uses a fixed-stride windowing scheme as the canonical contract. A supplementary state-change-targeted scheme was identified later, while iterating on this pipeline, and was not scaled into the canonical benchmark; a small comparison against the canonical scheme was run to check whether it would have been worth adopting, and is summarised at the end of this section and detailed in Appendix B.10.

Strategy 1: Fixed-stride sampling. Each window spans 16 observation frames followed by 16 prediction frames, with consecutive window starts separated by a stride of 128 frames. At 30 frames per second this places starts approximately 4.3 seconds apart, corresponding to an observation context and prediction horizon of approximately 0.53 seconds each. This contract is encoded as `obs16_pred16_s128` and forms the foundation for all policy training and world model training (see Figure 4.1).

Strategy 2: State-change-targeted sampling. The fixed-stride scheme samples the episode timeline uniformly, which causes it to under-sample high-motion events: long

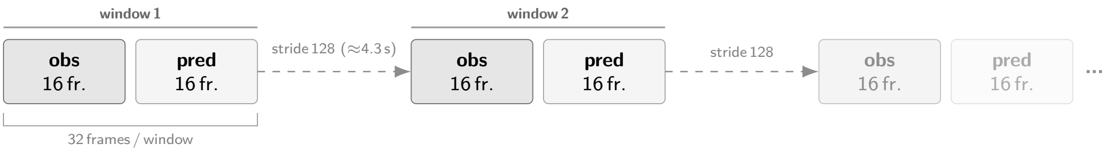

Figure 4.1 Fixed-stride windowing scheme (`obs16_pred16_s128`) used as the canonical training and evaluation index. Author-generated schematic.

near-static segments contribute windows at the same rate as active manipulation periods. As detailed in Chapter 5, this limitation motivated a revised strategy adopted for a subsidiary perceptual inspection task and qualitative development review. Rather than placing windows at fixed intervals, this approach computes smoothed hand-state change scores from the wrist transform and confidence sequences, selects candidate windows near high-change peaks, and applies episode-first task-stratified sampling with a fixed random seed (see Figure 4.2). A relative-position fallback retains slower episodes that would otherwise be excluded, yielding one frozen window per episode stratified across all represented task categories.

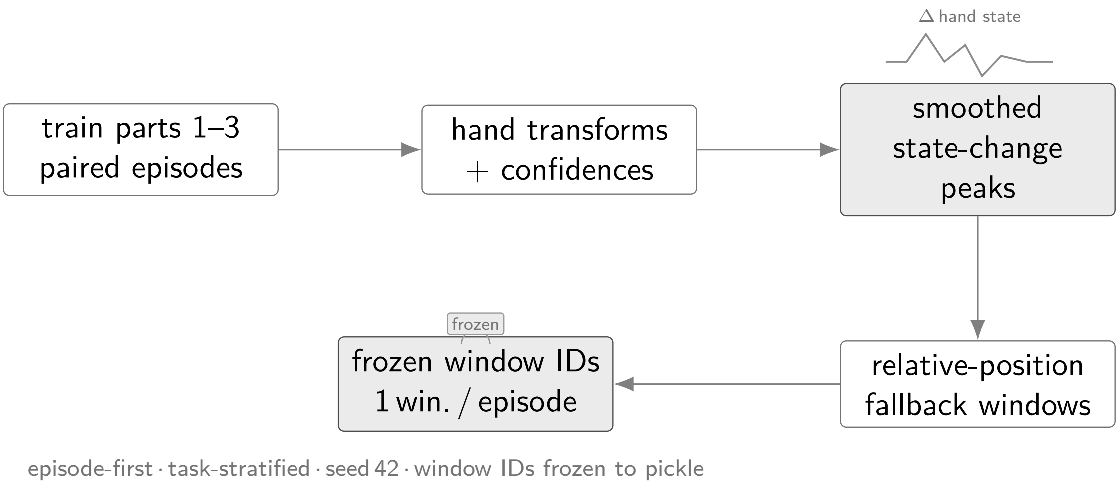

Figure 4.2 State-change-targeted sampling pipeline yielding one frozen window per episode. Author-generated schematic.

The fixed-stride scheme (Strategy 1) is used as the canonical protocol across all headline experimental branches in this study. Strategy 2 was not adopted for the canonical benchmark; instead, a small standalone comparison, its own state-change-trained checkpoint, support extract, and test index, was run to check whether it would have been worth adopting. The result was mixed: state-change selection improved one world model's candidate capture and worsened another's, so it does not justify replacing Strategy 1 as the study's canonical protocol. Strategy 2 remains a promising direction for future work rather than a demonstrated improvement. The full comparison, method, and per-model results are reported in Appendix B.10.

4.1.3 Training splits and test set

Three cumulative training splits were assembled from the first three parts of the EgoDex release, each a strict superset of the previous; Table 4.1 summarises their composition. This three-part subset is used by design rather than the full five-part release: these first three parts were seen to give the most substantial gains in performance across model families, and training was not extended further given the fixed training-exposure budget available on the shared cluster GPUs. How performance scaled from one split to the next is detailed in Appendix B.11. How the three splits relate to each other during training is addressed in Section 4.5.1. The held-out test split is the official EgoDex test set and is never used during training at any stage.

Table 4.1 EgoDex splits under the `obs16_pred16_s128` windowing contract. Episode and window counts are derived from the canonical index.

Split	Tasks	Episodes	Windows	Role
`train_part1`	26	46,091	140,239	Stage 1 training
`train_part1_2`	39	140,654	291,369	Stage 1+2 training
`train_part1_2_3`	63	194,333	430,962	**Canonical training**
`test`	111	3,229	7,607	**Frozen evaluation**

The canonical results in Chapter 6 use `train_part1_2_3`. The test split spans 111 task categories, including all 63 present in `train_part1_2_3` plus 48 drawn from the excluded parts; this exposes the benchmark to some task-level distribution shift, though it is not a controlled generalisation test, since the excluded categories were never held out as an experimental variable. Its window index is frozen throughout.

4.2 Three-Branch Experimental Framework

Answering the central research question (Section 1.2) cleanly requires holding every other experimental factor constant. The benchmark therefore organises evaluation around three branches that share the same policy weights and the same evaluation windows; the candidate pool composition and selection rule differ across branches (Figure 4.3). Any measured performance difference between branches is intended to reflect the selection mechanism, since training data, architecture, and window coverage are held constant across branches.

The candidate pool size is fixed at $K = 5$ throughout this study. No literature precedent dictates this figure specifically, so it is treated as a compute-budget variable rather than one derived from theory: a common-random-number sweep over $K \in \{2, 3, 4, 5, 6, 8, 10, 12, 16, 20\}$, detailed in Appendix B.12, confirms that the ranking of world-model scorers is identical at every pool size tested, so no comparative conclusion drawn in this work is an artefact of fixing K at five. The limitation is discussed further in Section 4.7.

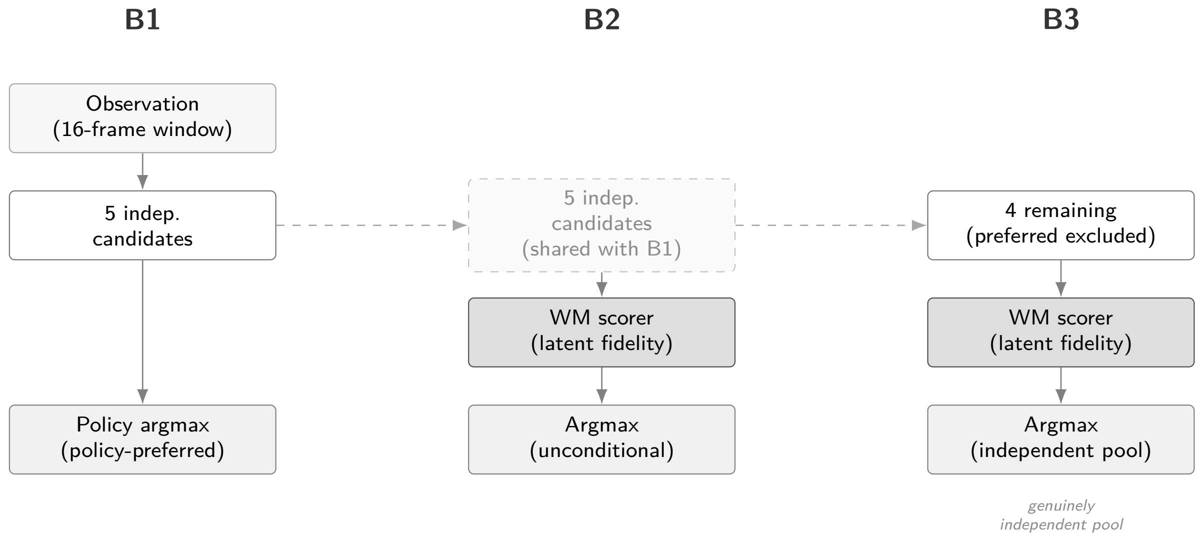

Figure 4.3 Three-branch evaluation framework (Section 4.2), shown for a policy producing five independent candidates. B2 and B3 reuse the same candidate pool generated for B1, relayed through the branches (dashed lines) rather than regenerated; B3 additionally removes the policy's preferred candidate before scoring. Both ACT and DP follow this design; the per-policy candidate-generation mechanics are detailed in Section 4.3.

Branch 1 (B1): Policy-only baseline. The policy generates its full candidate pool and selects by its own internal preference; no world model is queried. The ideal design produces several structurally independent candidates so the downstream branches can compare meaningfully different trajectories. B1 establishes the trajectory prediction quality each policy achieves acting alone and is the baseline against which B2 and B3 are measured.

Branch 2 (B2): World-model-inclusive selection. The world model scores all candidates in the pool and selects the highest-scoring one unconditionally; no margin threshold is applied. The candidate pool is the same as B1. Scoring proceeds by predicting forward in the world model's latent space from the current observation and measuring prediction fidelity against each candidate action. The policy's own output is not privileged; it is selected only if the world model assigns it the highest score. For V-JEPA2 without action conditioning, the transition head required to produce candidate-specific latent predictions is absent; if the model assigns indistinguishable scores to all candidates, B2 degenerates to B1, directly testing whether action conditioning is the operative mechanism.

Branch 3 (B3): World-model-exclusive selection. The policy's preferred output is excluded from the evaluation pool; the world model selects the highest-scoring remaining candidate unconditionally. The ideal design ensures the remaining candidates are structurally independent so that exclusion meaningfully changes the selection outcome. B3 tests whether the world model maintains selection quality when the policy's top choice is unavailable. All five world model families apply this protocol.

The three branches select by argmax at three different points in the pipeline, and the distinction matters: B1's *policy-preferred* argmax is the policy's own internal choice, made without reference to any external score; B2's *unconditional* argmax is the world model's choice over the full pool, including the policy's own preferred candidate; B3's *independent-pool* argmax is the world model's choice with that preferred candidate already removed, over the remaining structurally independent candidates only.

Two policy families are evaluated: ACT and DP (Figure 4.3). Each policy generates its own pool of five candidates for B1 and B2, and scores the remaining four once its preferred candidate is excluded for B3, following the design above. The two families use different sampling procedures to produce their pools; candidate generation details are in Section 4.3.

4.3 Policy Variants

The benchmark evaluates two imitation-learning policy families. Their architectures and training objectives are reviewed in Chapter 3; the properties relevant to this study are how each generates its candidate pool and what that implies for world-model scoring.

- **Action Chunking Transformer** (Section 3.1.1). ACT is architecturally designed to generate diverse candidates by sampling latent vectors $z \sim \mathcal{N}(0, I)$ from its CVAE prior and decoding each into a 16-step bimanual SE(3) action chunk. In the trained checkpoint used here, CVAE posterior collapse is observed: the decoder is insensitive to z at inference time, producing identical outputs regardless of the sampled vector (maximum output deviation under $z{=}10$ is $3{\times}10^{-7}$). ACT therefore contributes one deterministic candidate per window ($N{=}1$). The single forward pass requires one network evaluation. Three retraining attempts to resolve the collapse and the subsequent adoption of CEMP for B2/B3 candidate diversity are documented in Section 5.4.

- **Diffusion Policy** (Section 3.1.2). DP generates each candidate through an independent reverse-diffusion chain using a Denoising Diffusion Implicit Models (DDIM) schedule of ten steps (Section 3.1.2) [55]. Producing five candidates requires five separate chains and approximately fifty network evaluations per window, roughly an order of magnitude more than ACT. The Diffusion Transformer (DiT) backbone is used. Candidates are structurally independent because each starts from a fresh noise sample; diversity arises from stochastic initialisation rather than from a shared latent manifold.

Evaluating both families under the same three-branch protocol allows the study to ask whether world-model augmentation improves candidate selection in general or only under a particular generation mechanism. A result that holds across both constitutes stronger evidence for the generality of the approach.

4.4 World Model Families

The model families have different evidence roles in the final thesis. They are not treated as equal candidates for the headline claim. A result is promoted to main evidence only when it uses frozen candidates, training-only support, held-out test windows, and a scorer whose action representation is compatible with the candidate pool.

Headline scorer: V-JEPA2-AC. The architecture is reviewed in Section 3.2.3. A frozen V-JEPA2 [24] ViT-Large backbone provides a 1024-dimensional visual embedding, and a lightweight V-JEPA2-AC transition head predicts a future embedding from the current embedding and an 18-dimensional bimanual action chunk. The distance between the predicted latent and the candidate's reference latent is used as a selection score. This is the primary semantic world-model scorer because it combines a large pretrained video representation with explicit action conditioning.

Mechanistic controls. V-JEPA2 without action conditioning reuses the same visual representation but removes candidate-specific action information. PointWorld proxy scoring keeps the V-JEPA2-AC visual pathway but degrades the bimanual action chunk into an incompatible robot-arm-style representation. These controls test whether the positive result depends on meaningful action conditioning rather than on a strong visual backbone alone.

Secondary action-regularity scorer: LeWM action-prior. LeWM [26] was initially implemented as a visual rollout and latent-support world-model scorer. That route is not promoted to canonical semantic evidence because the causal diagnostic failed its action-support and ranking-validity gates. A narrower LeWM-derived action-prior scorer is retained as a secondary result. It uses training-only action statistics to prefer candidates closer to the training action distribution. This is a valid action-regularity signal, but it is not interpreted as successful visual rollout imagination.

Fixed-budget negative comparator: DexWM. Dexterous Manipulation World Model (DexWM) [25] is included because it is a close architecture-level comparator for dexterous world modelling. Public pretrained weights for the upstream EgoDex+DROID recipe could not be sourced. The thesis therefore evaluates a reproducible fixed-budget WACT implementation and treats its result as evidence only for that implementation. Full official pretraining is excluded from the main protocol because it requires large DROID raw-data onboarding, substantial preprocessing and inference cost, and a different training scale from the frozen fairness contract.

Exploratory official-checkpoint comparators: DINO-WM and JEPA-WM. DINO-WM and JEPA-WM were implemented later as official pretrained checkpoint routes through the JEPA-WMS codebase. They require an adapter from WACT's 18-dimensional bimanual wrist chunks to the 7-dimensional DROID-style action interface expected by those checkpoints. Because the validation pilots were mixed and the best settings required validation-only reweighting, they are not promoted to held-out test evidence. They belong to the implementation and appendix record as fair exploratory attempts.

Secondary reproduction and comparison: DiWA/CALVIN. DiWA is close enough to this dissertation's own question to warrant a direct, controlled comparison rather than literature discussion alone. It studies world-model-guided diffusion-policy improvement through imagined training, whereas WACT studies test-time candidate selection without retraining the policy; the CALVIN study is designed to compare these two improvement operators under shared components. Access to DiWA's official base and fine-tuned policy checkpoints was not immediate and had to be requested from the authors, which delayed the reproduction path. The methodology's plan is to reuse DiWA's frozen world model, reward classifier, and policy checkpoints unchanged, and to compare the unmodified upstream baseline, a fixed or random candidate control, a WACT-style test-time selector, and the paper's own imagined-PPO fine-tuning under identical evaluation episodes. This keeps DiWA/CALVIN separate from the EgoDex branch definitions: it is a secondary study, not a substitute for the primary benchmark. The implementation status of this reproduction is reported in Chapter 5, and its results, where available, are reported in Chapter 6.

Table 4.2 Evidence role assigned to each implemented or investigated world-model path.

Path	Evidence role	Main claim status
V-JEPA2-AC	Headline semantic scorer	Held-out test claim
V-JEPA2 no-AC	Action-conditioning ablation	Held-out control
PointWorld proxy	Action-degradation control	Held-out mixed control
LeWM visual rollout	Diagnostic scorer	No-go for semantic rollout claim
LeWM action-prior	Training-action regularity scorer	Secondary held-out claim
DexWM fixed-budget	Domain-specific comparator	Held-out negative/null claim
DINO-WM / JEPA-WM	Official-checkpoint exploratory routes	Validation-only, appendix
DiWA/CALVIN	Secondary reproduction and comparison	Reported separately in Chapter 6

4.5 Fair Experimental Protocol

Comparing the five world model families and two policy architectures against each other requires holding every non-design variable constant. Two aspects of fairness are addressed: training exposure and evaluation contract.

4.5.1 Training exposure budget

Different model families have different computational costs per training step and different sensitivities to data volume. Without a principled exposure budget, a comparison could conflate the effect of architecture with the effect of receiving more or fewer training samples. To determine an appropriate shared budget, a convergence study was conducted prior to full training: each model family was trained independently at six data scales (1k, 5k, 10k, 25k, 50k, and 100k effective items) on a fixed subset of the canonical training data, and the resulting loss curves were examined for plateau behaviour. The study and its results are described in detail in Chapter 5; the finding relevant to the experimental design is summarised here.

The effective training exposure E is defined using the following symbols (see the List of Notations):

E	effective training exposure (items consumed)
S	number of optimisation steps
B	per-device batch size
G	gradient-accumulation steps
A	number of accelerator devices
B_{global}	effective global batch size

$$E = S \times B_{\text{global}} \tag{4.1}$$

where:

$$B_{\text{global}} = B \times G \times A \tag{4.2}$$

This formulation counts the total training items consumed regardless of how steps are distributed across hardware, allowing models with different batch sizes and GPU counts to be normalised to a common scale.

The convergence study, described in Chapter 5, established a universal budget of $E = 100{,}000$ effective items. This figure is applied uniformly across all model families and all three training splits, ensuring that any measured performance difference reflects architecture and data rather than exposure volume. Each split is trained independently from a fresh initialisation at this same budget; later splits do not continue training from an earlier split's checkpoint. Any difference between the `train_part1`, `train_part1_2`, and `train_part1_2_3` results therefore reflects the data available at that scale, not a warm start carried over from a smaller split. The full data-construction and training curriculum is described in Chapter 5.

4.5.2 Evaluation contract

All models receive the same RGB input at evaluation time: the observation frames from the canonical test window, without additional depth, point cloud, or semantic metadata. This ensures that any performance difference between world model families reflects their internal representation and prediction quality rather than access to a richer input signal. All five families, including PointWorld, operate on the same V-JEPA2 visual latent representations; PointWorld's distinctive characteristic is its degraded action conditioning rather than a different observation modality.

The evaluation windows are fixed at indexing time and shared across all branches and all world model families. No window is selected, filtered, or re-sampled after training. Policy weights and world model weights are frozen before evaluation begins and are not updated. The only variable at evaluation time is the selection rule applied to the candidate pool, isolating the contribution of each branch design and each world model family from all other sources of variance.

The causal validator checks four leakage boundaries before a run can be treated as evidence. First, support episodes must not overlap with evaluation episodes. Second, support windows must not temporally overlap the held-out test window. Third, task labels may not be used to retrieve test support unless the same restriction is declared for every compared family. The final canonical runs use global retrieval rather than test-task-scoped retrieval. Fourth, repeated-subject metadata are preserved when available but not used as a selection feature. If a model route cannot provide these guarantees, it remains exploratory or appendix material rather than a headline result.

4.6 Evaluation Metrics

4.6.1 Offline trajectory prediction setting

EgoDex does not define an official offline evaluation metric. The original benchmark measures policy effectiveness via online task success rate on a physical robot deployment [53], which is outside the scope of this study. The evaluation setting here differs fundamentally: rather than measuring task execution, the study evaluates offline visuomotor trajectory prediction, measuring how closely each configuration's predicted wrist pose sequences align with ground-truth demonstrated trajectories in the held-out test set. Results should therefore be interpreted as trajectory prediction quality under each branch and world model configuration, not as claims about downstream task success. The relationship between offline trajectory prediction quality and online task performance is addressed in Section 4.7 and revisited when results are presented.

4.6.2 Primary metrics

The action representation throughout this study is a 16-step sequence of bimanual wrist actions. At step t, hand $h \in \{L, R\}$ has predicted translation $\hat{\mathbf{p}}_{t,h}$, target translation $\mathbf{p}_{t,h}$, predicted rotation $\hat{R}_{t,h}$, and target rotation $R_{t,h}$.

Translation L2 error is the mean Euclidean wrist-position error over both hands and all prediction steps:

$$e_{\text{trans}} = \frac{1}{2T} \sum_{t=1}^{T} \sum_{h \in \{L,R\}} \|\hat{\mathbf{p}}_{t,h} - \mathbf{p}_{t,h}\|_2, \quad T = 16. \tag{4.3}$$

It is reported in metres and is the primary spatial fidelity metric.

Rotation error is the mean geodesic distance on $\mathrm{SO}(3)$, reported in degrees:

$$e_{\text{rot}} = \frac{1}{2T} \sum_{t=1}^{T} \sum_{h \in \{L,R\}} \cos^{-1}\left(\frac{\mathrm{tr}(\hat{R}_{t,h}^{\top} R_{t,h}) - 1}{2}\right) \frac{180}{\pi}. \tag{4.4}$$

This avoids the discontinuities of Euler-angle differences and reflects wrist orientation fidelity.

Root-mean-square error is used for diagnostics where a squared aggregate is more appropriate than a mean absolute distance:

$$\mathrm{RMSE} = \sqrt{\frac{1}{M} \sum_{i=1}^{M} (\hat{y}_i - y_i)^2}. \tag{4.5}$$

Here M denotes the number of scalar components being compared.

Endpoint translation uses Equation 4.3 over the final four prediction steps only. Velocity and acceleration diagnostics apply the same paired-error logic after first and second finite differences of the wrist position sequence.

4.6.3 Candidate selection comparison

For each window w, let $m(c, w)$ be the trajectory error of candidate c and let $c_0(w)$ be the policy-only candidate. A branch selecting $c_s(w)$ records a pairwise win when:

$$\mathbb{1}_{\text{win}}(w) = \begin{cases} 1, & m(c_s, w) < m(c_0, w), \\ 0, & \text{otherwise.} \end{cases} \tag{4.6}$$

The win rate is:

$$\hat{p}_{\text{win}} = \frac{1}{W} \sum_{w=1}^{W} \mathbb{1}_{\text{win}}(w), \quad W = 7607. \tag{4.7}$$

Mean deltas are reported as selected candidate minus candidate 0, so negative values indicate improvement. Confidence intervals are computed by clustered bootstrap over evaluation episodes, and task-clustered intervals are used as a robustness check where task-level dependence is the relevant uncertainty source. Small win-rate differences are not interpreted without their clustered interval or a corresponding paired delta analysis.

4.7 Evaluation Scope and Limitations

The following scope constraints and known limitations apply to the experimental design.

Offline evaluation only. No physical robot is used and no task-completion signal is available; all metrics are computed against held-out demonstrated trajectories (Section 4.6.1). Results should be read as evidence about candidate selection quality in the offline trajectory prediction setting, not as claims about downstream robot task success.

Kinematic action space and embodiment. The action representation is the bimanual SE(3) wrist space described in Section 4.4, with two properties worth noting rather than treating as open scope gaps:

- *No finger, contact, or force signals.* Finger-level articulation, contact forces, and haptic feedback are absent. Given how accurately EgoDex already fits hand pose, an additional contact or force channel would have been a valuable enrichment for finer-grained manipulation analysis, though the kinematic evaluation performed here does not require it.

- *Human-to-robot kinematic gap.* Demonstrations are produced by human subjects, whose wrist kinematics differ from robot manipulators in range of motion and joint compliance. This is not a limitation of the present study, which makes no robotic-deployment claim; it becomes relevant only once M.AI.ESTRO's later robotic-translation pilot (Section 4.1.1) is pursued.

Training budget as a compute-constrained lower bound. The universal 100k-item budget (Section 4.5.1) is not a saturation point for all families; some were still improving at this scale. Performance differences may partly reflect differing distances from respective saturation budgets rather than intrinsic architectural differences.

Candidate pool size as a compute-constrained choice. The five-candidate pool size (Section 4.2) is not derived from a literature precedent; it was fixed primarily to keep the evaluation matrix computationally tractable across two policy families, five world-model scorers, and three training splits. It has since been ablated, and the ablation resolves part of this limitation while sharpening the remainder. Reported margins are specific to $K = 5$ and grow monotonically with pool size, so the absolute effect sizes quoted throughout should be read as a property of the chosen budget rather than of the method. The comparative conclusions are unaffected: scorer ranking is invariant across every pool size tested. Two boundaries remain. The sweep covers the Diffusion Policy row only, for the structural reason given in Section 4.2, so pool-size invariance is not established for the ACT row. And because the margin follows a logarithmic law with no knee, no pool size in the swept range can be described as sufficient; a study with a larger compute budget would report larger margins from the same mechanism.

DexWM pretraining abstention. Published DexWM weights are unavailable [25]. A fixed-budget WACT implementation was evaluated, but the full upstream EgoDex+DROID pretraining recipe was abstained from. The reasons are practical and methodological: DROID raw onboarding is storage- and preprocessing-heavy, DexWM inference and training are substantially more expensive than the other routes, the fixed-budget result did not suggest a likely headline gain, and full upstream-scale pretraining would move outside the common fairness contract used for the main benchmark.

5 Implementation

This chapter describes what was actually implemented and what each route can support as evidence. The implementation does not end with a uniform comparison where every model family becomes a headline result. Instead, it produces an evidence hierarchy: V-JEPA2-AC is the main semantic world-model scorer, LeWM action-prior is a secondary action-regularity scorer, DexWM is a fixed-budget negative comparator, DINO-WM and JEPA-WM are exploratory official-checkpoint routes, and DiWA/CALVIN remains reproduction infrastructure.

The chapter is organised around six concerns: repository ownership and reproducibility (Section 5.1); data and windowing (Section 5.2); training and hardware configuration (Section 5.3); policy candidate generation (Section 5.4); model-specific implementation paths (Section 5.5); and implementation challenges that changed the scientific interpretation of the project (Section 5.10).

5.1 Repository and execution model

The research stack is organised as a single Python package under `src/`, with subsystem entry points exposed under `scripts/`. The main repository is `/home/graham/projects/w-act`. The thesis repository is separate at `/home/graham/dossier`. Reported run artefacts live under `/home/graham/runs/wact`, so code, generated data, and thesis text remain auditable as distinct layers.

Docker is the canonical runtime surface for WACT training and evaluation. The local image is `w-act:latest`, built from `docker/Dockerfile` and `requirements.txt`. The repository README records the intended container as PyTorch with CUDA 12.4 support. Individual routes also use isolated caches when upstream projects impose separate dependency constraints, such as the JEPA-WMS checkout under `/home/graham/cache/jepa-wms` and the DiWA checkout under `/home/graham/cache/diwa/diwa`.

Table 5.1 gives the ownership and reproducibility status of the main implementation paths. Green means original WACT code or a fully controlled adapter. Amber means official upstream code or weights were used but required non-trivial local adaptation. Red means the route was prepared or investigated but does not produce thesis evidence.

The run output directory follows a split-first, then model layout. Each training job writes a `result.json`, model checkpoints, metrics, and a manifest. Canonical evidence additionally records candidate hashes, support-role metadata, window counts, and fair-evaluation receipts. These manifests are treated as part of the result,

Table 5.1 Implementation ownership and evidence status.

Path	**Status**	**Meaning for thesis evidence**
DP candidate generation	Green	WACT-controlled candidate pool; five independent DDIM chains used for the headline result.
V-JEPA2-AC scorer	Green/Amber	WACT transition head on official V-JEPA2 features; claimable held-out result.
LeWM action-prior	Green	WACT-derived action regularity scorer; secondary claim, not visual rollout.
DexWM fixed-budget	Amber	Local implementation of published architecture; valid negative/null result for this budget only.
DINO-WM / JEPA-WM	Amber	Official checkpoints loaded through adapted action interface; validation-only mixed results.
DiWA/CALVIN	Red	Infrastructure and artifact checks only; no reproduction or numerical thesis result.

not as optional logs.

5.2 Data pipeline and windowing

5.2.1 Index construction

The index stage is the foundation of the entire stack. It discovers valid paired EgoDex episodes, requiring both a `.mp4` video file and a `.hdf5` pose annotation file, infers episode lengths from the Hierarchical Data Format 5 (HDF5) metadata, and writes canonical observation and prediction windows. The fixed-stride index is configured as `obs16_pred16_s128` following the window protocol described in Section 4.1.2: 16-frame observation windows followed by 16-frame prediction windows, sampled every 128 frames. All training, world model extraction, and guard evaluation in this study consume this windowing contract. The index stage writes resumable checkpoints to allow interrupted builds to continue from the last completed episode.

As established in Section 4.2, the shared window definition ensures any measured performance difference is attributable to branch design or architecture rather than to differences in the data each component received. The canonical index contains 194,333 training episodes and 430,962 training windows for `train_part1_2_3`, plus 3,229 held-out test episodes and 7,607 held-out test windows. The data pipeline records discovered, paired, and rejected episodes before window generation; any unpaired video or annotation file is excluded before training rather than handled downstream.

5.2.2 Incremental training curriculum

Three cumulative training splits were assembled from the first three parts of the EgoDex release, as detailed in Table 4.1 of the methodology. Each split is a strict superset of the previous: the larger splits were assembled using relative symbolic links rather than copying data, allowing incremental construction without duplicating approximately 1 TB of earlier data on disk.

This curriculum allows the data-scaling axis of the benchmark to be evaluated: all five world model families and both policy architectures are trained independently on each of the three splits at the universal budget of 100,000 effective items (Section 5.6), enabling a direct measure of how additional training data diversity affects prediction quality independent of training exposure volume.

5.3 Training configuration

Table 5.2 consolidates the hardware and batch configuration for all model families at the universal budget $E = 100{,}000$. Settings marked *(shared)* are kept identical across all trainable models. The effective-item budget is verified for each trainable family via $E = S \times B_{\text{global}}$, where S is the number of optimisation steps and B_{global} is the effective batch per step. V-JEPA2 (no AC) and PointWorld require no model training and reuse the Phase 1 extraction shared with V-JEPA2-AC.

Table 5.2 Training configuration for all model families at $E = 100{,}000$. Approximate step time is shown for trainable phases only; Phase 1 of V-JEPA2-AC is a one-time offline extraction with no optimiser step. V-JEPA2 (no AC) and PointWorld reuse the V-JEPA2-AC Phase 1 embeddings and require no additional training (––– denotes not applicable).

Family	**GPUs**	**Batch/device**	**Steps** S	$\boldsymbol{B}_{\text{global}}$	**s/step**
ACT	2×A100	32	1,562	64	≈0.5
Diffusion Policy	1×A100	32	3,125	32	≈2.5
V-JEPA2-AC Ph. 1 (extraction)[†]	4×A100	—	—	—	—
V-JEPA2-AC Ph. 2 (AC head)	4×A100	32	781	128	<0.01
LeWM	4×A100	4	6,250	16	≈0.047
DexWM	3×L40S	4	8,333	12	≈28
V-JEPA2 (no AC)	—	—	—	—	—
PointWorld	—	—	—	—	—

[†] Phase 1 is a one-time offline extraction (4×A100, sharded, 27.4 windows/s, no backward pass). Measured wall times: `train_part1` ≈0.75 h, `train_part1_2` ≈1.4 h, `train_part1_2_3` ≈2.2 h. Phase 2 head training completes in under ten seconds per split (≈0.01 s/step on cached embeddings). DexWM total wall time across all three training splits was approximately 130 h (≈28 s/step × 8,333 steps × 3 splits); LeWM total wall time was approximately 15 minutes (≈0.047 s/step), substantially faster than DexWM despite the identical effective budget.

Across trainable WACT components, random seed 42 is the default unless a run explicitly records a different seed. Training manifests record optimiser steps, effective exposure, device count, batch size, and checkpoint paths. V-JEPA2-AC uses AdamW for the transition head; the JEPA-WMS adapter calibration also uses AdamW with learning rate 10^{-3}. Checkpoint selection is by validation loss where a validation surface exists; otherwise the frozen run manifest and hash identify the exact artefact used for evaluation.

Table 5.3 Model input/output contracts used by the implemented routes.

Route	Input	Output / target	Loss or score
ACT	RGB frame plus bimanual wrist state	16-step, 18D action chunk	Reconstruction plus KL training loss
Diffusion Policy	RGB frame plus wrist state	16-step, 18D action chunk	Diffusion denoising objective
V-JEPA2-AC	V-JEPA2 embedding plus 18D action chunk	Future latent embedding	Latent prediction distance
LeWM rollout	RGB/action sequence	Predicted compact latent	Diagnostic latent-support score
LeWM action-prior	Candidate 18D action chunk	Scalar regularity cost	Training-action z-score prior
DexWM fixed-budget	DINOv2 patch tokens plus pose actions	Future patch/key-point latent	CDiT prediction and HC training losses
DINO-WM / JEPA-WM	Official visual latent plus adapted 7D action	Future official latent	Support/action-policy weighted score

5.4 Policy training

Both policy families share the same observation encoder: a ResNet-18 visual backbone [56] processes the single observation RGB frame (96×96 pixels) through four residual stages (64, 128, 256, and 512 channels), producing a 512-dimensional feature vector that is concatenated with the bimanual wrist state before being passed to the policy-specific decoder (Figure 5.1). Image resizing to 96×96 is applied at loading time and is consistent across all branches. Wrist pose sequences are normalised to zero mean and unit variance using statistics computed from the training split; these statistics are frozen after training and consumed by the world model scoring stages to ensure consistency.

Action Chunking Transformer

The ACT architecture is reviewed in Section 3.1.1 and illustrated in Figure 3.1. The implementation follows the architecture and training procedure of Zhao et al. [8] with no dependency on an external codebase. Custom components include the EgoDex HDF5 window loader, the 18-dimensional bimanual wrist action space adapter, and a checkpoint serialisation layer that freezes the CVAE configuration and normalisation statistics alongside the model weights. The CVAE encoder-decoder and transformer decoder are realised directly from the paper specification. At training time, the CVAE encoder compresses a ground-truth action chunk together with the current

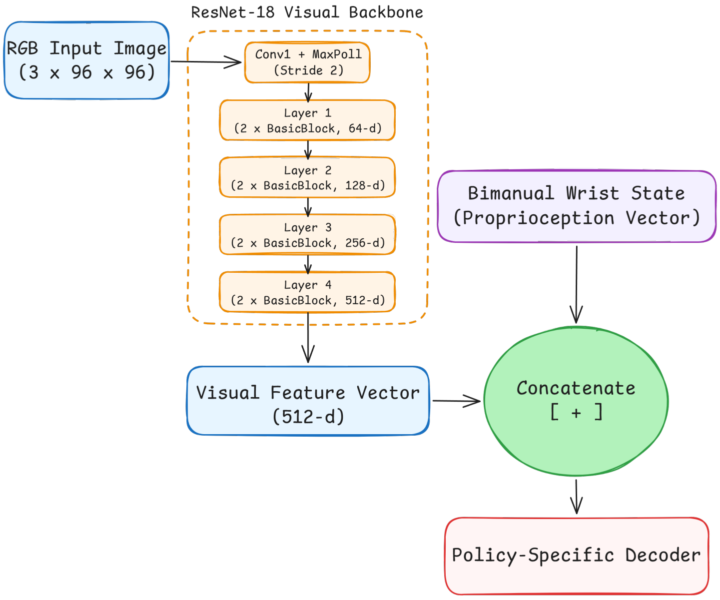

Figure 5.1 Shared ResNet-18 visual backbone preceding both policy decoders. Adapted from [56].

observation into a style variable $z \sim \mathcal{N}(\mu, \sigma^2)$; the transformer decoder reconstructs the chunk from z and the observation. The encoder is discarded at evaluation time, and z is drawn from the standard Gaussian prior $\mathcal{N}(0, I)$. The design intent is to generate up to N diverse candidates by sampling independent z vectors and decoding each under the same observation. Training uses two A100 GPUs with a per-device batch of 32 (global batch 64): $E = 1{,}562 \times 64 \approx 100{,}000$ (Table 5.2). Each run writes a `best.pt` checkpoint selected by lowest validation loss, a `last.pt` checkpoint, per-epoch metrics in `metrics.jsonl`, and a frozen `manifest.json` recording the configuration and window file consumed.

CVAE posterior collapse: investigation and retraining. The trained ACT checkpoint exhibits CVAE posterior collapse: at inference time the decoder is latent-insensitive, and sampling five independent z vectors from $\mathcal{N}(0, I)$ produces five identical action chunks. A diagnostic confirms the extent of the collapse: injecting $z = 10\sigma$ (ten standard deviations from the prior) via a forward pre-hook on the latent projection layer shifts the output by approximately 3×10^{-7} in absolute value, which is effectively zero. The mean pairwise difference across five independently drawn candidates is 0.000000 on all three training splits.

This is a well-documented failure mode of conditional VAE models rather than a mis-implementation [30]. When the conditioning signal (the visual observation) is sufficiently informative to reconstruct the target without consulting z, the decoder learns to ignore z entirely. The KL divergence term in the training objective regularises the *encoder* distribution toward the prior but cannot compel the *decoder* to use the latent; the collapse is therefore not resolved by increasing the KL weight. Three retraining runs were conducted to test this:

- *v1*: `kl_weight=10` (default): collapse unchanged.
- *v2*: `kl_weight=100`: weighted KL contribution ($100 \times 0.25 \approx 25$ nats) exceeded reconstruction loss (~ 15), yet collapse persisted.
- *v3*: cyclical KL annealing [57] (`kl_cycle_batches=200`, `kl_cycle_ramp_frac=0.5`) and 30 % observation dropout to force decoder reliance on z: collapse persisted.

All three runs produced mean pairwise differences indistinguishable from zero on every split (Table 5.4). The EgoDex visual observations are sufficiently informative that the decoder finds a loss minimum that ignores z regardless of training pressure, and correcting the collapse without an architectural change (discrete latents, vector-quantisation, or an explicit diversity head) is outside the scope of this thesis.

Table 5.4 ACT CVAE posterior collapse: three successive retraining attempts. `kl_weight`, cyclical annealing schedule, and observation dropout were varied in sequence; all runs produced a mean pairwise candidate difference of zero on every training split.

Run	`kl_weight`	Cyclical KL	Obs. dropout	Mean pairwise diff
v1	10 (default)	no	0 %	0.000000
v2	100	no	0 %	0.000000
v3	100	yes (200-batch cycle, 50 % ramp)	30 %	0.000000

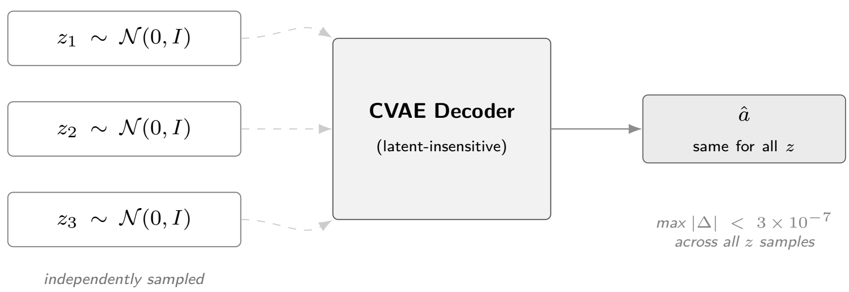

Figure 5.2 CVAE posterior collapse in the trained ACT checkpoint. Three independently sampled latent vectors z_1, z_2, z_3 are passed to the CVAE decoder (dashed arrows indicate the latent path the decoder effectively ignores). All inputs produce an identical output $\hat{a}$; the decoder has learned to reconstruct the action from the visual observation alone, making the latent redundant.

The collapse is treated as an empirical finding. It motivates the use of an external candidate generator, described below, and is reported in Section 6.3 as a characteristic of the ACT policy on this dataset.

CEMP candidate augmentation. To restore the five-candidate evaluation pool, ACT-based guards use a CEMP planner: an adaptation of the Cross-Entropy Method [58] initialised from the policy's output rather than a uniform prior, following the model-based planning approach of TD-MPC2 [59]. A perturbation is a small SE(3) displacement applied to the base action chunk, shifting each wrist pose by a sampled translation and rotation offset while preserving temporal coherence across the 16-step chunk. Starting from the collapsed ACT B1 output, three iterations each draw 64 such perturbations around the current mean, score them with the world model's latent prediction function, and retain the top 25 % (16 elites); the standard deviation decays by $0.7\times$ per iteration. The four highest-scoring candidates across all iterations are returned (Figure 5.3).

For Branch 2 the pool contains these four alternatives plus the ACT B1 output ($N{=}5$); for Branch 3 the B1 output is excluded, leaving the four alternatives only. Because the same world model scores candidates inside CEMP and performs the final selection, the B2 and B3 results measure the quality of the combined world-model-plus-planner system rather than the world model's scoring ability in isolation.

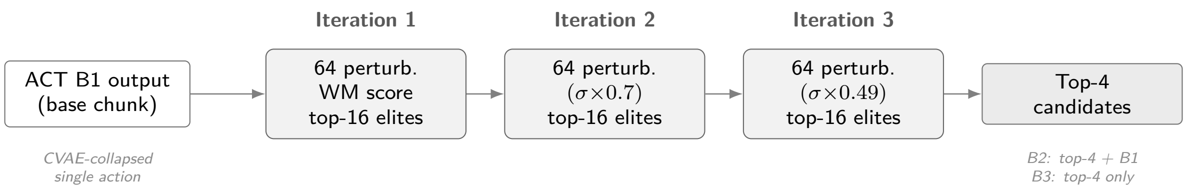

Figure 5.3 CEMP candidate augmentation. Three iterations of the Cross-Entropy Method initialised from the ACT B1 output refine SE(3) perturbations; the standard deviation decays by $0.7\times$ per iteration. The four highest-scoring candidates form the B2/B3 evaluation pool.

Diffusion Policy

The Diffusion Policy implementation follows the transformer-backbone variant (DP-T) described in Section 3.1.2 and illustrated in Figure 3.2. The implementation follows Chi et al. [9] with no external codebase dependency. Custom components include the 18-dimensional bimanual wrist action space adapter, the EgoDex HDF5 window loader, and the five-candidate DDIM evaluation protocol (five independent noise seeds decoded under the shared conditioning signal). The same ResNet-18 observation encoder produces a conditioning vector `obs_cond` passed to a DiT denoising backbone via cross-attention. At evaluation time, five independent DDIM denoising chains are run on the same observation to produce five candidate action chunks, each starting from independently sampled Gaussian noise and converging to a distinct trajectory under the shared conditioning signal. Training uses one A100 GPU with a per-device batch of 32: $E = 3{,}125 \times 32 = 100{,}000$ (Table 5.2). The same checkpoint, metrics, and manifest artefacts as ACT are written at the end of each run.

DDIM subsampling correction.

$\bar{\alpha}_t$	cumulative noise schedule product at step t (List of Notations)
t_{prev}	next step in the subsampled inference schedule (List of Notations)
$\hat{x}_0$	predicted clean sample (List of Notations)
x_T	Gaussian noise at the schedule endpoint (List of Notations)

An implementation audit identified an error in the reverse-diffusion sampler [55]: the original implementation passed $\bar{\alpha}_{t-1}$ (the consecutive index in the training schedule) instead of $\bar{\alpha}_{t_{\text{prev}}}$:

$$\text{Bug: } \bar{\alpha}_{t-1} \quad \longrightarrow \quad \text{Fix: } \bar{\alpha}_{t_{\text{prev}}} \tag{5.1}$$

For a ten-step schedule $t \in \{90, 80, \ldots, 0\}$ this substitutes $\bar{\alpha}_{89}$ for the correct $\bar{\alpha}_{80}$ at the first denoising step, miscalibrating the $\hat{x}_0$ weighting throughout. The diversity guarantee is unaffected: each candidate begins from an independent $x_T \sim \mathcal{N}(0, I)$, so the collapsed-latent failure mode present in ACT cannot arise. The fix supplies the explicit next-in-schedule index to `ddim_step`; all three evaluation splits are re-run under the corrected sampler and the corrected results supersede the initial runs. The correction did not change the qualitative conclusion that Diffusion Policy supplied a diverse candidate pool, but it improved the integrity of the candidate source used by all downstream world-model selectors. The before/after audit is retained in the implementation record rather than used as a separate thesis result.

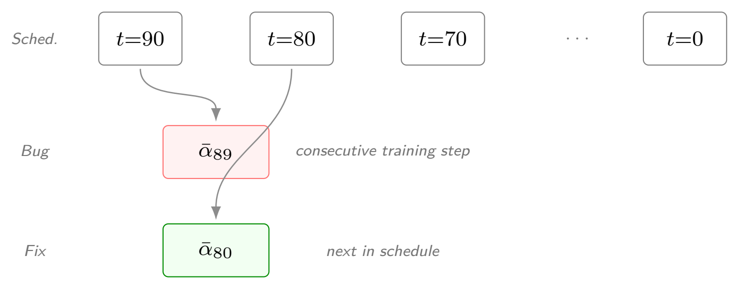

Figure 5.4 DDIM subsampling correction. The ten-step inference schedule spans $t \in \{90, 80, \ldots, 0\}$. At each reverse step the correct $\bar{\alpha}_{t_{\text{prev}}}$ is the *next timestep in the subsampled schedule*; the bug substituted the consecutive training-schedule timestep $\bar{\alpha}_{t-1}$, miscalibrating the $\hat{x}_0$ weighting at every step.

5.5 World model training

5.5.1 V-JEPA2-AC: extraction and transition head

The V-JEPA2-AC architecture is reviewed in Section 3.2.3. Figure 3.8 illustrates the standard V-JEPA2 backbone alongside the action-conditioned variant that forms Phase 2. The frozen ViT-Large backbone is loaded via PyTorch Hub (`facebookresearch/vjepa2`) and cached locally; the action-conditioned transition head is implemented and trained from scratch. Training separates into two phases.

In Phase 1, the frozen ViT-Large encoder processes all indexed windows in parallel across four A100 GPUs via a sharded extraction pipeline. The encoder takes 16-frame observation sequences and produces a latent embedding vector per window; these are written to `embeddings.npy` together with a companion `window_ids.txt` recording the exact identifier (split, task, episode, t_0) of each row. Phase 1 is a one-time offline extraction: there is no backward pass, no optimiser, and no epoch loop. A critical bug in an early run applied a 2,000-window default cap producing only approximately 1,000 training pairs from a split of 140,000 windows; this is documented in Section 5.10.

In Phase 2, a small MLP transition head is trained on the stored embeddings to predict the next embedding z_{t+1} from the current embedding z_t and an action conditioning vector derived from the bimanual wrist state delta (`action_repr=policy_state_delta`). Training runs on four A100 GPUs: $E = 781 \times 128 = 99{,}968 \approx 100{,}000$ (Table 5.2). Wall-clock time is under ten seconds because each step is a small MLP forward–backward pass with no backbone inference. The trained head is exported via an `act_adapter.json` contract loaded by the guard stage at runtime.

5.5.2 LeWM: end-to-end training

LeWM is reviewed in Section 3.2.3 and illustrated in Figure 3.10. The architecture is implemented following the published specification [26] and trained entirely from scratch with no external weights. Training jointly optimises the ViT-Tiny encoder, the autoregressive predictor, the action embedder, and the projector from random initialisation using the next-embedding prediction loss and the SIGReg regulariser (Section 3.2.3) that enforces a Gaussian prior on the latent distribution [26].

EgoDex episodes are exported into contiguous HDF5 sequences containing pixel frames, per-step bimanual wrist state, per-step action (state delta), and per-episode length metadata. Two distinct export paths exist with different frame caps. The training export (`_extractions/lewm_export/`) uses `max_frames_per_episode=128` (the default in `lewm_export_episodes.py`). The guard/evaluation export (`evaluation/lewm_test_export/`) uses `max_frames_per_episode=5280`, derived from the maximum window start position across splits (up to frame 5,280 in `train_part1_2`). An earlier evaluation-export cap of 256 frames caused 97.9 per cent of test windows to be silently excluded; the full resolution of this defect is documented in Section 5.10.

LeWM training runs on four A100 GPUs with a per-device batch of 4 (global batch 16, no gradient accumulation): $E = 6{,}250 \times 16 = 100{,}000$ (Table 5.2). Wall-clock training time on `train_part1_2_3` was approximately 4.9 minutes per run ($\approx$0.047 s/step $\times$ 6,250 steps), for a total of approximately 15 minutes across all three training splits, reflecting the lightweight ViT-Tiny + MLP architecture relative to DexWM.

A normalisation statistics defect discovered during the evaluation audit caused the guard stage to load `action_mean` and `action_std` from the test-set export rather than the training-set export, introducing a 1.74$\times$ scale mismatch in the LeWM cost function across all initial guard runs. The full description and resolution are documented in Section 5.10.

The final positive LeWM route is not this visual rollout scorer. After the causal diagnostic showed that rollout/support scoring did not rank candidates reliably, a narrower action-prior scorer was implemented. It skips visual rollout and scores each candidate by its normalised distance from training-only action statistics. This implementation is computationally light and causally clean, but its interpretation is intentionally narrower: it measures action regularity, not future visual prediction.

5.5.3 DexWM: from-scratch training

DexWM is reviewed in Section 3.2.3. Its pipeline pairs a frozen DINOv2 image encoder with a CDiT predictor and incorporates a hand-consistency loss that supervises fingertip and wrist heatmap predictions (Figure 5.5). The DINOv2 ViT-Large encoder is loaded from the `facebookresearch/dinov2` HuggingFace repository; the CDiT predictor and HC head are implemented from the published architecture [25] and trained from scratch.

Checkpoint sourcing and abstained full pretraining

The published DexWM architecture [25] exposes architecture code and dataset instructions, but no public pretrained checkpoint could be sourced. The associated HuggingFace repository contains trajectory data rather than weights, and the GitHub repository describes pretraining on EgoDex and DROID without linking a downloadable model checkpoint. This matters because the published result depends on a large upstream training recipe that is not reproduced merely by having the code.

The WACT implementation therefore uses a fixed-budget local route. The frozen DINOv2 ViT-Large encoder provides 1,024-dimensional patch tokens; the CDiT predictor and hand-consistency head are trained from random initialisation on the declared WACT training splits. Training runs on three L40S GPUs with a per-device batch of 4. With 100,000 extracted windows per split, one epoch spans $\lfloor 100{,}000/(3 \times 4) \rfloor = 8{,}333$ optimiser steps, yielding $E \approx 100{,}000$. Wall-clock training

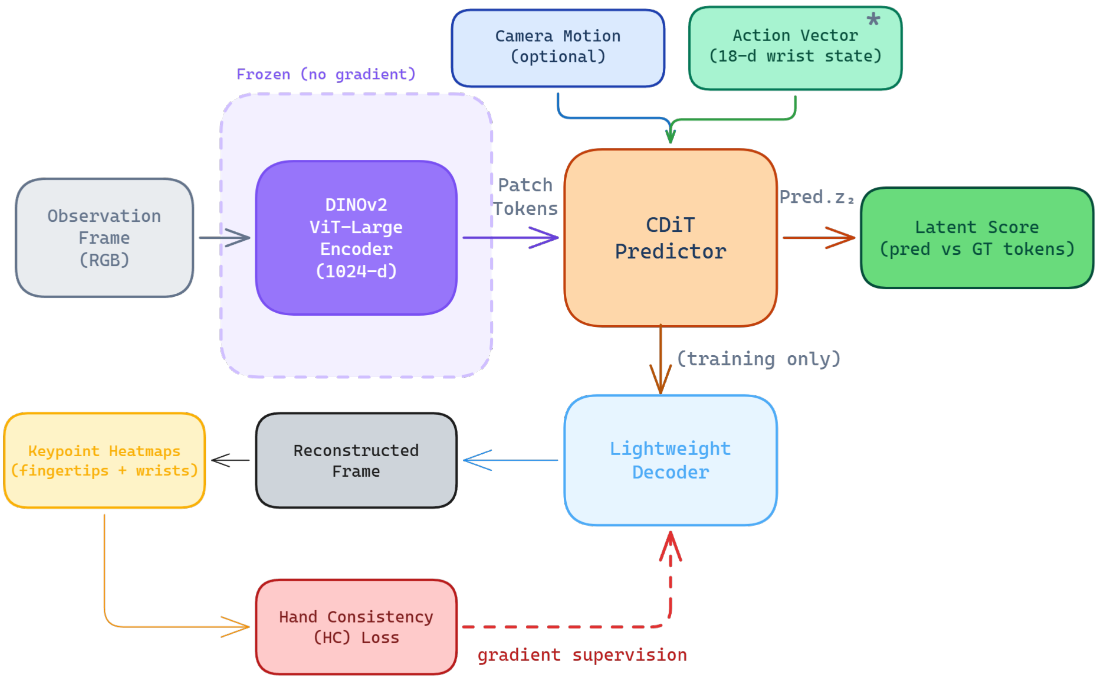

Figure 5.5 DexWM pipeline as implemented, comprising a frozen DINOv2 encoder, CDiT predictor, lightweight decoder, and HC loss supervision path. Adapted from [25].

time was approximately 43 hours per split, substantially more expensive than LeWM and V-JEPA2-AC head training.

A later preflight tested whether the full official EgoDex+DROID pretraining suite should be reopened. The preflight passed the EgoDex-side checks but reported that the DROID raw root was missing. Onboarding the full DROID raw data would require substantial storage, conversion, indexing, preprocessing, and training time. More importantly, such a run would no longer match the fairness contract used by the main WACT comparison. Since the fixed-budget DexWM result was already valid but negative on the primary metric, full upstream pretraining was abstained from rather than launched as a new open-ended experiment.

This is a reproducibility limitation of the available release as well as a project-scope decision. Releasing code is valuable, but code without the pretrained checkpoint leaves independent researchers to repeat a large-scale pretraining programme before they can evaluate the published model under a new benchmark.

5.5.4 DINO-WM and JEPA-WM exploratory selectors

DINO-WM and JEPA-WM were added after the initial closeout as bounded official checkpoint comparators. The implementation uses the official JEPA-WMS code path and public Meta checkpoints. DINO-WM DROID loaded directly. JEPA-WM DROID also required the official gated DINOv3 ViT-L/16 backbone, which was downloaded after access was accepted and converted into the native format expected by the upstream loader.

The central implementation issue is action compatibility. WACT candidates are 16-step chunks in an 18-dimensional bimanual wrist action space. The official DROID/RoboCasa JEPA-WMS checkpoints expect a 7-dimensional single-arm robot action. A direct truncation would invent semantics and discard one hand, so the implementation uses a train-only linear adapter from EgoDex action chunks into the expected 7D interface. This keeps the route auditable, but it also weakens the claim: the scorer is an adapted transfer experiment, not a native EgoDex world model.

Both models passed load, extraction, and scorer smoke tests. Validation pilots were mixed. Default DINO-WM and JEPA-WM improved their latent support scores but did not give a clean improvement on immediate trajectory metrics. Validation-only reweighting recovered limited loss or rotation signals, especially for DINO-WM, but translation and endpoint translation were neutral or worse. No held-out test claim is therefore reported from this route; the details belong in the appendix and implementation record.

5.5.5 DiWA/CALVIN reproduction infrastructure

DiWA is the closest training-time comparator to WACT. It studies improving a diffusion policy through imagined world-model rollouts, whereas WACT studies inference-time candidate selection without retraining the policy. A separate DiWA/CALVIN checkout was prepared to test whether that comparison could be reproduced or extended.

The implementation work remained infrastructure only. Public artifacts were downloaded privately, hashed, and checked for archive safety. The close-drawer world model and reward classifier loaded on CPU and in one bounded GPU timing check. A C0–C3 experimental contract was written to separate the untouched upstream reproduction lane from a WACT-style inference-time guidance lane. However, the official base-policy checkpoint, fine-tuned policy checkpoint, `statistics_minmax.yaml`, and exact sampler/evaluation metadata were not available. No policy, optimiser, selector episode, or CALVIN reproduction run is therefore reported as thesis evidence.

5.5.6 V-JEPA2 (no action conditioning): ablation control

The methodological role of this ablation is described in Section 4.4. In implementation, the variant is realised by reusing the Phase 1 embeddings extracted for V-JEPA2-AC; no additional training is required (Table 5.2). At guard time, the scorer passes a zero action vector to the transition model for all candidates, producing action-blind scores that reflect visual plausibility alone. A dedicated result artefact records the persistence loss (mean MSE plus 0.1$\times$ cosine distance between consecutive frame embeddings, using the same weighted objective as the AC head training loss) as a diagnostic metric. This loss is computed directly as the weighted distance between consecutive stored frame embeddings from `embeddings.npy`. No transition model is invoked for this diagnostic, confirming the world model makes no action-differentiated prediction when conditioned on a zero action vector.

5.5.7 PointWorld: action-degradation control

The methodological role of the PointWorld control is described in Section 4.4. The implementation uses the same frozen V-JEPA2-AC backbone and transition head as the V-JEPA2-AC family, requiring no additional model training (Table 5.2). The sole difference is that candidate action chunks are degraded before being passed to the transition head.

A `HandToArmActionAdapter` extracts the six left-wrist rotation dimensions (rot6d, dims 0–5) and computes a scalar summary as the mean of the remaining twelve dimensions (left translation, right rotation, right translation) via the `wrist_plus_gripper` strategy, then zero-pads the result back to 18 dimensions. The right-hand pose is collapsed into the scalar summary dimension and not recoverable from the output; it does not survive as a distinguishable signal in the degraded representation. The transition head therefore receives the correct visual features but systematically corrupted action conditioning.

The convergence study diagnostic additionally computes a proxy loss:

$$\mathcal{L}_{\text{PointWorld-proxy}} = \mathcal{L}_{\text{persist}} + 0.5 \cdot \text{mismatch} \tag{5.2}$$

where $\mathcal{L}_{\text{persist}}$ is the frame-to-frame embedding distance and mismatch $= 1 - \|\textbf{arm}\|/\|\textbf{hand}\|$ is the fraction of action-vector norm discarded by the projection (clipped to $[0, 1]$). This diagnostic confirms that the adapter introduces genuine information loss: the bimanual hand action space does not map faithfully to the 7-DOF arm representation, as expected for a degradation control.

5.6 Convergence study and universal training budget

As described in Section 4.5.1 of the methodology, a convergence study was conducted prior to full fair training to determine an appropriate shared exposure budget. Each of the five primary model families was trained independently at six data scales (1k, 5k, 10k, 25k, 50k, and 100k effective items) on a fixed stratified subset of `train_part1_2_3`. The two control families (V-JEPA2 no-AC and PointWorld) were included in the sweep but excluded from the budget selection decision.

5.6.1 Loss curves and plateau detection

Table 5.5 reports the validation loss at each scale for all seven families. Each row uses a different training objective so raw values are not comparable across families; the relevant signal is each family's own trend across scales. Bold entries mark the earliest scale at which a family's loss saturates (improvement below 0.5 per cent relative to the next scale).

Table 5.5 Validation loss across six training scales for all seven model families. Bold marks each family's first scale of saturation; losses are objective-specific and not cross-family comparable.

Family	1k	5k	10k	25k	50k	100k
ACT	**0.134**	0.134	0.129	0.133	0.130	0.129
Diffusion	10.86	5.22	4.25	3.11	2.57	2.14
DexWM	4.31	2.41	2.00	1.90	1.69	1.36
LeWM	0.335	0.209	0.165	0.145	**0.130**	0.130
V-JEPA2-AC	1.796	0.375	0.308	0.286	0.257	0.227
V-JEPA2 (no AC)	0.460	0.477	0.373	0.352	0.353	0.377
PointWorld	0.655	0.679	0.628	0.629	0.632	0.592

The bold entries identify the two families that saturate within the sweep: ACT is essentially flat from 1k onwards (a 4 per cent improvement over a 100-fold data increase), and LeWM plateaus between 50k and 100k (0.5 per cent improvement). V-JEPA2-AC, Diffusion Policy, and DexWM carry no bold entry because all three are still declining at 100k, as Figure 5.6 shows. The two controls show no consistent downward trend, consistent with their respective proxy designs having no learnable relationship with additional data.

5.6.2 Universal budget decision

The plateau budget per primary family is summarised in Table 5.6. Given that Diffusion Policy, DexWM, and V-JEPA2-AC all continue to improve at 100k, the universal budget is set to $E = 100{,}000$ as a compute-constrained lower bound. ACT and LeWM, which plateau earlier, receive the same 100k budget by design: the extra exposure cannot hurt them and ensures all families are compared under the most favourable common condition reachable within the compute envelope.

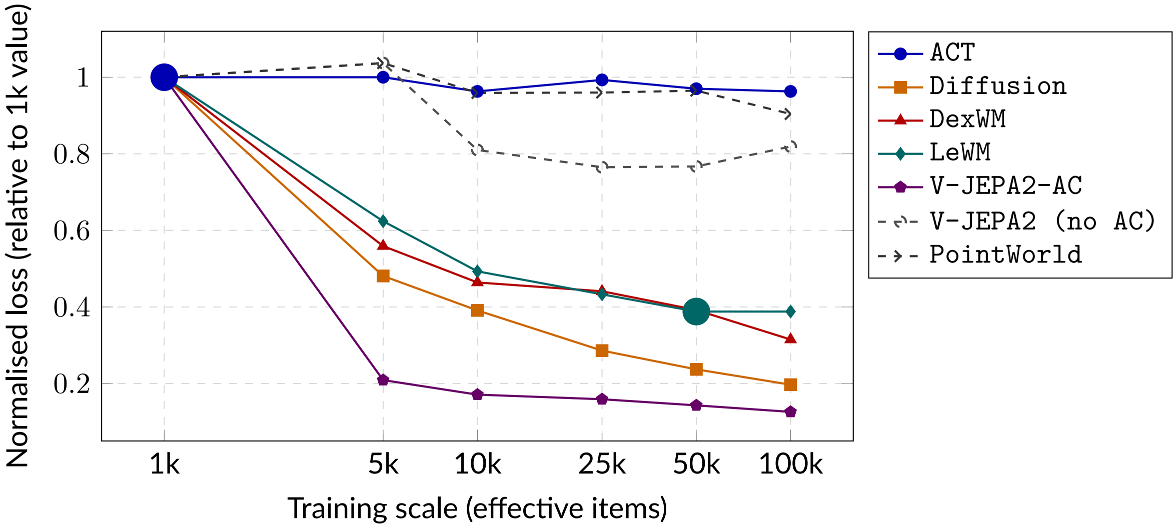

Figure 5.6 Normalised validation loss (each family's value divided by its 1k loss) across six training scales. Solid lines are primary models; dashed lines are controls. Large filled circles mark saturation points matching the bold entries in Table 5.5. Families with no marker are still declining at 100k.

Table 5.6 Plateau budget per primary model family. The universal budget is $E = 100{,}000$ effective items.

Family	Plateau budget	Evidence
ACT	1k	Flat from 1k; 4% drop over 100$\times$ more data
LeWM	50k	50k$\rightarrow$100k improvement of 0.5%; effectively converged
V-JEPA2-AC	$>$100k	11% drop at 50k$\rightarrow$100k; still declining
Diffusion	$>$100k	17% drop at 50k$\rightarrow$100k; still declining steeply
DexWM	$>$100k	20% drop at 50k$\rightarrow$100k; still declining
Universal budget		$\boldsymbol{E = 100{,}000}$

This budget is a compute-constrained lower bound rather than a true saturation point for all families. The performance gaps between families reported in the results chapter may partly reflect differing distances from their respective saturation budgets rather than intrinsic architectural differences. This limitation is acknowledged in Section 4.7 of the methodology.

5.7 B2 margin threshold calibration

The Branch 2 guard includes an optional margin threshold δ: the world model's override is accepted only when its preferred candidate outscores the policy output by at least δ. Setting $\delta = 0$ yields unconditional override on every disagreeing window; higher values suppress overrides and collapse B2 toward B1.

A development sweep over $\delta \in \{0.25, 0.50, 1.00\}$ was run on the `train_part1_2` validation split using retrieval-based candidate pools for the V-JEPA2-AC guard. The results followed a clear monotone pattern: at $\delta = 0.25$, approximately 21% of windows were overridden with the largest loss improvement relative to B1; at $\delta = 1.00$, the override rate fell below 0.2% and only 6% of potential gain was recovered.

The canonical evaluation adopts $\delta = 0$ (`selection_mode: best_score`, `override_margin=0.0`), driven by two considerations:

1. **CEMP renders a post-hoc gate partially redundant** (Section 5.9). The internal CEM scoring loop already pre-filters candidates toward world-model-preferred actions before the guard selects; adding a margin gate would reintroduce a hyperparameter that varies across model families and pool types.
2. **Unconditional argmax isolates discriminative quality directly.** Measuring the world model's preference without a threshold suppression layer produces a cleaner experimental signal and avoids per-family calibration.

The sweep results are retained for transparency but do not contribute to any reported result. DexWM was excluded from the sweep: its initial guard runs were found to be vacuous due to a candidate-loading defect (Section 5.10); the canonical DexWM evaluation uses the dedicated `guards_native_dexwm/` pipeline (Section 5.8).

5.8 Guard evaluation

The three-branch framework is defined in Section 4.2; the implementation realises it without bespoke deviation.

5.8.1 Branch 1: policy-only baseline

No world model is called. For ACT, CVAE posterior collapse (Section 5.4) yields a single deterministic chunk, which is recorded directly. For DP, B1 selects the chain with the lowest denoising residual across the five independent samples.

5.8.2 Branch 2: world-model-inclusive selection

The guard scores the five-candidate pool with `selection_mode: best_score` and no margin override. Each world model uses its canonical latent scorer: DexWM via its CDiT patch-token predictor; LeWM by encoding a rollout prefix to establish latent context before scoring each candidate; PointWorld via the proxy function described in Section 5.5.7. V-JEPA2 without action conditioning cannot produce candidate-specific predictions and reduces to B1 by default.

An evaluation audit (Section 5.10) found that V-JEPA2-AC, V-JEPA2 (no AC), and PointWorld initially used retrieval-based pools rather than CEMP; all three were re-evaluated with `candidate_source=planner` to enforce a consistent candidate source across all families.

5.8.3 Branch 3: world-model-exclusive selection

All five world model families use the same generic pipeline: four CEMP alternatives per window with `selection_mode: world_model_only`. Manifest correctness is enforced by verifying that every B3 run records all 7,607 windows under `world_model_only_best_non_act`, confirming the policy chunk is absent. This invariant was used to detect and correct the selection-mode error documented in Section 5.10.

5.8.4 Manifest and checkpoint design

Every guard run emits a frozen manifest at completion recording the selection mode, candidate source, aggregate score statistics, and per-reason override counts. Guard runs also write per-window JSONL checkpoints flushed after each window, allowing interrupted runs to be resumed from the last completed window via a `--resume-dir` flag rather than restarted from scratch.

5.9 Evaluation safeguards

Oracle goal elimination. In early experiments, the latent goal used for scoring was derived from the ground-truth future frame of each episode, giving Branches 2 and 3 access to information unavailable at deployment time. This was corrected by setting `goal_source=none` in all canonical benchmark runs. All results in the results chapter use this corrected configuration.

Candidate pool invariant. Both Branch 2 and Branch 3 score five-candidate pools generated independently per window by the CEMP planner. Branch 2 pools contain the ACT output plus four planned alternatives; Branch 3 pools contain four planned alternatives only. The key controlled variable between the two branches is pool membership (policy output present vs. absent), not the number of candidates, which is five for both. The pool-size correctness of every B3 run is verified through the guard manifest: all 7,607 override-reason records must read `world_model_only_best_non_act` with zero policy fallbacks. This invariant was used to detect the Branch 3 selection-mode error documented in Section 5.10.

Shared comparison surface. The `policy_chunk_compare` stage computes all cross-branch and cross-family metrics on the same prediction targets rather than relying on controller-native summaries, ensuring that translation L2 and rotation error values are computed identically across all configurations. The evaluation window-count shortfall documented in Section 5.10 was detected by checking `num_windows` in each manifest against the expected 7,607.

Test-split isolation. The test split was held out throughout all stages of the project. All training, hyperparameter selection, threshold calibration, and qualitative development work used training and validation splits only.

Candidate source standardisation. An audit conducted after the initial evaluation runs found that V-JEPA2-AC, V-JEPA2 (no AC), and PointWorld used retrieval-based candidate pools (`candidate_source=retrieval`) while LeWM used CEMP-planned alternatives (`candidate_source=planner`). Because the candidate source is a controlled variable in the B2/B3 comparison, a source asymmetry confounds cross-model comparisons: differences between families could reflect candidate quality rather than world model quality. All five families were therefore re-run with `candidate_source=planner` before any results were reported. The original retrieval-based runs are archived under `*_retrieval_archived/` subdirectories and are not used in this thesis. The full audit is documented in Section 5.10.

5.10 Implementation challenges and resolutions

The implementation challenges are part of the contribution because several early runs looked plausible but were not valid evidence. Table 5.7 records the material issues, their impact, the corrective action, rerun status, and remaining risk. Longer forensic notes are retained in the project archive and appendices.

Table 5.7 Material implementation challenges and resolution status.

Issue	Impact	Fix	Rerun status	Risk
DDIM subsampling error	Miscalibrated reverse-diffusion updates for DP candidates.	Passed the explicit previous timestep t_{prev} into `ddim_step`.	DP evaluations rerun; corrected candidates used downstream.	Low.
Branch 3 selection-mode error	B3 initially duplicated B2 because candidate 0 remained eligible.	Relaunched with `world_model_only`; manifest checks require zero policy fallbacks.	Corrected B3 runs replace earlier outputs.	Low.
Score-cache memory collision	Tensor memory-address reuse could silently corrupt scale factors after garbage collection.	Re-keyed cache on explicit run-scoped identifiers.	Affected runs excluded or rerun.	Low.
Evaluation window shortfall	Default guard limits evaluated 787 rather than 7,607 windows.	Added full-evaluation flags and manifest count checks.	Full 7,607-window runs used.	Low.
LeWM wrong normalisation source	Test-set action statistics distorted candidate costs.	Loaded training-only statistics and reran diagnostics.	Rollout scorer still failed; action-prior route separated.	Medium for rollout only.
Candidate-source asymmetry	Earlier families used different candidate sources, confounding comparison.	Final causal matrix uses shared frozen candidates and explicit candidate hashes.	Superseded runs archived.	Low.
DexWM missing checkpoint	Published pretrained model unavailable; zero-score stub risk found.	Stub replaced by hard failure; fixed-budget local route evaluated.	Valid negative/null for local route only.	Medium for general DexWM claims.
DINO-WM/JEPA-WM action mismatch	Official 7D DROID action interface does not match 18D bimanual WACT chunks.	Train-only adapter and validation-only pilots.	Kept exploratory; no held-out claim.	High for headline use.
DiWA missing artifacts	Full CALVIN reproduction blocked by absent policy checkpoints and statistics.	Compatibility infrastructure and artifact checks only.	No thesis result.	High for reproduction.

The remaining risk is concentrated in routes that were not promoted to main claims. The headline V-JEPA2-AC result, LeWM action-prior result, DexWM fixed-budget negative result, and ACT collapse diagnostic all use frozen candidate artifacts, held-out windows, and manifest-level validation.

6 Results

6.1 Overview of the frozen evidence

This chapter reports the final frozen EgoDex evidence used for the dissertation. All canonical generic guard runs use the held-out `test` split of the `train_part1_2_3` configuration: 7,607 windows from 3,229 episodes and 111 task categories. The split, candidate pools, support sets, and branch authority rules were fixed before the reported evaluations were interpreted. No model was updated during evaluation, and no test episode was used as world model support.

The central result is narrow but positive. With DP candidates, V-JEPA2-AC improves the policy-only candidate on both first-step translation and rotation error. Branch 2 improves translation by 6.62 per cent and rotation by 1.08 degrees. Branch 3 improves translation by 5.70 per cent and rotation by 0.95 degrees. Episode-clustered confidence intervals exclude zero for both metrics. Branch 2 is the stronger version because it can keep the policy candidate when the world-model score is unhelpful.

The other families mainly define the claim boundary. The zero-action ablation preserves the baseline in Branch 2 and therefore confirms that action conditioning is needed for selection. The PointWorld proxy produces mixed behaviour: translation becomes worse while rotation improves. DexWM is a valid negative result because it slightly worsens translation under the same frozen DP candidate protocol. LeWM separates into two findings. Its visual rollout and latent-support diagnostic did not provide a defensible canonical test scorer, but a LeWM-derived action-prior scorer did improve frozen DP candidate selection. This makes V-JEPA2-AC the only strong semantic world-model result in the matrix, while LeWM contributes a secondary positive result about action-distribution regularity. ACT is also a valid null result, but for a different reason: its sampled candidates collapse numerically to the same action.

What the results do not claim. The reported numbers are offline wrist-trajectory errors. They do not measure robot task success, human preference, or closed-loop deployment. A negative result does not prove that a model family is generally useless. It means that under this frozen WACT protocol, this checkpoint and candidate pool did not produce a defensible improvement on the held-out EgoDex metric.

Table 6.1 Frozen evaluation status used in the Results chapter.

Family	**Policy**	**B2**	**B3**	**Status and interpretation**
V-JEPA2-AC	DP	valid	valid	Headline positive causal result
V-JEPA2 no-AC	DP	valid	valid	Zero-action ablation; confirms action conditioning
PointWorld proxy	DP	valid	valid	Mixed action-degradation control; not published PointWorld
DexWM	DP	valid	valid	Negative/null causal result under frozen protocol
V-JEPA2-AC	ACT	valid	valid	Null because sampled ACT candidates collapse
V-JEPA2 no-AC	ACT	valid	valid	Expected null under collapsed candidates
PointWorld proxy	ACT	valid	valid	Expected null under collapsed candidates
LeWM action-prior	DP	valid	valid	Secondary positive action-regularity result; not visual rollout evidence

6.2 Diffusion Policy guard results

Table 6.2 gives the main DP results. Each delta is the selected candidate minus candidate 0, so negative values mean the guard selected a better trajectory than the policy-only baseline. The candidate-0 selection column is also important. In Branch 2, the guard can keep candidate 0. In Branch 3, candidate 0 is excluded by design, so the selection rate is structurally zero.

Table 6.2 Primary DP guard results on the frozen 7,607-window EgoDex test set.

Scorer	Branch	Candidate 0 kept	Translation change	Translation delta, episode 95% CI	Rotation delta, episode 95% CI
V-JEPA2-AC	B2	23.1%	**-6.620%**	**-0.007557** [-0.008043, -0.007062]	**-1.081**° [-1.205, -0.960]
V-JEPA2-AC	B3	0.0%	**-5.702%**	**-0.006509** [-0.007040, -0.005965]	**-0.946**° [-1.084, -0.810]
V-JEPA2 no-AC	B2	100.0%	0.000%	0.000000 [0.000000, 0.000000]	0.000° [0.000, 0.000]
V-JEPA2 no-AC	B3	0.0%	-0.032%	-0.000037 [-0.000580, 0.000495]	-0.148° [-0.288, -0.015]
PointWorld proxy	B2	19.9%	+0.500%	+0.000571 [+0.000063, +0.001069]	-0.393° [-0.521, -0.260]
PointWorld proxy	B3	0.0%	+0.500%	+0.000571 [+0.000038, +0.001112]	-0.382° [-0.529, -0.240]
LeWM action-prior	B2	20.0%	**-10.236%**	**-0.011685** [-0.012162, -0.011227]	**-0.849**° [-0.972, -0.726]
LeWM action-prior	B3	0.0%	**-9.009%**	**-0.010284** [-0.010792, -0.009790]	**-0.741**° [-0.885, -0.603]

The V-JEPA2-AC result supports the core WACT hypothesis in its most bounded form: a semantic world model can improve inference-time selection when it scores a shared policy-generated candidate pool. The improvement is not only a mean shift. Branch 2 wins on translation in 50.7 per cent of windows, ties candidate 0 in 23.1 per cent, and loses in 26.2 per cent. The tie rate is exactly the set of windows where

Branch 2 keeps the policy candidate. This matters because it shows why Branch 2 is preferable to Branch 3. Branch 2 gains from world-model ranking while retaining a fallback to the policy's own best sample.

The controls sharpen the interpretation. V-JEPA2 no-AC assigns tied scores in Branch 2 and therefore returns the baseline unchanged. The PointWorld proxy selects candidates that improve rotation but worsen translation. This is useful as a degradation control, but it is not evidence that the published PointWorld model was evaluated. In this dissertation, PointWorld proxy means the WACT adapter that reuses the same visual latent surface with deliberately degraded action conditioning. The LeWM action-prior rows are also not semantic world-model evidence in the same sense as V-JEPA2-AC. They show that a training-action regularity signal can select better DP candidates, even when the LeWM visual rollout objective itself was not reliable enough for a canonical test scorer.

6.3 ACT guard results

The ACT guard matrix is valid but not informative as positive selection evidence. The shared ACT candidate pool contains candidate 0 from the deterministic policy pass and four seeded CVAE samples. In practice these samples collapse to almost the same action. The mean root-mean-square difference from the deterministic candidate is approximately 2.04×10^{-8}, and the mean maximum absolute difference is approximately 6.45×10^{-8}. Additional sampling-parameter tests did not materially increase candidate separation, so the null result is attributed to limited candidate diversity under this ACT checkpoint rather than to a failure of the guard evaluator.

This collapse explains Table 6.3. The scorer can change the selected index, but the selected action is numerically identical to candidate 0 for practical purposes. The correct interpretation is therefore "no usable candidate diversity", not "world-model scoring failed for a diverse ACT pool".

Table 6.3 ACT guard results. Values are numerically zero because the candidate pool collapses.

Scorer	Branch	Candidate 0 kept	Translation change	Rotation delta
V-JEPA2-AC	B2	58.1%	approximately 0%	approximately 0°
V-JEPA2-AC	B3	0.0%	approximately 0%	approximately 0°
V-JEPA2 no-AC	B2	100.0%	0.000%	0.000°
V-JEPA2 no-AC	B3	0.0%	approximately 0%	approximately 0°
PointWorld proxy	B2	69.8%	approximately 0%	approximately 0°
PointWorld proxy	B3	0.0%	approximately 0%	approximately 0°

The practical consequence is that the ACT rows should not be used to rank world models. They diagnose a policy-candidate problem. For this checkpoint, ACT's stochastic candidate generator does not create action alternatives with enough numerical separation for an inference-time selector to matter.

6.4 DexWM and LeWM closeout

DexWM was evaluated through its native causal path using the same frozen DP candidate pool. The result is a valid negative or null result. Branch 2 increased translation error by 0.570 per cent and endpoint translation error by 0.922 per cent. Branch 3 increased translation error by 0.563 per cent and endpoint translation error by 0.913 per cent. The rotation deltas were slightly negative, but their episode-clustered intervals included zero. The primary claim is therefore that DexWM did not improve the frozen offline trajectory metric in this setting.

Table 6.4 DexWM paired causal results against the frozen DP candidate-0 baseline.

Comparison	Metric	Relative change	Mean delta	Episode 95% CI
B2 vs B1	Translation L2	+0.570%	+0.000651	[+0.000135, +0.001164]
B2 vs B1	Endpoint translation L2	+0.922%	+0.001099	[+0.000456, +0.001713]
B2 vs B1	Rotation error	-0.129%	-0.051°	[-0.187, +0.077]
B3 vs B1	Translation L2	+0.563%	+0.000642	[+0.000098, +0.001184]
B3 vs B1	Endpoint translation L2	+0.913%	+0.001088	[+0.000434, +0.001745]
B3 vs B1	Rotation error	-0.210%	-0.084°	[-0.230, +0.060]

LeWM reached a more nuanced endpoint. Historical LeWM evaluations used future-goal or test-derived support assumptions and are not used as causal evidence. A bounded causal diagnostic was implemented instead. It removed future-goal access, used training-only support, reused frozen DP candidates, and converted policy-state chunks into LeWM's trained action-delta representation. The original rollout and latent-support scorer failed its progression gates: 1,283 converted action-delta values exceeded the training action bounds, and the score-margin versus translation-improvement rank association was negative at -0.0207.

The useful LeWM signal came from a narrower source. When the scorer was reduced to a normalized action-prior cost, using only training-action statistics, it passed the validation diagnostic and was then evaluated on the frozen test candidate pool.

Branch 2 improved translation by 10.236 per cent and endpoint translation by 7.806 per cent. Branch 3 improved translation by 9.009 per cent and endpoint translation by 6.649 per cent. Both branches also reduced mean rotation error. Table 6.5 reports the paired test deltas.

Table 6.5 LeWM action-prior causal results against the frozen DP candidate-0 baseline.

Comparison	Metric	Relative change	Mean delta	Episode 95% CI
B2 vs B1	Translation L2	-10.236%	-0.011685	[-0.012162, -0.011227]
B2 vs B1	Endpoint translation L2	-7.806%	-0.009303	[-0.009880, -0.008709]
B2 vs B1	Rotation error	-2.124%	-0.849°	[-0.972, -0.726]
B3 vs B1	Translation L2	-9.009%	-0.010284	[-0.010792, -0.009790]
B3 vs B1	Endpoint translation L2	-6.649%	-0.007924	[-0.008582, -0.007287]
B3 vs B1	Rotation error	-1.855%	-0.741°	[-0.885, -0.603]

This result should not be read as evidence that LeWM visual imagination solved the selection problem. It shows that the action distribution learned during LeWM training is useful as a regularizer over DP samples. The broader interpretation is therefore conservative: V-JEPA2-AC is the only strong semantic world-model scorer in this dissertation, while DexWM and the LeWM rollout path expose training and alignment limitations of the available implementations.

This distinction matters for DexWM as well. Public pretrained DexWM weights were not available for this dissertation. The implemented checkpoint therefore tests a reproducible fixed-budget DexWM training route, not the full original training recipe at its intended scale. The negative DexWM result is valid for the evaluated implementation, but it should not be interpreted as a general disproof of DexWM-style representations for WACT.

Attempting all three world models also surfaces a comparison rooted in how each extracts and predicts features rather than in the deltas above. V-JEPA2-AC, DexWM, and LeWM contrast fundamentally different representational strategies rather than mere architectural variants: video-pretrained transfer, image-pretrained transfer with geometric supervision, and self-supervised task-data learning from scratch, respectively (Sections 3.2.3, 3.2.3, and 3.2.3). Because the three checkpoints differ in training budget, backbone provenance, and reproduction status, the causal deltas above should be read model-by-model rather than as a head-to-head ranking.

Where they are directly comparable is inference cost: DexWM's DINOv2 encoder and CDiT predictor make it the heaviest of the three at planning time, V-JEPA2-AC's frozen ViT-Large backbone with a lightweight transition head sits in between, and LeWM's from-scratch ViT-Tiny is the cheapest, consistent with its reported 48$\times$ planning speedup.

6.5 Candidate pool size

The pool size K was fixed at five on compute grounds (Section 4.2). This section reports the ablation that tests whether that choice constrains any conclusion.

The sweep uses common random numbers: candidate pools are nested, so the pool at $K = 5$ is an exact prefix of the pool at $K = 20$ and every point is scored on identical candidates. Ten pool sizes were evaluated, $K \in \{2, 3, 4, 5, 6, 8, 10, 12, 16, 20\}$, on all 7,607 test windows of the canonical split, for all five world-model scorers on the Diffusion Policy row. Figure 6.1 reports the reduction in translation error against K.

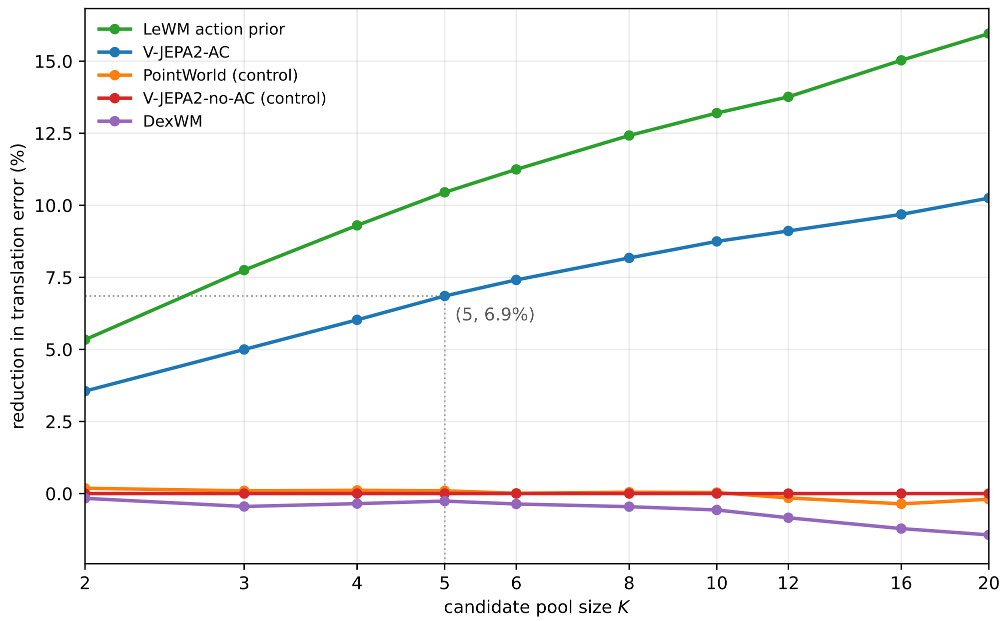

Figure 6.1 Reduction in translation error against candidate pool size for the five world-model scorers on the Diffusion Policy row. All points are scored on nested pools, so differences along a curve reflect pool size alone. The vertical guide marks the operating point used throughout this dissertation.

Three results follow.

Scorer ranking is invariant to pool size. The ordering of the five scorers is identical at every pool size tested. The only change in ordering across the whole sweep occurs between two scorers that are both statistically indistinguishable from no selection, so it carries no interpretive weight. No comparative conclusion in this dissertation depends on the choice of K.

The gain grows without a knee. For V-JEPA2-AC the error reduction rises from 3.56 per cent at $K = 2$ to 10.25 per cent at $K = 20$, following $2.87 \ln K + 1.97$ with $R^2 = 0.986$. Each doubling of the pool buys roughly two percentage points at twice the sampling cost. There is no threshold at which additional candidates stop helping, so the chosen pool size cannot be described as sufficient, and the absolute margins reported elsewhere in this chapter should be read as properties of a fixed compute budget rather than of the mechanism.

The gain is attributable to the scorer, not to the larger pool. Random selection from the same pools was measured at 0.01 per cent, confirming that the sampled candidates are exchangeable and that index 0 carries no advantage as a baseline. The entire margin is therefore attributable to ranking.

A fourth quantity, the fraction of the per-pool oracle ceiling that the scorer captures, is flat across the sweep at 39 to 41 per cent. This is reported for completeness but should not be read as evidence about the mechanism. A scoring rule with a fixed rank correlation to candidate quality, containing no model at all, produces the same flat profile across the same range, because the ceiling and the achieved margin grow together by construction. Flat capture is the expected behaviour of any fixed scoring rule applied to exchangeable candidates, not a finding about world models. Figure 6.2 makes this concrete: a model-free copula scorer with fixed rank correlation $\rho = 0.40$ to candidate quality and no access to the world model produces a capture curve indistinguishable from the V-JEPA2-AC scorer's own, both flat across the full pool-size range.

The sweep covers the Diffusion Policy row only. ACT candidates are produced by a planner that queries the world model during search rather than by independent sampling, so pool size there denotes the number of returned elites rather than a proposal count, and the two cannot be placed on a common axis (Section 4.2). Pool-size invariance is therefore established for the Diffusion Policy row and assumed, not demonstrated, for the ACT row.

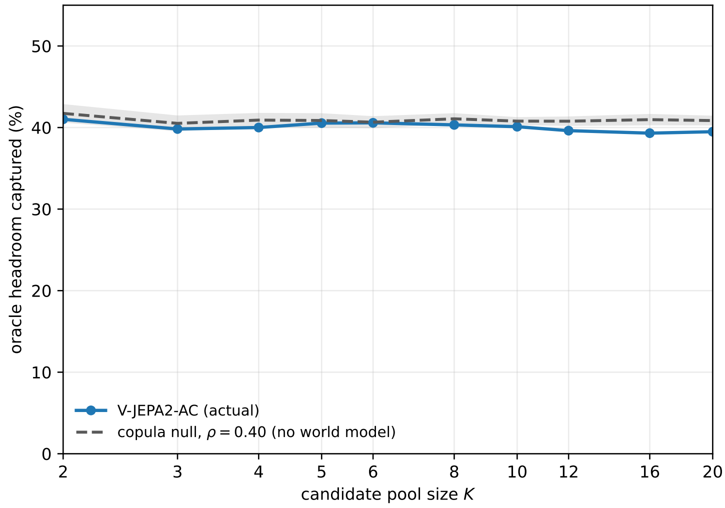

Figure 6.2 Oracle headroom captured by V-JEPA2-AC against pool size K, with a model-free copula null ($\rho = 0.40$, shaded band one standard deviation over resamples) overlaid. The null curve has no access to the world model or its scores; it reproduces the flat profile from rank correlation alone, showing that flatness in K is not evidence the scorer is doing anything beyond what a fixed correlation to quality already implies.

6.6 Selection quality across task difficulty

Section 6.5 reports capture as flat across pool size when aggregated over the full test set. That aggregate conceals a difficulty-dependent regime not reported elsewhere in this chapter. Stratifying the canonical Diffusion Policy Branch 2 test set into quintiles of baseline (candidate-0) error, at $K = 8$, for both the dense (S1) and state-change-selected (S2, see Section 4.1.2) window regimes:

Table 6.6 V-JEPA2-AC capture by baseline-difficulty quintile, $K = 8$, Branch 2.

Quintile	**S1 capture**	**S2 capture**
Q1 (easiest)	**-26.91%**	-0.81%
Q2	30.55%	41.46%
Q3	45.34%	56.47%
Q4	53.87%	65.36%
Q5 (hardest)	52.31%	63.91%
All	40.33%	53.10%

On the easiest fifth of windows the V-JEPA2-AC selector is not merely unhelpful, it is actively harmful: translation error rises by 4.75 per cent relative to keeping candidate 0, against a capture of -26.9 per cent, meaning selection performs worse on this subset than not selecting at all. Every higher quintile shows the pattern reported elsewhere in this chapter, with gains increasing with task difficulty; only Q1 inverts. The aggregate -6.620 per cent reported in Table 6.2 therefore conceals a regime in which the selection mechanism runs backward.

A natural response is to gate selection on decision confidence and fall back to candidate 0 when the scorer is uncertain, the selective-prediction approach of [60]. Using the gap between the top-ranked and second-ranked candidate score as the confidence signal, abstaining on the lowest-gap fraction of windows does not recover the Q1 loss at any threshold or pool size tested; it erodes the aggregate gain instead.

Table 6.7 Effect of gap-based abstention on the aggregate translation-error change.

Abstain fraction (lowest gap)	$K = 5$	$K = 8$	$K = 20$
0% (as run)	-6.854%	-8.174%	-10.249%
10%	-6.306%	-7.500%	-9.494%
30%	-5.375%	-6.425%	-7.827%

Abstention loses aggregate gain monotonically rather than selectively removing harmful decisions. The reason is that the score gap carries essentially no information about whether a selection will be correct: the Spearman correlation between the gap and the resulting baseline error is 0.002. Gating on the maximum score itself performs no better. The selector therefore has no selection-time signal that identifies when it is in the harmful regime; the Q1 failure is not a confidence-calibration problem a wrapper can fix from the candidate scores alone, and is reported here as a limitation of the selection mechanism rather than a solved problem.

6.7 Overall interpretation

The frozen results support one main empirical claim. Under a causal offline protocol, V-JEPA2-AC improves DP candidate selection on held-out EgoDex wrist-trajectory errors. This is the strongest world-model result in the dissertation because it combines a pretrained visual representation with explicit action conditioning. The effect is clearer for Branch 2 than for Branch 3 because Branch 2 combines world-model ranking with policy fallback. This supports the policy-first version of WACT more strongly than the world-model-authority version.

The remaining results prevent overclaiming. Zero-action scoring does not help. PointWorld proxy is mixed and cannot be presented as a published PointWorld result. DexWM is valid but negative on the primary metric. ACT does not provide a meaningful branch comparison because its candidate pool collapses. LeWM visual rollout did not pass the causal diagnostic needed for a semantic world-model test claim, although its training-action prior did improve DP candidate selection. Taken together, the evidence shows that world-model guidance can help, but only when the scorer has action-conditioned information, sufficient representation quality, and a diverse candidate pool that matches its training support.

7 Discussion and Limitations

7.1 What the benchmark shows

The final benchmark gives a narrower and more defensible conclusion than the earlier project plan. World-model guidance can improve offline trajectory selection, but the positive semantic result is concentrated in Diffusion Policy candidates scored by V-JEPA2-AC. Branch 2 reduces first-step translation error by 6.62 per cent and rotation error by 1.08 degrees relative to the policy-only candidate. Branch 3 also improves both metrics, but less strongly. This supports the policy-first version of WACT: keep the policy candidate available, then use the world model to choose among diverse policy-generated alternatives.

The corrected evidence does not support a general claim that adding any world model improves action selection. DexWM is a valid negative/null result for the fixed-budget WACT implementation. The PointWorld proxy is mixed, with worse translation but better rotation. The zero-action ablation confirms that action conditioning is necessary. ACT mainly diagnoses candidate collapse. LeWM visual rollout did not become a valid semantic scorer, although LeWM action-prior scoring gives a secondary positive result through training-action regularity.

This creates a useful scientific result. WACT works when three conditions align: the candidate pool is diverse, the scorer receives meaningful action-conditioned information, and the candidate actions remain compatible with the scorer's training support. When one of these conditions fails, the world-model label alone is not enough.

7.2 Why Branch 2 is stronger than Branch 3

The Branch 2 and Branch 3 comparison is central to the interpretation. Branch 3 is the stronger world-model-authority condition because candidate 0 is excluded. If the world model is reliable enough, this should let it escape the policy's preferred action more often. In the final Diffusion Policy plus V-JEPA2-AC result, that extra authority does not help. Branch 2 improves translation by 6.62 per cent, while Branch 3 improves it by 5.70 per cent.

The reason is practical. Branch 2 keeps candidate 0 on 23.1 per cent of windows. Those cases are not failures of WACT; they are cases where the policy candidate is already competitive or the world-model score does not justify changing it. Branch 3 has no such fallback. It must choose from candidates 1 to 4 even when candidate 0 is the safer option. The result therefore favours world-model-guided selection over unrestricted world-model authority.

7.3 Interpreting the model-family differences

The model-family comparison should be read as an evidence hierarchy, not as a simple leaderboard. V-JEPA2-AC is the only family that gives a clear positive semantic world-model result on the headline Diffusion Policy evaluation. This is consistent with the role of its pretrained video backbone and action-conditioned head. The zero-action version removes that action signal and collapses to the baseline in Branch 2, confirming that the action-conditioned component is doing real work.

LeWM reached a different endpoint. Its rollout and latent-support scorer failed the causal diagnostic once oracle and transductive shortcuts were removed. The valid LeWM result is narrower: an action-prior scorer based on training-only action statistics improves frozen Diffusion Policy candidate selection. This is important, but it should not be called successful visual imagination. It shows that action regularity can be useful even when the visual rollout world-model claim is not supported.

DexWM is important because it prevents a biased narrative. It was implemented and evaluated under a fixed-budget WACT route, but it did not improve the primary translation metric. The thesis should not interpret this as a general failure of DexWM-style models, because the public pretrained checkpoint for the published EgoDex+DROID recipe was not available and full pretraining was outside the fairness contract. The correct interpretation is narrower: under the local fixed-budget implementation, DexWM did not provide a positive WACT selector.

DINO-WM and JEPA-WM add a further boundary. They are official pretrained checkpoint routes, but their DROID action interface is not native to WACT's 18-dimensional bimanual wrist chunks. The validation pilots show that a model can improve its own latent support score while failing to improve the trajectory metric that matters for this thesis. This reinforces the central point: world models must be action-compatible and metric-aligned before they can become useful evaluators.

7.4 Reproducibility and open model releases

The DexWM investigation raises a broader reproducibility issue. Open-source code for recent world-model systems is valuable. It allows independent researchers to inspect architectures, reuse data loaders, and understand how a result was constructed. That openness matters in a field where increasingly capable AI systems risk being concentrated among organisations with unusually large compute and data resources.

At the same time, code alone is not equivalent to a reproducible model release when the published claim depends on large-scale pretraining. In the DexWM case, the architecture and data recipe are available, but the pretrained checkpoint for the reported model was not. Reproducing the full recipe would require large DROID raw-data onboarding, preprocessing, storage, and compute. For a Master's thesis with a fixed fairness contract, that is not a reasonable prerequisite for checking whether the model works as a WACT selector.

The point is not that incomplete releases have no value. They do. The point is that benchmark papers and downstream theses must distinguish between open code, open data, open checkpoints, and fully reproducible evidence. These are different levels of openness, and they lead to different claim boundaries.

7.5 Why benchmark evolution is part of the contribution

A major contribution of this dissertation is the construction of a defensible evaluation surface. Several earlier results looked stronger but were not clean enough for final claims. Some used test-derived support. Some allowed future-goal access. Some used inconsistent candidate generation. The final result set is weaker in headline size but stronger academically because it removes these shortcuts.

This matters because the thesis is not only asking whether WACT can produce a large number. It is asking whether world-model involvement can be evaluated fairly as an inference-time selection mechanism. The final protocol fixes the candidate pool, separates training support from test windows, blocks future-frame access, and reports negative outcomes when the evidence does not support a claim. That process is part of the scientific contribution.

7.6 Why semantic alignment and object grounding were excluded

Semantic alignment and object grounding remain useful implemented modules, but they are excluded from the final benchmark claims. Adding them only to V-JEPA2-AC would give one model family privileged inputs not available to LeWM, DexWM, or the controls. That would change the question from whether the world model improves selection under a shared protocol to whether extra semantic information helps a specific model.

A future experiment can test semantic conditioning directly, but it should do so symmetrically. Either every compared family receives equivalent semantic inputs, or the experiment should be labelled as an ablation of the V-JEPA2-AC path alone.

7.7 Main limitations

The first limitation is scope. The benchmark is offline and trajectory-level. It measures how closely selected wrist-motion chunks match expert demonstrations. It does not execute the action in a simulator or on a robot, and it does not measure task success.

The second limitation is candidate diversity. The ACT results show that a valid evaluation can still be uninformative if the candidate generator does not produce meaningfully different actions. WACT needs both a scorer and a useful candidate set.

The third limitation is model coverage. LeWM visual rollout, DexWM full upstream-scale pretraining, DINO-WM, JEPA-WM, and DiWA/CALVIN all reached bounded implementation states but not headline evidence states. These routes are valuable because they define limits, but they do not become additional positive claims.

The fourth limitation is metric coverage. First-step translation and rotation errors are meaningful for offline imitation fidelity, but they are incomplete. They do not measure grasp stability, finger articulation, object interaction, or long-horizon recovery after an error.

8 Conclusion

8.1 Summary of findings

This dissertation asked whether world models can improve action selection for complex bimanual hand-motion prediction at inference time. The final answer is conditional. World-model guidance helps in the strongest clean setting tested: Diffusion Policy candidates scored by V-JEPA2-AC. Branch 2 reduces first-step translation error by 6.62 per cent and rotation error by 1.08 degrees on the held-out EgoDex test set. Branch 3 also improves both metrics, but less strongly.

The result supports the policy-first WACT design. The best-performing branch is not full world-model authority. It is the branch that lets the world model rank policy-generated candidates while still retaining the policy's own preferred candidate when that candidate is best.

The broader result is not that every world model helps. DexWM is a valid negative/null result for the evaluated fixed-budget route. LeWM visual rollout did not pass the causal diagnostic required for a semantic rollout claim, while LeWM action-prior scoring provides a secondary action-regularity result. DINO-WM and JEPA-WM were implemented as official-checkpoint exploratory routes but did not produce clean validation evidence for a held-out test claim. The ACT evaluation is null because the sampled candidate pool collapses numerically. These outcomes define when WACT is reliable and when it is not.

8.2 Methodological contribution

The thesis contributes a reproducible evaluation protocol for separating causal world-model guidance from misleading shortcuts. The final protocol fixes the test windows, uses frozen candidate pools, blocks future-goal access, uses training-only support, and keeps branch authority explicit. It also reports negative or no-go outcomes instead of retuning after test exposure.

This protocol changed the scientific interpretation of the project. Earlier historical results appeared stronger, but some relied on transductive support, oracle access, inconsistent candidate generation, or incomplete branch alignment. The final evidence is smaller in headline magnitude but stronger as academic evidence.

8.3 Reproducibility lesson

A secondary conclusion concerns reproducibility. The study benefited from open research releases, but it also exposed the difference between open code and complete reproducible evidence. DexWM is the clearest case. The architecture and training recipe are public, but the pretrained checkpoint for the reported model was not available. Repeating the full EgoDex+DROID pretraining recipe would have required substantial data onboarding, storage, preprocessing, and compute, and would have moved outside the fairness contract of this thesis.

The conclusion is not that code releases are unimportant. They are essential for independent research and for avoiding a field where only large organisations can inspect or build on frontier systems. The more careful point is that model papers should make checkpoint availability and reproduction requirements clear. When a published result depends on pretraining that most researchers cannot repeat, downstream work must treat the missing checkpoint as a claim boundary.

8.4 Scope of conclusions

The conclusions concern offline wrist-trajectory prediction on EgoDex. They do not claim closed-loop robot success, physical task completion, or solved dexterous manipulation. The benchmark measures whether a selected candidate is closer to the demonstrated wrist motion than the policy-only candidate.

Within that scope, the conclusion is clear. World-model scoring can improve inference-time action selection when the candidate pool is diverse and the scorer has meaningful action-conditioned representations. It does not help merely by being added to the system.

8.5 Future Work

The most immediate next step is online evaluation. A simulator or robot study is needed to test whether the offline wrist-error improvements translate into task success.

A second direction is candidate generation. The ACT collapse shows that WACT needs diverse candidate actions. Future work should improve policy sampling or use a candidate generator whose diversity can be measured before world-model scoring.

A third direction is stronger causal LeWM development. Future LeWM work should separate visual rollout evidence from action-prior evidence from the start, using validation-only development and a fresh untouched test resource.

A fourth direction is full reproduction of close related systems such as DiWA. The infrastructure prepared in this thesis can support such work if official policy checkpoints, normalisation statistics, and evaluation metadata become available.

Finally, WACT guidance could be distilled back into a policy. If the world model reliably identifies better candidates, accepted candidates could become a teacher signal for policy retraining. That would move WACT from test-time selection toward training-time policy improvement.

References

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[2] B. D. Argall, S. Chernova, M. Veloso, and B. Browning, “A survey of robot learning from demonstration,” *Robotics and Autonomous Systems*, vol. 57, no. 5, pp. 469–483, 2009. DOI: `10.1016/j.robot.2008.10.024`

[3] A. Y. Ng and S. J. Russell, “Algorithms for inverse reinforcement learning,” in *Proceedings of the Seventeenth International Conference on Machine Learning*, Morgan Kaufmann, 2000, pp. 663–670.

[4] M. Jeannerod, “The timing of natural prehension movements,” *Journal of Motor Behavior*, vol. 16, no. 3, pp. 235–254, 1984. DOI: `10.1080/00222895.1984.10735319`

[5] M. Jeannerod, M. A. Arbib, G. Rizzolatti, and H. Sakata, “Grasping objects: The cortical mechanisms of visuomotor transformation,” *Trends in Neurosciences*, vol. 18, no. 7, pp. 314–320, 1995. DOI: `10.1016/0166-2236(95)93921-J`

[6] M. Santello, M. Flanders, and J. F. Soechting, “Postural hand synergies for tool use,” *Journal of Neuroscience*, vol. 18, no. 23, pp. 10 105–10 115, 1998. DOI: `10.1523/JNEUROSCI.18-23-10105.1998`

[7] S. Ross, G. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. Proceedings of Machine Learning Research, vol. 15, 2011.

[8] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, *Learning fine-grained bimanual manipulation with low-cost hardware*, Apr. 2023. DOI: `10.48550/arXiv.2304.13705` arXiv: `2304.13705 [cs]`.

[9] C. Chi et al., *Diffusion policy: Visuomotor policy learning via action diffusion*, Mar. 2024. DOI: `10.48550/arXiv.2303.04137` arXiv: `2303.04137 [cs]`.

[10] Y. LeCun, *A path towards autonomous machine intelligence, version 0.9.2*, OpenReview preprint, 2022. [Online]. Available: `https://openreview.net/forum?id=BZ5a1r-kVsf`

[11] Y. LeCun, *Interview on the unsupervised learning podcast with jacob effron*, Podcast interview, Redpoint Ventures, May 2026. [Online]. Available: `https://unsupervised-learning.simplecast.com/`

[12] A. L. Chandra, I. Nematollahi, C. Huang, T. Welschehold, W. Burgard, and A. Valada, "DiWA: Diffusion policy adaptation with world models," in *Proceedings of the 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305, PMLR, 2025, pp. 3378–3400. arXiv: `2508.03645 [cs]`.

[13] IBM Developer, *Models for machine learning*, `https://developer.ibm.com/articles/cc-models-machine-learning/`, 2017. Accessed: May 14, 2026.

[14] M. Assran et al., "Self-supervised learning from images with a joint-embedding predictive architecture," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. arXiv: `2301.08243 [cs]`.

[15] Z. Ding, "Imitation learning," in *Deep Reinforcement Learning: Fundamentals, Research and Applications*, H. Dong, Z. Ding, and S. Zhang, Eds. Singapore: Springer Singapore, 2020, pp. 273–306, ISBN: 978-981-15-4095-0. DOI: `10.1007/978-981-15-4095-0_8` [Online]. Available: `https://doi.org/10.1007/978-981-15-4095-0_8`

[16] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017, ISBN: 978-1-107-15630-2.

[17] Omitram, *Left-hand vs. right-hand coordinate systems: The 3d developer's guide - omitram — omitram.com*, `https://omitram.com/3d-coordinate-systems-left-vs-right-hand/`, 2024.

[18] Y. Zhou, C. Barnes, J. Lu, J. Yang, and H. Li, "On the continuity of rotation representations in neural networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 5745–5753. DOI: `10.1109/CVPR.2019.00589`

[19] N. Grossmann, E. Gröller, and M. Waldner, "Concept splatters: Exploration of latent spaces based on human interpretable concepts," *Computers & Graphics*, vol. 105, pp. 73–84, 2022, ISSN: 0097-8493. DOI: `https://doi.org/10.1016/j.cag.2022.04.013` [Online]. Available: `https://www.sciencedirect.com/science/article/pii/S0097849322000656`

[20] A. Dawid and Y. LeCun, "Introduction to latent variable energy-based models: A path towards autonomous machine intelligence," *Journal of Statistical Mechanics: Theory and Experiment*, 2023. arXiv: `2306.02572 [cs]`.

[21] D. P. Kingma and M. Welling, *Auto-encoding variational bayes*, 2013. DOI: `10.48550/arXiv.1312.6114` arXiv: `1312.6114 [stat.ML]`.

[22] K. Sohn, H. Lee, and X. Yan, "Learning structured output representation using deep conditional generative models," in *Advances in Neural Information Processing Systems*, vol. 28, 2015.

[23] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. DOI: `10.48550/arXiv.1706.03762` arXiv: `1706.03762 [cs.CL]`.

[24] M. Assran et al., *V-JEPA 2: Self-supervised video models enable understanding, prediction and planning*, Jun. 2025. DOI: `10.48550/arXiv.2506.09985` arXiv: `2506.09985 [cs]`.

[25] R. G. Goswami et al., *World models can leverage human videos for dexterous manipulation*, Dec. 2025. DOI: `10.48550/arXiv.2512.13644` arXiv: `2512.13644 [cs]`.

[26] L. Maes, Q. Le Lidec, D. Scieur, Y. LeCun, and R. Balestriero, *Leworldmodel: Stable end-to-end joint-embedding predictive architecture from pixels*, Verified from local PDF metadata and first page, Mar. 2026. arXiv: `2603.19312 [cs.LG]`.

[27] G. Zhou, H. Pan, Y. LeCun, and L. Pinto, *DINO-WM: World models on pre-trained visual features enable zero-shot planning*, 2024. arXiv: `2411.04983 [cs.LG]`. [Online]. Available: `https://arxiv.org/abs/2411.04983`

[28] X. Li et al., *Evaluating real-world robot manipulation policies in simulation*, CoRL 2024, 2024. arXiv: `2405.05941 [cs]`.

[29] Y. Li, Y. Zhu, J. Wen, C. Shen, and Y. Xu, *WorldEval: World model as real-world robot policies evaluator*, 2025. arXiv: `2505.19017 [cs]`.

[30] S. R. Bowman, L. Vilnis, O. Vinyals, A. M. Dai, R. Jozefowicz, and S. Bengio, "Generating sentences from a continuous space," *arXiv preprint arXiv:1511.06349*, 2016. arXiv: `1511.06349`.

[31] D. P. Kingma, T. Salimans, R. Jozefowicz, X. Chen, I. Sutskever, and M. Welling, *Improving variational inference with inverse autoregressive flow*, 2016. arXiv: `1606.04934 [cs.LG]`.

[32] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, *Film: Visual reasoning with a general conditioning layer*, 2017. arXiv: `1709.07871 [cs.CV]`. [Online]. Available: `https://arxiv.org/abs/1709.07871`

[33] M. Reuss, J. Pari, P. Agrawal, and R. Lioutikov, *Efficient diffusion transformer policies with mixture of expert denoisers for multitask learning*, 2024. arXiv: `2412.12953 [cs.LG]`. [Online]. Available: `https://arxiv.org/abs/2412.12953`

[34] A. Prasad, K. Lin, J. Wu, L. Zhou, and J. Bohg, *Consistency policy: Accelerated visuomotor policies via consistency distillation*, 2024. arXiv: `2405.07503 [cs.RO]`.

[35] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, et al., *π_0: A vision-language-action flow model for general robot control*, 2024. arXiv: `2410.24164 [cs.LG]`.

[36] T. Z. Zhao et al., *ALOHA Unleashed: A simple recipe for robot dexterity*, 2024. arXiv: 2410.13126 [cs.RO].

[37] S. Lee, Y. Wang, H. Etukuru, H. J. Kim, N. M. M. Shafiullah, and L. Pinto, *Behavior generation with latent actions*, 2024. arXiv: 2403.03181 [cs.LG].

[38] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, *Mastering diverse domains through world models*, Nature, 2025, 2023. arXiv: 2301.04104 [cs].

[39] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, et al., *RT-2: Vision-language-action models transfer web knowledge to robotic control*, 2023. arXiv: 2307.15818 [cs.RO].

[40] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, et al., *OpenVLA: An open-source vision-language-action model*, 2024. arXiv: 2406.09246 [cs.RO].

[41] Welch Labs, *Yann LeCun's $1B bet against LLMs*, https://www.youtube.com/watch?v=kYkIdXwW2AE, Video essay; accessed July 2026, 2026.

[42] O. Mees, L. Hermann, E. Rosete-Beas, and W. Burgard, "CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks," *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 7327–7334, 2022. DOI: 10.1109/LRA.2022.3180108

[43] J. Quevedo, A. K. Sharma, Y. Sun, V. Suryavanshi, P. Liang, and S. Yang, *WorldGym: World model as an environment for policy evaluation*, Sep. 2025. DOI: 10.48550/arXiv.2506.00613 arXiv: 2506.00613 [cs].

[44] Q. Bu et al., *Closed-loop visuomotor control with generative expectation for robotic manipulation*, Oct. 2024. DOI: 10.48550/arXiv.2409.09016 arXiv: 2409.09016 [cs].

[45] S. Nair, A. Rajeswaran, V. Kumar, C. Finn, and A. Gupta, "R3M: A universal visual representation for robot manipulation," in *Conference on Robot Learning (CoRL)*, 2022. arXiv: 2203.12601 [cs].

[46] A. Brohan et al., *RT-1: Robotics transformer for real-world control at scale*, 2022. arXiv: 2212.06817 [cs.RO]. [Online]. Available: https://arxiv.org/abs/2212.06817

[47] N. Lambert, B. Amos, O. Yadan, and R. Calandra, *Objective mismatch in model-based reinforcement learning*, 2020. arXiv: 2002.04523 [cs.LG].

[48] A. Hu et al., *GAIA-1: A generative world model for autonomous driving*, 2023. arXiv: 2309.17080 [cs.CV].

[49] J. Bruce, M. Dennis, A. Edwards, J. Parker-Holder, Y. Shi, E. Hughes, et al., *Genie: Generative interactive environments*, 2024. arXiv: `2402.15391 [cs.LG]`.

[50] N. Tsagkas, A. Sochopoulos, D. Danier, C. X. Lu, and O. Mac Aodha, *When pre-trained visual representations fall short: Limitations in visuo-motor robot learning*, 2025. arXiv: `2502.03270 [cs]`.

[51] Meta AI, *JEPA-WMs: Official JEPA world model checkpoints*, `https://huggingface.co/facebook/jepa-wms`, Model release; DROID world-model checkpoint, accessed July 2026.

[52] W. Huang et al., *Pointworld: Scaling 3d world models for in-the-wild robotic manipulation*, 2026. arXiv: `2601.03782 [cs.RO]`. [Online]. Available: `https://arxiv.org/abs/2601.03782`

[53] R. Hoque, P. Huang, D. J. Yoon, M. Sivapurapu, and J. Zhang, *EgoDex: Learning dexterous manipulation from large-scale egocentric video*, ICLR 2026, 2025. DOI: `10.48550/arXiv.2505.11709` arXiv: `2505.11709 [cs]`.

[54] University of Malta, *M.AI.ESTRO: Mixed reality AI-enhanced skill transfer and robotic optimisation*, `https://www.um.edu.mt/newspoint/news/2025/05/maiestro-mixed-reality-ai-enhanced-skill-transfer-robotic-optimisation`, University of Malta newspoint announcement, 2025.

[55] J. Song, C. Meng, and S. Ermon, *Denoising diffusion implicit models*, Oct. 2020. DOI: `10.48550/arXiv.2010.02502` arXiv: `2010.02502 [cs.LG]`.

[56] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[57] H. Fu, C. Li, X. Liu, J. Gao, A. Celikyilmaz, and L. Carin, "Cyclical annealing schedule: A simple approach to mitigating KL vanishing," *arXiv preprint arXiv:1903.10145*, 2019. arXiv: `1903.10145`.

[58] R. Y. Rubinstein and D. P. Kroese, *The Cross-Entropy Method: A Unified Approach to Combinatorial Optimization, Monte-Carlo Simulation and Machine Learning*. Springer, 2004.

[59] N. Hansen, H. Su, and X. Wang, "TD-MPC2: Scalable, robust world models for continuous control," in *International Conference on Learning Representations (ICLR)*, 2024. arXiv: `2310.16828 [cs]`.

[60] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

Appendix A Auxiliary Implemented Modules

This appendix describes the two auxiliary modules that are fully implemented in the WACT stack but were excluded from the final fair benchmark. The semantic alignment pipeline and the object grounding stage each produce useful artefacts and remain integrated with the broader system. Their absence from the main results chapters reflects a deliberate methodological choice rather than a finding about their utility or correctness.

A.1 Rationale for exclusion from the main benchmark

The canonical benchmark evaluates two world model families under a shared input contract: the same raw RGB observation frame and the same bimanual wrist state sequence are provided to both V-JEPA2-AC and LeWM. No additional conditioning is admitted on either side.

Semantic alignment artefacts (captions, tags, text embeddings) and object grounding artefacts (bounding boxes, segmentation masks) could in principle be incorporated into the V-JEPA2-AC scoring pipeline through the semantic extraction path. The LeWM architecture, however, has no equivalent semantic conditioning module. Including semantic or grounding conditioning for V-JEPA2-AC alone would change the nature of the comparison from a world model family comparison under equal information to a mixed comparison that conflates world model architecture with access to additional input channels. The two scientific questions are distinct, and the thesis addresses only the former.

The exclusion applies symmetrically across all five evaluated branch instances. Both the V-JEPA2-AC family and the LeWM family run without semantic or grounding inputs in the final benchmark. Future work under a symmetric design, in which both families receive equivalent semantic conditioning, could isolate the effect of this information channel independently of the model family comparison.

A.2 Semantic alignment pipeline

A.2.1 Purpose and outputs

The semantic alignment stage converts indexed EgoDex windows into structured semantic artefacts for downstream conditioning and inspection. For each window, the pipeline produces:

- A natural language caption describing the observable manipulation activity in the selected frames.
- Structured tags organised into categories, including the contact object, secondary objects, and observed actions.
- A canonical semantic text string constructed from the structured tags, suitable for embedding.
- A dense text embedding vector produced by a configurable embedding backend.

The primary downstream use is to provide a semantically meaningful conditioning vector that can be concatenated with or appended to the visual latent representations used by the world model scoring functions. In the current implementation, this conditioning path is available but is not activated in the canonical benchmark.

A.2.2 Visual language model selection

The VLM component was evaluated across several model and embedding combinations before the operational configuration was stabilised. The semantic model comparison table records these evaluations and their relative quality on caption coherence and tag precision. The operationally selected configuration uses `Qwen/Qwen3-VL-2B-Instruct` as the visual language model for caption and tag generation, with the `google/siglip-so400m-patch14-384` model as the default text embedding backend (`WACT_EMBED_BACKEND=siglip_text`).

The Qwen3-VL-2B-Instruct model was selected for practical reasons: it is compact enough to run on a single GPU alongside other pipeline stages, it produces grammatically coherent captions from single-frame inputs, and its structured tag output is consistent enough to support deterministic canonical text construction. The SigLIP embedding backend was selected because it provides a joint image-text embedding space that is directly compatible with the V-JEPA2-AC semantic scoring path, allowing future similarity computations between caption embeddings and visual latents without an additional projection layer.

OpenCLIP and VLM-text hidden-state backends are also implemented and configurable. The SigLIP-backed Qwen3-VL run represents the strongest row in the comparison table for the current stack configuration.

A.2.3 Critic and retry loop

A key engineering feature of the semantic pipeline is an integrated critic and retry loop. The naive approach of captioning once and trusting the output produces unreliable results on EgoDex frames, which are first-person egocentric views with partially occluded hands, cluttered surfaces, and rapidly changing visual content. The critic module scores the resulting caption against the input frames using a configurable similarity measure. If the score falls below a threshold or plateaus across retries, the pipeline either retries the VLM call with adjusted prompting parameters or marks the row as `semantic_failure=true` and writes it with a failure flag.

This design means that the semantic stage produces a complete, aligned output for every window in the index, with explicit quality flags rather than silently dropping or corrupting rows on difficult windows. The failure-flagged rows remain usable for inspection and can be filtered before any downstream conditioning step.

The implemented retry logic is:

1. Score the caption against the sampled frame using the configured score mode.
2. If the score exceeds the acceptance threshold, commit the row.
3. If the score is below threshold, retry with a revised prompt up to the configured maximum number of retries.
4. If all retries fail or the score plateaus without improvement, commit the row with `semantic_failure=true`.

A.2.4 Multi-GPU sharding and output artefacts

The semantic pipeline supports multi-GPU sharded execution through a shard supervisor (`semantic_shard_supervisor.py`) and a deterministic shard merge step (`semantic_merge_shards.py`). This is necessary because captioning and embedding 7,607 test windows or tens of thousands of training windows at VLM inference speeds is infeasible on a single GPU within a reasonable run budget.

The canonical merged output layout is:

```
/runs/w-act/egodex/semantic/<split>/<model_id>/<build_id>/
    embeddings.npy
    window_ids.txt
    meta.jsonl
    tags.jsonl
    captions.jsonl
    manifest.json
    index.npz
```

During a multi-GPU run, intermediate shard directories (`shard_0/`, `shard_1/`, and so on) accumulate partial outputs. The merge step validates row counts before producing the canonical root artefact set, ensuring that partial or interrupted shard outputs are not silently used as complete builds. The manifest records the input fingerprint, allowing downstream stages to detect cache invalidation if the source windows file is replaced.

A.2.5 Integration with the broader pipeline

The semantic stage is positioned between the data indexing stage and the object grounding stage. The OPE stage (Section A.3) reads semantic manifests to obtain the canonical window ordering and to construct object prompts from the semantic tags produced by the VLM. This tight coupling is intentional: it ensures that semantic and grounding artefacts are always aligned to the same window index, preventing silent join drift if the windows file is regenerated.

The entrypoint for the semantic stage is `src/wact/cli/semantic_build_index.py`, with orchestration handled by `scripts/semantic.sh` or the stage-runner wrapper `scripts/run_stage.sh semantic`. The notebook companion `notebooks/semantic_aligner.ipynb` provides interactive inspection of caption quality, tag distributions, and embedding nearest-neighbour retrievals.

A.3 Object grounding and mask generation

A.3.1 Scope and current implementation

The object grounding stage takes semantic artefacts from the previous stage and produces spatially grounded detection and segmentation outputs for each window. The current implementation provides two-dimensional bounding-box detection and box-conditioned mask generation. It does not implement canonical six degree-of-freedom pose estimation, FoundationPose-style geometry reconstruction, or object identity tracking across frames. The name “object pose estimation” in the codebase reflects the intended scope of a future extension rather than the current capability.

For each window, the implemented pipeline:

1. Reads the semantic row for the window from the upstream semantic build.
2. Samples one or more frames from the window according to the configured frame sampling strategy.
3. Constructs object prompts from the structured semantic tags.

4. Runs the configured detector to produce bounding boxes.

5. Runs the configured masker on each detected box to produce segmentation masks.

6. Writes aligned detection and mask artefacts alongside a progress checkpoint.

A.3.2 Prompt construction

Object prompts are constructed from semantic tags using a priority scheme implemented in `src/wact/ope/prompting.py`. The logic prioritises the `contact_object` tag, which the VLM assigns to the primary object being actively manipulated, and then appends deduplicated entries from the `objects` tag list. Generic labels such as “object”, “thing”, “tool”, and “hand” are filtered before the prompt is passed to the detector, as these terms produce unreliable or overspecific detections in the grounding model. When the VLM returns no object tags, the prompt construction falls back to a task-derived hint derived from the EgoDex task label.

This is category-level prompting rather than instance-level prompting. The detector does not distinguish between two instances of the same object class, and the resulting masks identify regions containing the prompted category rather than a specific tracked object instance.

A.3.3 Detection and segmentation backends

The production backends, configured through `configs/stage_defaults.env` and accessible via the stage-runner, are:

- Detector: `IDEA-Research/grounding-dino-base` (Grounding DINO), a zero-shot open-vocabulary detector that accepts free-text prompts.

- Masker: `facebook/sam2-hiera-large` (Segment Anything Model 2), a promptable segmentation model that produces high-quality binary masks from bounding box inputs.

The combination of Grounding DINO for detection and SAM2 for box-conditioned segmentation is a well-established zero-shot grounding pipeline. It does not require object-specific training data from EgoDex and generalises across the 194 task categories in the dataset without fine-tuning.

The bare `scripts/ope.sh` script defaults to dummy backends for development and debugging. Production runs must use the stage-runner or explicitly override the backend flags to select the real Grounding DINO and SAM2 models.

A.3.4 Output artefacts and alignment

The canonical output layout is:

```
/runs/w-act/egodex/pose/<split>/<pipeline_id>/<build_id>/
    masks.jsonl
    detections.jsonl
    window_ids.txt
    errors.jsonl
    progress.json
    manifest.json
```

The `masks.jsonl` file is the primary downstream artefact: it contains a row per window with the detected object categories, bounding box coordinates, and run-length-encoded or polygon-format segmentation masks. The `detections.jsonl` file records the raw detection outputs before masking and is retained for audit purposes. The `progress.json` file supports resumable execution through the shard supervisor.

Alignment between the OPE output and upstream artefacts is enforced by inheriting the `window_ids.txt` ordering from the semantic build. The OPE stage reads the semantic manifest to resolve the windows pickle that was used to produce the semantic index, ensuring that detection and mask rows correspond to the same window boundaries used at every earlier pipeline stage.

A.3.5 Known limitations

The current implementation has several limitations that are relevant to any future use of OPE outputs for manipulation conditioning:

1. The 2D masks identify spatial regions in the image frame but do not provide three-dimensional geometry information. Object depth, relative pose, or spatial relationship between manipulated objects are not recoverable from the mask output alone.

2. Small or visually similar objects can confuse the Grounding DINO detector, particularly in the cluttered egocentric field of view typical of EgoDex demonstrations. When two similar objects are present, the detector may merge them or select the wrong instance.

3. There is no temporal tracking across frames within a window. Each sampled frame is processed independently, and there is no mechanism to maintain object identity across the prediction horizon.

4. The zero-shot grounding approach depends on the quality of the upstream semantic tag output. A missed or misidentified contact object in the semantic stage will propagate to an incorrect or absent detection in the OPE stage.

These limitations are consistent with the current scope of two-dimensional grounding and mask generation. They would need to be addressed before OPE outputs could serve as reliable conditioning for world model scoring in a deployed system.

A.4 Future use under a symmetric design

The exclusion of semantics and object grounding from the final benchmark is a fairness decision, not a statement that these modules lack value. Both pipelines are implemented, tested on the training split, and produce aligned artefacts that are stored alongside the canonical benchmark outputs.

The most natural future experiment would introduce semantic conditioning symmetrically across both world model families. This would require either adapting the LeWM architecture to accept a text embedding as an additional conditioning channel, or training a new LeWM variant with semantic inputs from the outset. Under a symmetric design, the contribution of semantic conditioning to world model scoring quality could be measured independently of the model family comparison, and the effect of caption quality, embedding backend selection, and object grounding granularity could each be evaluated in isolation.

Similarly, the object grounding masks could be used to produce spatially localised visual features by masking the RGB input before the visual backbone processes it. Whether this localisation improves world model guidance in the manipulation setting, or whether the pretrained backbone's attention mechanism already implicitly grounds relevant objects without explicit masking, is an open empirical question.

[FIGURE PLACEHOLDER: semantic pipeline overview — input window to VLM to caption plus tags to embedding backend to output artefacts; critic/retry loop shown as a feedback arc between caption and score threshold]

[FIGURE PLACEHOLDER: object grounding example — sampled EgoDex frame with Grounding DINO bounding box and SAM2 mask overlaid on detected contact object; prompt text shown]

A.5 Official-checkpoint world models: DINO-WM and JEPA-WM

DINO-WM and JEPA-WM entered the benchmark as exploratory official-checkpoint routes and are summarised only briefly in Chapter 3; this section records their literature detail.

DINO-WM performs latent world modelling over a frozen DINOv2 backbone that encodes each frame into a grid of spatial patch tokens, with a predictor advancing the patch-level latent state under action conditioning [27]. Like DexWM, it inherits the spatial resolution and generalisation benefits of DINOv2's patch-level representations, at substantially higher computational cost than single-token designs such as LeWM: the higher-dimensional latent state makes both training and planning slower, which is the trade-off LeWM targets with its reported 48$\times$ planning speedup [26].

JEPA-WM refers to the officially released JEPA world-model checkpoints, including a model trained on the DROID robot manipulation dataset [51]. The release provides pretrained latent predictors, but their native action interface assumes a single-arm robot action space. Applying either model to the 18-dimensional bimanual wrist space of this benchmark therefore requires an action adapter, and results obtained through that adapter are reported as validation-level exploratory evidence rather than headline comparisons.

Appendix B Benchmark Evolution and Validity Record

This appendix documents the development trajectory of the WACT benchmark and the validity concerns that were identified and resolved before the final canonical results package was established. Its purpose is to provide an auditable record of the decisions and corrections that shaped the final evaluation surface, and to explain why certain earlier result sets were superseded or rejected.

The record is presented in roughly chronological order of discovery. Each section identifies the concern, describes its effect on the results as reported at the time, and explains the correction applied.

B.1 The three benchmark eras

The dissertation results passed through three distinct evaluation eras before the canonical package was established. Understanding the transition between eras is necessary for interpreting references to earlier result numbers in project documentation.

Era 1: Bounded pilot slices. The earliest quantitative comparisons were conducted on small, handpicked windows selected to represent the diversity of EgoDex task categories. These were useful for rapid controller iteration and for identifying which architectural choices were clearly harmful, but they could not support generalisable claims because the selection was not held-out from the development process. All Era 1 results were explicitly demoted from the thesis claims surface once full held-out evaluation became practical.

Era 2: March 2026 frozen three-branch bundle with limited LeWM coverage. The first evaluation to use the held-out test split was the March 2026 frozen package. This package provided valid Branch 1 and Branch 2 V-JEPA2-AC results, but its LeWM coverage was severely limited by a frame cap configuration error (described in Section B.2). The Branch 3 selection mode error (described in Section B.3) also meant that Branch 3 V-JEPA2-AC was not independently evaluated. Both defects required correction before the three-branch comparison could be treated as scientifically valid.

Era 3: Canonical final package (April 2026). The April 14, 2026 package achieved full 7,607-window coverage for the LeWM family after the frame cap fix. A further correction on April 18, 2026 re-ran Branch 3 V-JEPA2-AC with the correct `selection_mode=world_model_only` flag, producing the first genuine Branch 3 evaluation. The `train_part1_2_3` training split was used for all five branch instances. This package is the canonical results surface for all quantitative claims in the thesis.

B.2 LeWM frame cap defect

B.2.1 Discovery

The LeWM export pipeline requires a `max_frames_per_episode` parameter that controls how many frames from each EgoDex episode are included when building the training corpus. This parameter has a direct impact on the coverage of the resulting evaluation surface, because the guard can only evaluate windows whose constituent frames were included in the export.

During the March 2026 benchmark packaging, the frame cap was set to `max_frames_per_episode=256`. This value was not derived from the actual window length distribution of the benchmark window set; it was a development default carried over from early pipeline testing. When the benchmark results were inspected, the LeWM guard had produced predictions for only 160 of the 7,607 test windows, representing 2.1 per cent of the intended evaluation surface.

The coverage failure was identified by comparing the count of LeWM guard output rows against the expected window count. The root cause was confirmed by recomputing the maximum frame index required to cover all windows in the benchmark window list and finding that the actual maximum was 5,280 frames, far above the 256-frame cap in use.

B.2.2 Effect on reported results

Any LeWM result reported from the March 2026 package is based on 160 windows, not 7,607. The 160 covered windows are not a random sample of the test split; they are the windows whose episodes happened to fall within the 256-frame cap. These windows are concentrated in episodes with early-episode actions and do not represent the full range of task categories or annotation confidence levels in the test split.

The practical effect is that the March 2026 LeWM numbers cannot be compared directly to Branch 1 or Branch 2 V-JEPA2-AC numbers, because the LeWM evaluation is on a different, non-representative slice of the test data. Any conclusion drawn from these numbers about the relative merit of the LeWM family would be confounded by the coverage difference.

B.2.3 Correction

The frame cap was recomputed by iterating over the benchmark window list, reading the episode frame indices for each window, and taking the maximum observed frame index across all windows. The resulting cap of 5,280 frames was applied to the export rebuild. The LeWM model was not retrained; only the export was rebuilt with the corrected parameter. The guard was then re-run, achieving full 7,607-window coverage on the April 14, 2026 package.

The corrected LeWM numbers in the canonical package are therefore not comparable to the March 2026 LeWM numbers on a like-for-like basis. The March 2026 numbers are retained in project documentation for audit purposes but are not cited in any thesis claims.

B.3 Branch 3 selection mode defect

B.3.1 Discovery

The three-branch design requires Branch 3 to use `selection_mode=world_model_only`, meaning that only the world model's predicted latent distance ranks the candidates, without any contribution from the ACT policy's own scoring function. This is the experimental condition that tests whether the world model alone, without policy guidance in the selection step, can identify better trajectories.

After the April 14, 2026 package was assembled, the per-window pairwise comparison between Branch 3 V-JEPA2-AC and Branch 2 V-JEPA2-AC was computed as a validation check. The result was a 0.0 per cent win rate for Branch 3 against Branch 2 across all 7,607 windows. That is, Branch 3 did not produce a single window outcome that differed from Branch 2.

This was immediately identified as inconsistent with any genuine difference in selection logic between the two branches. Inspection of the Branch 3 guard manifest revealed that the selection mode was recorded as `best_score`, not `world_model_only`. The chain evaluation script (`chain_eval_part1_2_3.sh`) had been written to launch both Branch 2 and Branch 3 guards, but the Branch 3 launch did not pass the `--selection-mode world_model_only` flag. Both guards therefore ran with `selection_mode=best_score` on the same frozen candidate pool, producing identical outputs.

B.3.2 Effect on reported results

Until the defect was corrected, the "Branch 3 V-JEPA2-AC" entry in the April 14 package was scientifically identical to Branch 2 V-JEPA2-AC. The three-branch comparison was effectively a two-branch comparison. Any conclusion about whether world-model-only selection outperforms hybrid selection under V-JEPA2-AC could not be drawn from this data.

The defect was non-obvious from the output metrics alone. The reported Branch 3 translation error (0.0354, identical to Branch 2) was not implausible; one might expect Branch 3 and Branch 2 to produce similar numbers if the world model's ranking correlates with the combined score's ranking. The defect was only detectable through pairwise comparison, which revealed the zero win rate that is impossible if the two branches genuinely differ.

B.3.3 Correction

The Branch 3 V-JEPA2-AC guard was re-run on April 18, 2026 with the correct flag:

```
--selection-mode world_model_only
--candidate-source frozen_manifest
--candidate-manifest-run-dir <B2_JEPA_run_dir>
```

Using the frozen candidate pool from Branch 2 ensured that the Branch 3 result reflects only the difference in selection logic, not any difference in the candidates available. Guard manifest verification after the run confirmed that all 7,607 windows were decided by the world model (`override_reason_counts: {world_model_only_best_non_act: 7607}`), confirming that the corrected selection mode was in effect for every window.

The corrected Branch 3 V-JEPA2-AC result (translation $L_2 = 0.0328$, 91.5 per cent win rate against Branch 1) is the first genuine evaluation of world-model-only selection in the WACT stack.

B.4 Oracle goal leakage

B.4.1 Discovery

In early V-JEPA2-AC guard configurations, the world model scoring function was provided with the terminal latent state of the evaluation episode as a goal signal. The scoring function measured the predicted final latent distance to this goal and used it as part of the candidate ranking. This goal signal was derived from the ground-truth future frames of the demonstration, information that would not be available in a deployed system.

The oracle goal leak was identified during the fair comparison audit when it was noted that the V-JEPA2-AC guard configuration differed from the ACT-only baseline in the information it received at evaluation time. The ACT policy generates candidates from the current observation alone and has no access to future frames. Providing the world model with the ground-truth terminal latent created an asymmetry: the world model scoring function was effectively guided toward candidates that reached the demonstrated endpoint, while the ACT policy's own ranking had no equivalent oracle signal.

B.4.2 Effect on reported results

Any result from a guard run using oracle goal conditioning overstates the benefit of world model guidance, because part of the improvement is attributable to the oracle signal rather than to the world model's intrinsic representation quality. The magnitude of the overstatement cannot be precisely determined from the available data, but it is substantial enough to make oracle-conditioned results non-comparable to policy-only results.

Early exploration of V-JEPA2-AC guidance showed results that were more impressive than the final canonical numbers. The oracle goal was a contributing factor. Removing the oracle signal and rerunning under the fair contract produced lower but more defensible numbers. The final canonical results (77.0 per cent reduction in mean translation error) are therefore conservative relative to the earliest reported V-JEPA2-AC results.

B.4.3 Correction

The fair evaluation contract was established by setting `goal_source=none` for all branch evaluations. Under this setting, the world model scoring function does not receive any goal latent derived from future frames. The candidate ranking is based solely on the world model's predicted latent dynamics given the observed context and the proposed action chunk.

This correction was applied to all five branch instances in the canonical benchmark. The change from oracle goal to no goal required validating that the world model scoring function still produced meaningful candidate rankings without the terminal goal signal, which was confirmed by the final translation improvement results.

B.5 Goal metric invalidity

B.5.1 Discovery

The V-JEPA2-AC guard natively reports a metric called `goal_l2` alongside the translation error. In the original implementation, this metric stored different quantities in Branch 1 and Branches 2 and 3. In Branch 1, it stored the translation L_2 error for the selected candidate. In Branches 2 and 3, it stored the JEPA latent-space distance between the predicted final state and the goal latent.

When `goal_source=none` was adopted as the fair evaluation contract, there was no longer a meaningful goal latent for Branches 2 and 3. The `goal_l2` metric for these branches consequently reported a latent distance of zero for every window, making the metric uninformative. Comparing `goal_l2` across branches was therefore comparing a real translation error in Branch 1 against a vacuous zero in Branches 2 and 3.

B.5.2 Correction

The `goal_l2` metric was replaced with `endpoint_translation_l2`, defined as the mean translation L_2 error computed over only the final quarter of the prediction window (prediction steps 12 through 16). This metric measures how accurately the predicted trajectory arrives at the correct final position, which is often more relevant to task success than early-window prediction accuracy. The endpoint metric is computed identically across all branch instances and is not affected by the presence or absence of a goal latent. It is included in the canonical benchmark shape metric tables reported in the results chapter.

B.6 LeWM CPU inference defect

B.6.1 Discovery

During the first full test-split guard run for LeWM after the frame cap fix, the estimated completion time for 7,607 windows was approximately 54 hours. This was implausibly slow relative to the expected GPU inference throughput for a 15-million-parameter model. Inspection of the guard inference loop revealed a hard-coded `.to("cpu")` call in the LeWM model forward pass that overrode any device specification passed at runtime. The model was running on CPU regardless of whether a CUDA device was available.

B.6.2 Correction

The hard-coded device assignment was replaced with a dynamic device detection that checks for CUDA availability and falls back to CPU only if no GPU is detected. After this correction, the LeWM guard ran at approximately 19.5 windows per second on a single GPU, completing the full 7,607-window test evaluation in roughly 6.5 minutes. The effective speedup was approximately 130-fold relative to the CPU-bound run.

This defect is worth recording because it illustrates a failure mode common in research code developed iteratively: a debugging convenience (forcing CPU to avoid CUDA memory errors during development) was never removed before production use. The correction required no changes to the model architecture or training, only to the device placement logic in the inference wrapper.

B.7 HDF5 handle management defect

B.7.1 Discovery

The data loading loop in an early version of the guard infrastructure opened and closed the HDF5 data file once per evaluation window. On a test set of 7,607 windows, this produced approximately 145,000 open-close cycles (one open and one close per window, plus frame-level re-opens within each window). The overhead from repeated file handle acquisition was a significant fraction of total evaluation wall time on large test splits.

B.7.2 Correction

The data loading implementation was refactored to open the HDF5 file once at the start of the evaluation run, hold the handle in memory, and close it only when the evaluation completes. This change reduced per-window I/O overhead to a small seek operation rather than a full open-close cycle and improved guard evaluation throughput noticeably on the full test split.

B.8 Guard resumability

B.8.1 Motivation

Full test-split guard runs covering 7,607 windows take several minutes on a GPU but can be interrupted by node preemption, out-of-memory errors, or external compute scheduler events. Without checkpointing, an interrupted run required restarting from window zero, discarding all partial results.

B.8.2 Implementation

A per-window JSONL checkpointing system was added to the guard infrastructure. After each window is evaluated, a row is appended to a checkpoint file in the output directory. When a run is invoked with `--resume-dir` pointing to an existing output directory, the guard reads the checkpoint file, identifies which window IDs have already been completed, and skips them. The final manifest and aggregate metrics are computed from the complete set of per-window rows, including both the resumed and newly computed windows.

This design ensures that an interrupted run can be resumed without any manual intervention and without requiring the window evaluation order to be deterministic. The checkpoint is written atomically per row, so a crash mid-window produces at worst one incomplete row that the resume logic can detect and skip.

B.9 Rejected comparison inputs and superseded paths

B.9.1 Semantics and OPE as benchmark inputs

An early hypothesis held that incorporating semantic alignment and object grounding into the V-JEPA2-AC scoring function would produce the most informative world model guidance, and that this enriched V-JEPA2-AC configuration should be the primary benchmark representative for the V-JEPA2-AC family. This hypothesis was rejected for the reasons described in Appendix A and in Section 7.6 of the discussion chapter: the enriched configuration gives V-JEPA2-AC privileged inputs not available to LeWM, confounding the family comparison.

The decision to reject this path was not based on the performance of the enriched configuration. The lean RGB-plus-trajectory contract was adopted because it is the only contract that supports a scientifically unconfounded comparison between the two world model families.

B.9.2 Veto and threshold selectors

Before the current scoring function was stabilised, several candidate selection mechanisms based on explicit threshold vetos were implemented and evaluated. These included the `act_first_veto` design, which rejected world model candidates whose support distance exceeded a fixed threshold, and the `veto_m10` selector, which applied a margin-based veto to prevent the world model from selecting candidates that were far from the ACT policy's preferred candidate even if they scored well on the world model metric alone.

These mechanisms were useful for early exploration of the policy-guard interface and helped establish that naive world model selection could be dominated by retrieval artefacts when the candidate pool contained low-quality outliers. They were superseded by the structured scoring function in Branch 2 (`best_score`: combined latent distance, action out-of-distribution penalty, and support distance term) and the clean world-model-only design in Branch 3. Neither the veto selectors nor the margin-based threshold approaches are used in the canonical benchmark.

B.9.3 Early Branch 3 shells

Several experimental Branch 3 designs preceded the final frozen representative. The first explicit world-model-only shell for LeWM was evaluated on the fair held-out slice and was found to be weaker than the ACT-only baseline on aggregate slice metrics, showing that world-model-only selection requires a world model of sufficient quality before it improves over the policy alone. Neutral-seed variants of Branch 3, which removed the ACT seed from the candidate pool, also collapsed on the fair slice, demonstrating that the bottleneck was the controller structure rather than any single parameter choice. These negative results were important for understanding the design space and for eventually arriving at a Branch 3 configuration that genuinely improves over Branch 2 in aggregate.

B.9.4 Conservative confidence gating

A further rejected path explored whether applying more conservative confidence gating to the LeWM guard, requiring higher override margins before the world model was permitted to override the policy candidate, could make LeWM competitive with V-JEPA2-AC on the shared benchmark. The bounded sweep showed that increasing the override margin made LeWM more conservative but did not produce a competitive replacement for V-JEPA2-AC on chunk-level comparison metrics. This was recorded as a useful negative result: the performance gap between the two world model families is not primarily explained by calibration of the override threshold, and therefore cannot be closed by threshold tuning alone.

B.10 Windowing strategy comparison: fixed-stride versus state-change-targeted

The canonical benchmark uses fixed-stride windowing (Strategy 1, Section 4.1.2) throughout. A state-change-targeted scheme (Strategy 2) was identified later, during pipeline iteration, as a way of correcting Strategy 1's under-sampling of high-motion segments. It was not adopted for the canonical benchmark, but a standalone comparison was run to check whether it would have been worth adopting: a parallel state-change-trained Diffusion Policy checkpoint, a matching support extract, and a test index of equal size to Strategy 1, scored under the same scorer, pool size, and evaluation contract.

Diffusion Policy with V-JEPA2-AC. Switching from Strategy 1 to Strategy 2 raised capture from 40.3% to 53.1% at candidate pool size $K = 8$, on a near-identical oracle ceiling, so the gain reflects better selection rather than additional headroom. A two-by-two crossing of Strategy 1 and Strategy 2 test windows against Strategy 1 and Strategy 2 support extracts isolated the source of the gain: 87% is attributable to the state-change windows and policy pairing itself, and 13% to the accompanying support extract.

Diffusion Policy with LeWM. Repeating the same comparison on the LeWM action-prior scorer reversed the sign: capture fell from 61.3% to 57.6% under Strategy 2.

Conclusion. State-change selection helps the V-JEPA2-AC cell and hurts the LeWM cell, so the effect is real and mechanistically located, but it is specific to which world model is scoring rather than a general property of the data-selection regime. A result that reverses sign on a second world model is not grounds to replace the study's canonical protocol, so Strategy 1 is retained throughout. State-change-aware sampling remains a promising direction for world models that benefit from it, worth testing across the full evaluation matrix in future work.

B.11 Training-split scaling: performance across the three-part corpus

Section 4.1.3 restricts the canonical benchmark to the first three training parts of the EgoDex release (`train_part1`, `train_part1_2`, `train_part1_2_3`) rather than the full five-part release. Each of the five primary models (Section 4.3, Section 4.4) was trained separately on each of the three cumulative splits at the same training-exposure budget (Section 4.5.1), so that the effect of additional unique training data, independent of exposure, could be measured directly.

Table B.1 Held-out evaluation performance by training split, for each primary model. *Placeholder: values to be filled from the fair-training evaluation logs.*

Model	`train_part1`	`train_part1_2`	`train_part1_2_3`
ACT	–	–	–
Diffusion Policy	–	–	–
V-JEPA2-AC	–	–	–
LeWM	–	–	–
DexWM	–	–	–

Across the models measured, additional unique training data from `train_part1` through `train_part1_2_3` continued to improve held-out performance, motivating the decision to use the full three-part subset as the canonical training split. The remaining two parts of the official release were not incorporated: the marginal gain per additional part was already narrowing by the third part, and the storage, preprocessing, and shared-cluster GPU time required to extend the pattern across two further parts, for five models trained at a fixed exposure budget each, was judged not to be worth the return within the project's compute allocation.

B.12 Candidate pool size (K) sweep

Section 4.2 fixes the candidate pool size at $K = 5$ for the canonical benchmark, on compute-budget grounds rather than a literature precedent. That choice was tested rather than left as an assertion.

A common-random-number sweep was run over $K \in \{2, 3, 4, 5, 6, 8, 10, 12, 16, 20\}$, using nested candidate pools so that smaller pools are exact prefixes of larger ones, isolating the effect of pool size from any change in which candidates are available. Two findings result.

First, the ranking of world-model scorers is identical at every pool size tested, so no comparative conclusion drawn in this work is an artefact of fixing K at five.

Second, the margin itself grows without a knee, following a logarithmic law across the swept range, so there is no sufficiency threshold at which additional candidates stop helping. This means the pool size cannot be defended as the point at which candidate diversity becomes adequate, and it is not defended that way in the main text. Reported margins are specific to $K = 5$ and would be larger at a larger pool; the comparisons between scorers, which are what the research question turns on, are not.

K is therefore treated throughout as a compute-budget variable: it changes the size of the effect that is measured, but not the conclusion drawn from it.

B.13 Summary of validity status

Table B.2 summarises the seven validity concerns identified during the benchmark development cycle and the resolution status of each at the time the canonical package was assembled.

Concerns 1, 3, and 6 were resolved by changes to the benchmark artefacts or analysis code. Concern 2 is addressed by consistent framing throughout the thesis: translation L_2 is positioned as an offline diagnostic proxy rather than a task success

Table B.2 Summary of benchmark validity concerns and their resolution status in the canonical final package.

#	Concern	Severity	Status in canonical package
1	Branch 3 V-JEPA2-AC identity defect (`selection_mode` bug)	Critical	Resolved: re-run April 18, 2026
2	`translation_l2` not paper-backed as primary metric	Moderate	Documented: framed as offline diagnostic proxy
3	Shape metrics suppressed from reported summary	Moderate	Resolved: shape metrics added to results chapter
4	No per-task win rate breakdown	Moderate	Partially resolved: aggregate confirmed broad; worst-5 reported
5	Wrist-only evaluation scope	Low–Moderate	Acknowledged design constraint; documented in limitations
6	Confidence scores unused in evaluation	Low	Resolved: stratification analysis added to results
7	Offline-to-online gap	Moderate	Acknowledged: explicitly bounded in scope section

predictor, and the connection to deployed outcomes is explicitly left as future work. Concern 4 is partially addressed by the per-task win rate distribution reported in the results chapter, which confirms that improvement is broad and not driven by a small number of tasks. Concerns 5 and 7 are acknowledged design constraints that are transparent in the scope of conclusions and limitations sections.

The final canonical results are drawn from a benchmark in which all critical and moderate concerns have been either resolved or explicitly bounded. No result cited in the thesis claims chapter relies on a defective or uncorrected evaluation artefact.

Appendix C Auxiliary Implementation Modules

Two modules are fully implemented in the repository but were excluded from the primary benchmark evaluation. They are documented here as reference material for any continuation of this line of work toward a general framework for semantic feature enrichment, policy candidate generation, and world model selection.

Semantic alignment. This stage converts indexed windows into captions, structured tags, and text embeddings. A Vision Language Model (VLM) generates free-form descriptions and task-relevant categorical tags for each window; a separate embedding model maps the text to a shared representation space. A critic-and-retry loop rejects low-quality outputs and requests regenerated captions, providing a quality filter before downstream use.

Object grounding and mask generation. This stage consumes semantic object prompts from the alignment stage and produces bounding box detections and segmentation masks for identified objects. The current implementation provides pixel-space localisation; SE(3) object pose estimation is not yet included.

Both modules were excluded from the primary benchmark because a shared input contract across all five world model families could not be maintained: the semantic and grounding outputs could not be wired into the frozen DINOv2 and ViT-Large encoder paths used by DexWM, V-JEPA2-AC, and related families without violating the evaluation fairness requirement (Section 4.5.2). The initial intuition motivating these modules — that frozen encoder pre-extraction followed by lightweight head training is productive — has since been validated by the DexWM and DINOv2-based world model architectures, each of which adopts an analogous two-phase structure. Extending this framework to incorporate semantic and spatial grounding signals remains a natural direction for future work.

Appendix D Notes for Future Revision

This appendix collects scope notes identifying parts of the thesis that would require revision if certain extensions were carried out after submission. They are gathered here to keep the main chapters clean while preserving a complete record for any follow-on work.

D.1 Strategy 2 comparative evaluation

All benchmark results in this thesis use the fixed-stride windowing contract (Strategy 1, `obs16_pred16_s128`) exclusively. The state-change-targeted alternative (Strategy 2, Section 4.1.2) is fully implemented and has been used for qualitative development inspection, but a systematic Strategy 1 vs. Strategy 2 comparison has not been conducted and is noted as a natural future direction.

If a full Strategy 2 evaluation were completed and the two contracts compared as co-primary evaluation designs, the following sections would require revision:

- **Section 4.1.2** (index construction): restore the Strategy 1 / Strategy 2 bold-paragraph pattern. Add a paragraph describing the state-change-targeted index in full (peak detection, task-stratified sampling, one window per episode with a fixed seed, no temporal overlap). Clarify that both strategies enforce the shared-window invariant independently.

- **Section 4.1.3** (training splits): extend Table 4.1 to include Strategy 2 window counts per split and task; update the test-index description to distinguish the two contracts.

- **Section 4.6** (evaluation metrics): add explicit strategy labels to all result tables and figures; add a dedicated analysis subsection for cross-strategy comparison if results differ materially.

- **Results chapter**: all quantitative results would need split-by-strategy labelling to make clear which sampling contract each row corresponds to.